

ШИНЖЛЭХ УХААН ТЕХНОЛОГИЙН ИХ
СУРГУУЛЬ
МЭДЭЭЛЭЛ, ХОЛБООНЫ ТЕХНОЛОГИЙН СУРГУУЛЬ



Эрдэнэсүх Есүдэй

**AI шийдэл бүхий хөгжим
боловсруулалтын вэбсайтын
дизайн ба хөгжүүлэлт**

БАКАЛАВРЫН ДИПЛОМЫН АЖИЛ

Улаанбаатар хот

ШИНЖЛЭХ УХААН ТЕХНОЛОГИЙН ИХ
СУРГУУЛЬ
МЭДЭЭЛЭЛ, ХОЛБООНЫ ТЕХНОЛОГИЙН СУРГУУЛЬ

Компьютерын ухааны тэнхим

АI шийдэл бүхий хөгжим
боловсруулалтын вэбсайтын
дизайн ба хөгжүүлэлт

Мэргэжлийн индекс: 061301000000002306

Мэргэжил: Компьютерын ухаан

Удирдагч: Проф. PhD. Б.Туяацэцэг

Зөвлөгч: Доктор(PhD) Ж.Оргил, Магистр Э.Батцэцэг

Гүйцэтгэгч: Э.Есүдэй

Улаанбаатар хот

2026 он 5 сар

Батлав. Компьютерын ухааны тэнхимын эрхлэгч:

..... /Дэд проф. PhD. А.Хүдэр/

Удирдагч:

..... /Проф. PhD. Б.Туяацэцэг/

ТӨГСӨЛТИЙН АЖЛЫН ТӨЛӨВЛӨГӨӨ

Сэдэв: "AI шийдэл бүхий хөгжим боловсруулалтын
ВЭБСАЙТЫН ДИЗАЙН БА ХӨГЖҮҮЛЭЛТ"

№	Ажлын бүлэг, хэсгийн нэр	Эзлэх хувь	Дуусах хугацаа
1	Удиртгал	5%	2016-10-05
2	Онолын хэсэг	25%	2016-10-30
3	Судалгааны хэсэг	30%	2016-11-15
4	Төслийн хэсэг	30%	2016-12-20
5	Дүгнэлт	10%	2016-12-25

Төлөвлөгөөг боловсруулсан оюутан: /Э.Есүдэй/

ТӨГСӨЛТИЙН АЖЛЫН ЯВЦ

№	Хийж гүйцэтгэсэн ажил	Биелсэн хугацаа	Удирдагчийн гарын үсэг
1	Удиртгал	2016-10-05	
2	Онолын хэсэг	2016-10-30	
3	Судалгааны хэсэг	2016-11-15	
4	Төслийн хэсэг	2016-12-20	
5	Дүгнэлт	2016-12-25	

Ажлын товч дүгнэлт

.....

.....

.....

.....

.....

.....

.....

Удирдагч: /Проф. PhD. Б.Туяацэцэг/

ЗӨВШӨӨРӨЛ

Оюутан Э.Есүдэйгын бичсэн төгсөлтийн ажлыг УШК-д хамгаалуулахаар тодорхойлов.

Салбарын эрхлэгч: /Дэд проф. PhD. А.Хүдэр/

ШИНЖЛЭХ УХААН ТЕХНОЛОГИЙН ИХ СУРГУУЛЬ

Мэдээлэл, Холбооны Технологийн Сургууль

ШҮҮМЖИЙН ХУУДАС

Компьютерын ухааны тэнхимын салбарын төгсөх курсийн оюутан Э.Есүдэйн "AI шийдэл бүхий хөгжим боловсруулалтын вэбсайтын дизайн ба хөгжүүлэлт" сэдэвт төгсөлтийн ажлын шүүмж.

1. Төслөөр дэвшүүлсэн асуудал, үүнтэй холбоотой онолын материал уншиж судалсан байдал. Энэ талаар хүмүүсийн хийсэн судалгаа, түүний үр дүнг уншиж тусгасан эсэх.

.....

.....

.....

.....

.....

.....

.....

2. Төслийн ерөнхий агуулга. Шийдсэн зүйлүүд, хүрсэн үр дүн. Өөрийн санааг гарган, харьцуулалт хийн, дүгнэж байгаа чадвар.

.....

.....

.....

.....

.....

.....

.....

3. Эмх цэгцтэй, стандарт хангасан өөрөөр хэлбэл диплом бичих шаардлагуудыг биелүүлсэн эсэх. Төсөлд анзаарагдсан алдаанууд, зөв бичгийн болон өгүүлбэр зүйн гэх мэт /Хуудас дугаарлагдаагүй, зураг хүснэгтийн дугаар болон тайлбар байхгүй, шрифт хольсон, хувилсан зүйл ихээр оруулсан/.

.....
.....
.....
.....
.....
.....
.....

4. Төслөөр орхигдуулсан болон дутуу болсон зүйлүүд. Цаашид анхаарах хэрэгтэй зүйлүүд.

.....
.....
.....
.....
.....
.....
.....

5. Төслийн талаар онцолж тэмдэглэх зүйлүүд.

.....
.....
.....
.....
.....
.....
.....

6. Ерөнхий оноо. (5 оноо)

.....

Шүүмж бичсэн: /—/

Ажлын газар:

Хаяг (Утас)

Зохиогч эрхийн хамгаалал

Миний бие Э.Есүдэй, "AI шийдэл бүхий хөгжим боловсруулалтын вэбсайтын дизайн ба хөгжүүлэлт" сэдэвт энэ ажил нь минийх бөгөөд дараахыг нотолж байна. Үүнд:

- Горилогч энэ ажлыг тус сургуулиас боловсролын зэрэг авахаар бүхэлд нь буюу голлон хийсэн болно.
- Энэ ажлын аль нэг хэсгийг тус сургуульд эсвэл өөр байгууллагад боловсролын зэрэг, мэргэшил авахаар өмнө нь илгээсэн бол түүнийгээ тодорхой заасан болно.
- Бусад хүмүүсийн хэвлүүлсэн ажлаас зөвлөгөө авсан бол түүнийгээ үндэслэсэн болно.
- Бусад хүмүүсийн ажлаас ишлэл хийхдээ гол эх үүсвэрийг нь заасан болно.
- Миний ажилд тусалсан голлох бүх эх үүсвэрт талархаж байна.
- Ажлыг бусадтай хамтарсан бол алийг нь бусад хүмүүс хийсэн болохыг тодорхой заасан болно.

Гарын үсэг: _____

Огноо: _____

“Thanks to my solid academic training, today I can write hundreds of words on virtually any topic without possessing a shred of information, which is how I got a good job in journalism.”

Dave Barry

ШИНЖЛЭХ УХААН ТЕХНОЛОГИЙН ИХ СУРГУУЛЬ

МЭДЭЭЛЭЛ, ХОЛБООНЫ ТЕХНОЛОГИЙН СУРГУУЛЬ

Хураангуй

AI шийдэл бүхий хөгжим боловсруулалтын вэбсайтын
дизайн ба хөгжүүлэлт

Э.Есүдэй
esudei2845@gmail.com

Түлхүүр үгс: AI, хөгжим, веб сайт, React, TensorFlow.js, Web Audio API, машин сургалт

Өнөөгийн дижитал хөгжмийн үйлдвэрлэлд FL Studio зэрэг DAW програмууд өргөн ашиглагдаж байгаа боловч эхлэн суралцагчдын хувьд beat хийх, melody бичих, arrangement хийх, mixing/mastering хийх дараалал ойлгомжгүй хэвээр байна. Интернет орчинд сургалтын материал олон байгаа ч тэдгээр нь нэгтгэсэн бүтэц, явцын хяналт, дадлага, мэргэжлийн зөвлөгөөний орчноор дутмаг байдаг.

Энэхүү төгсөлтийн ажлын хүрээнд FL Studio болон music production сурах хүсэлтэй хэрэглэгчдэд зориулсан төлбөртэй онлайн сургалтын вэбсайтыг боловсруулж, хөгжүүлэв. Уг систем нь мэргэжлийн продюсер багш нарын бэлтгэсэн tutorial video, course catalog, paid lesson access, lesson player, dashboard, self-check quiz, progress tracking зэрэг сургалтын үндсэн функцуудыг агуулна. Сургалтын агуулга нь rhythm, melody, harmony, arrangement, beat making, mixing, mastering зэрэг чадварыг шат дараатай эзэмшүүлэх workflow болон дадлага дээр суурилна.

Хиймэл оюун ухааны хэсэг нь системийн үндсэн бүтээгдэхүүн бус, харин суралцагчийн асуултад туслах нэмэлт боломж юм. Тухайлбал, FL Studio, beat making, mixing, mastering болон хөгжмийн онолын асуултад хариулах RAG технологид суурилсан chatbot нь хичээл үзэх явцад ойлгоогүй зүйлээ лавлахад ашиглагдана. Иймээс энэхүү ажил нь мэргэжлийн продюсеруудын төлбөртэй видео хичээлийг бүтэцтэй хүргэх, суралцагчийн явцыг хянах, хэрэгцээтэй үед AI туслах зөвлөгөө авах боломжийг нэгтгэсэн сургалтын платформ болж байгаагаараа практик ач холбогдолтой.

Талархал

Бусад хүмүүст зориулсан талархлыг энд оруулна. Удирдагчаа оруулахаа
битгий мартаарай . . .

Товчилсон үгс

CPU	C entral P rocessing U nit
UML	U nified M odelling L anguage
АНУ	А мерикийн Н эгдсэн У лс

Физик тогтмолууд

Гэрлийн хурд $c_0 = 2.997\,924\,58 \times 10^8 \text{ m s}^{-1}$

Таних тэмдэгтүүд

a	distance	m
P	power	W (J s^{-1})
ω	angular frequency	rad

Өөрийн . . . -д зориулав

Гарчиг

Зохиогч эрхийн хамгаалал	i
Хураангуй	iii
Талархал	iv
Товчилсон үгс	v
Физик тогтмолууд	vi
Таних тэмдэгтүүд	vii
Удиртгал	xvi
1 Төслийн түүх, холбоотой судалгаа ба систем хөгжүүлэх арга зүй	1
1.1 Бүлгийн зорилго ба судалгааны хүрээ	1
1.2 Төслийн үүсэл, хөгжлийн түүх	1
1.3 Судалгааны асуудал	3
1.4 Хэрэглэгч ба оролцогч талуудын шинжилгээ	4
1.5 Функциональ шаардлагын эхний шинжилгээ	4
1.6 Функциональ бус шаардлагын эхний шинжилгээ	6
1.7 Ижил төстэй системүүдийн судалгаа	6
1.7.1 Онлайн сургалтын платформууд	6
1.7.2 Хөгжмийн сургалтын эх сурвалжууд	8
1.7.3 AI хөгжмийн системүүд	8
1.8 Холбоотой судалгааны ажлууд	9
1.9 Систем хөгжүүлэх аргачлалуудын харьцуулалт	11
1.9.1 Waterfall аргачлал	11
1.9.2 Prototype аргачлал	11
1.9.3 Spiral аргачлал	11
1.9.4 Agile ба Scrum	12
1.9.5 Kanban	12
1.9.6 Lean UX ба хэрэглэгч төвтэй хандлага	12
1.9.7 Design Science Research	12
1.10 Сонгосон арга зүйн загвар	13
1.11 Арга зүйн үе шат ба ажлын төлөвлөгөө	14
1.12 Үнэлгээний арга	15
1.13 Бүлгийн дүгнэлт	15

2	Технологийн судалгаа ба AI онолын шинжилгээ	16
2.1	Бүлгийн зорилго	16
2.2	Веб платформ хөгжүүлэх технологийн судалгаа	16
2.2.1	Next.js ба React архитектур	16
2.2.2	TypeScript-ийн үүрэг	17
2.2.3	Tailwind CSS ба UI зохион байгуулалт	17
2.3	Өгөгдлийн сан ба хэрэглэгчийн эрхийн технологи	17
2.3.1	Supabase ба PostgreSQL	17
2.3.2	Нэвтрэлт ба эрхийн хяналт	18
2.4	AI туслах системийн онолын үндэс	19
2.4.1	Transformer архитектур	19
2.4.2	Том хэлний загвар	19
2.4.3	Retrieval-Augmented Generation	19
2.4.4	Sparse, lexical ба dense сэргээн хайлт	20
2.5	AI үйлчилгээний abstraction давхарга	20
2.6	Хөгжим ба аудио AI-ийн судалгааны чиглэлүүд	21
2.6.1	Хөгжмийн ангилал ба аудио төлөөлөл	21
2.6.2	Эх үүсвэр салгалт	21
2.6.3	Текстээс хөгжим үүсгэх ба аудио генераци	22
2.6.4	Яриа болон аудио-текстийн хам төлөөлөл	22
2.7	RAG dataset ба өгөгдлийн чанарын онол	22
2.8	Технологийн шаардлага ба шийдлийн уялдаа	23
2.9	Client-server boundary ба rendering онол	24
2.10	Өгөгдлийн загварчлал ба RLS онол	25
2.11	Prompt engineering ба hallucination бууруулах онол	26
2.12	Аудио өгөгдөл ба ирээдүйн AI боломж	27
2.13	Технологийн эрсдэлийн судалгаа	28
2.14	Security, privacy ба ёс зүйн технологийн судалгаа	29
2.15	UX, accessibility ба сургалтын технологийн судалгаа	30
2.16	AI үнэлгээний онол ба хэмжүүр	31
2.17	Deployment ба configuration технологийн судалгаа	31
2.18	Технологийн сонголтын нэгтгэл	32
2.19	Бүлгийн дүгнэлт	33
3	Платформын дизайн ба хөгжүүлэлт	34
3.1	Бүлгийн зорилго	34
3.2	Төслийн файлын бүтэц	34
3.3	Фронтенд платформын дизайн	35
3.3.1	App Router-ийн route бүтэц	35
3.3.2	Компонентын зохион байгуулалт	35
3.4	Курс ба хичээлийн өгөгдлийн дизайн	37
3.4.1	Статик курсийн өгөгдөл	37
3.4.2	Курсийн каталогийн хэрэгжилт	37
3.5	Хичээлийн дэлгэрэнгүй ба тоглуулагч	37
3.5.1	Хандалтын хяналт	37
3.5.2	Хичээлийн тоглуулагч	38
3.5.3	Өөрийгөө шалгах сорил	38

3.6	Нэвтрэлт, худалдан авалт ба ахицын хэрэгжилт	40
3.6.1	Нэвтрэлтийн hook	40
3.6.2	Худалдан авалтын хандалт	40
3.6.3	Ахицын хяналт	40
3.7	Өгөгдлийн сангийн хэрэгжилт	40
3.7.1	Supabase schema	40
3.7.2	RLS бодлого	41
3.8	AI чатботын хэрэгжилт	42
3.8.1	API route-ийн үүрэг	42
3.8.2	Мэдээлэл сэргээх хэрэгжилт	43
3.8.3	Prompt бүрдүүлэлт	43
3.8.4	Үйлчилгээний abstraction ба fallback	43
3.9	RAG dataset builder-ийн хэрэгжилт	44
3.9.1	Pipeline-ийн бүтэц	44
3.9.2	Dataset-ийн одоогийн төлөв	45
3.10	Админ удирдлагын хэрэгжилт	45
3.11	Хэрэгжилтийн traceability шинжилгээ	45
3.12	Route бүрийн хэрэгжилтийн дэлгэрэнгүй	46
3.13	Component ба hook түвшний хэрэгжилт	47
3.14	AI chat API-ийн алгоритмын тайлбар	48
3.15	Dataset pipeline-ийн хэрэгжилтийн дэлгэрэнгүй	49
3.16	Өгөгдлийн сангийн implementation pattern	50
3.17	Туршилт ба шалгалтын хэрэгжилт	51
3.18	Хэрэгжилтийн эрсдэл ба сайжруулалтын чиглэл	52
3.19	Бүлгийн дүгнэлт	52
4	Платформын архитектур ба системийн бүтэц	54
4.1	Бүлгийн зорилго	54
4.2	Системийн ерөнхий архитектур	54
4.3	Оролцогч ба use case бүтэц	55
4.4	Үйл ажиллагааны урсгал	56
4.5	Класс ба домайн бүтэц	58
4.6	Өгөгдлийн сангийн архитектур	58
4.6.1	ER загвар	58
4.6.2	Entity бүлгүүд	58
4.6.3	Constraint ба integrity	59
4.7	Frontend архитектур	60
4.8	Backend ба API архитектур	60
4.9	AI туслахын архитектур	61
4.10	Нэвтрүүлэлтийн архитектур	62
4.11	Аюулгүй байдлын архитектур	62
4.12	Өргөтгөх боломж ба ачаалал даах төлөв	63
4.12.1	Курсийн өгөгдлийн өргөтгөл	63
4.12.2	RAG мэдлэгийн сангийн өргөтгөл	63
4.12.3	AI үйлчилгээний өргөтгөл	63
4.13	Засварлахад хялбар байдал ба кодын зохион байгуулалт	64
4.14	Чанарын баталгаажуулалтын архитектур	65

4.15	Архитектурын шийдвэрийн бүртгэл	65
4.16	Архитектурын view model	66
4.17	Data lifecycle архитектур	67
4.18	Failure mode ба resilience	68
4.19	Performance architecture	69
4.20	Observability ба operation	69
4.21	Architecture evaluation	70
4.22	Migration ба хөгжүүлэлтийн roadmap	71
4.23	C4 загварын тайлбар	72
4.24	Security zone ба trust boundary	73
4.25	Configuration ба environment архитектур	74
4.26	Decision trade-off шинжилгээ	74
4.27	System integration scenario	75
4.28	Архитектурын баримтжуулалтын ач холбогдол	76
4.29	Architecture review checklist	77
4.30	Бүлгийн дүгнэлт	79
Ном зүй		81

Зургийн жагсаалт

1.1	Төслийн хөгжлийн үе шат	2
2.1	AI туслахын онолын стек	20
3.1	Суралцагчийн үйл ажиллагааны урсгал	39
3.2	Lesson ба progress дарааллын диаграм	42
3.3	AI mentor дарааллын диаграм	42
4.1	Платформын давхаргат архитектур	54
4.2	Use Case диаграм	55
4.3	Суралцагчийн үйл ажиллагааны диаграм	57
4.4	Класс диаграм	58
4.5	Өгөгдлийн сангийн ER диаграм	59
4.6	AI туслахын дарааллын архитектур	61
4.7	Deployment architecture	62

Хүснэгтийн жагсаалт

1.1	Төслийн хөгжлийн үе шат ба судалгааны утга	2
1.3	Системийн үндсэн функциональ шаардлага	4
1.2	Оролцогч талууд ба хэрэгцээ	6
1.4	Функциональ бус шаардлагын ангилал	7
1.5	Ижил төстэй системүүдийн харьцуулалт	8
1.6	Арга зүй ба систем хөгжүүлэлтийн холбоотой эх сурвалжууд .	9
1.7	Аргачлалуудын харьцуулсан шинжилгээ	12
1.8	Сонгосон арга зүйн бүрэлдэхүүн	14
1.9	Үнэлгээний шалгуур	15
2.1	Өгөгдлийн технологийн сонголтын үндэслэл	18
2.2	AI provider сонголтын харьцуулалт	21
2.3	RAG dataset чанарын үндсэн үзүүлэлт	23
2.4	Шаардлага ба технологийн сонголтын уялдаа	23
2.5	Client ба server талын логикийн ялгаа	25
2.6	RLS бодлогын онолын ангилал	26
2.7	AI хариултын эрсдэл ба бууруулах арга	27
2.8	Ирээдүйн audio AI модулийн боломжит түвшин	28
2.9	Технологийн эрсдэл ба удирдлагын арга	28
2.10	Security ба privacy-ийн технологийн шаардлага	29
2.11	UX ба accessibility технологийн шалгуур	30
2.12	AI туслахын үнэлгээний боломжит хэмжүүр	31
2.13	Deployment configuration-ийн үндсэн ангилал	32
2.14	Технологийн сонголтын нэгтгэл	32
3.1	Төслийн гол хавтас ба үүрэг	34
3.2	Frontend route-ууд ба хэрэглэгчийн урсгал	36
3.3	Frontend компонентуудын үүрэг	36
3.4	Course catalog-ийн гол боломжууд	38
3.5	Өгөгдлийн сангийн хүснэгтүүд ба үүрэг	40
3.6	AI chatbot-ийн инженерчлэлийн шийдлүүд	43
3.7	Dataset бүрэлдэхүүнүүд	45
3.8	Шаардлагаас хэрэгжилт хүртэлх traceability	46
3.9	Route бүрийн implementation responsibility	47
3.10	Component ба hook-ийн хэрэгжилтийн үүрэг	48
3.11	AI chat API-ийн хэрэгжилтийн алхам	49
3.12	Dataset pipeline-ийн artifact-ууд	50
3.13	Database implementation pattern	50
3.14	Implementation verification checklist	51

3.15	Хэрэгжилтийн эрсдэл ба сайжруулалтын төлөвлөгөө	52
4.1	Use case тайлбар	55
4.2	Frontend архитектурын түвшин	60
4.3	API архитектурын үүрэг	61
4.4	Security control matrix	63
4.5	Өргөтгөлийн чиглэл	64
4.6	Чанарын шалгалтын түвшин	65
4.7	Архитектурын шийдвэрүүд	65
4.8	Архитектурын view model	67
4.9	Өгөгдлийн lifecycle	68
4.10	Failure mode ба resilience strategy	68
4.11	Performance bottleneck ба optimization	70
4.12	Observability төлөвлөгөө	70
4.13	Архитектурын чанарын үнэлгээ	71
4.14	Архитектурын roadmap	71
4.15	C4 архитектурын тайлбар	72
4.16	Security zone ба trust boundary	73
4.17	Environment architecture	74
4.18	Архитектурын trade-off шинжилгээ	75
4.19	Integration scenario дахь давхаргын оролцоо	76
4.20	Архитектурын баримтжуулалтын хэрэглэгч	77
4.21	Архитектурын review checklist	77

Удиртгал

Дижитал хөгжмийн үйлдвэрлэл хөгжихийн хэрээр FL Studio зэрэг дижитал аудио ажлын орчин (DAW) ашиглан хэмнэл бүтээх, ая зохиох, бүтээлийн зохиомж боловсруулах, холилт болон мастеринг хийх сонирхолтой хэрэглэгчдийн тоо нэмэгдэж байна. Гэвч эхлэн суралцагчдын хувьд хөгжмийн онол, програмын хэрэгсэл, бүтээлч ажлын урсгал, техникийн сонголтуудыг нэгэн зэрэг ойлгох шаардлагатай байдаг тул суралцах үйл явц ихэвчлэн тасралттай, тодорхой дараалалгүй болдог. Интернет орчинд хөгжмийн сургалтын материал олон боловч тэдгээр нь чанар, түвшин, дараалал, дадлагын чиглүүлэг, ахицын хяналтын хувьд харилцан адилгүй байна.

Ийм нөхцөлд суралцагч зөвхөн тусдаа видео үзэх бус, харин мэргэжлийн багшийн бэлтгэсэн хичээл, дараалсан дадлага, өөрийгөө шалгах сорил, төлбөртэй хандалтын удирдлага, хувийн ахицын хяналт, шаардлагатай үед асуултаа лавлах хиймэл оюун ухааны туслахыг нэг орчинд ашиглах хэрэгцээ үүсэж байна. Энэхүү дипломын ажил нь уг хэрэгцээнд тулгуурлан FL Studio болон хөгжим боловсруулалт сурах хэрэглэгчдэд зориулсан, хиймэл оюун ухааны дэмжлэгтэй веб сургалтын платформын дизайн ба хөгжүүлэлтийг авч үзнэ.

Зорилго

Энэхүү дипломын ажлын зорилго нь FL Studio болон хөгжим боловсруулалт суралцагчдад зориулан мэргэжлийн видео хичээл, төлбөртэй хичээлийн хандалт, хичээлийн тоглуулагч, өөрийгөө шалгах сорил, суралцах ахицын хяналт болон RAG технологид суурилсан AI туслахыг нэгтгэсэн веб сургалтын платформ зохион бүтээж, хөгжүүлэхэд оршино.

Зорилт

Энэхүү зорилгыг хэрэгжүүлэхийн тулд дараах зорилтуудыг дэвшүүлэв:

- FL Studio болон хөгжим боловсруулалт суралцагчдад тулгардаг үндсэн хүндрэл, онлайн сургалтын хэрэгцээг тодорхойлох;

- ижил төстэй сургалтын платформ, AI туслах систем болон хөгжмийн AI судалгааны ажлуудыг харьцуулан судлах;
- курсийн каталог, төлбөртэй хичээлийн хандалт, хичээлийн тоглуулагч, хяналтын самбар, өөрийгөө шалгах сорил, ахицын хяналт бүхий сургалтын урсгалыг загварчлах;
- Next.js, React, TypeScript, Supabase болон PostgreSQL-д тулгуурласан веб системийн архитектур, өгөгдлийн загварыг боловсруулах;
- FL Studio, хэмнэл бүтээх арга, холилт, мастеринг болон хөгжмийн онолын асуултад контекстэд тулгуурлан хариулах RAG суурьтай чатботыг системд нэгтгэх;
- хөгжүүлсэн системийн үндсэн ажиллагаа, хэрэглэгчийн урсгал, өгөгдлийн хадгалалт болон AI туслахын хэрэглээний боломжийг турших.

Систем хөгжүүлэх үндэслэл

Хөгжим боловсруулалтын сургалтын платформ хөгжүүлэх хэрэгцээ нь сургалтын зохион байгуулалт, хэрэглэгчийн туршлага, технологийн боломж гэсэн гурван үндэслэлээр тайлбарлагдана.

Систем хөгжүүлэх гол үндэслэл нь:

1. FL Studio суралцах үйл явц нь хэмнэл бүтээх, аялгуу ба гармони боловсруулах, зохиомж хийх, холилт, мастеринг зэрэг олон чадварыг үе шаттай эзэмшихийг шаарддаг. Иймд тархай эх сурвалжаас суралцахаас илүү дараалсан сургалтын хөтөлбөр, дадлага, өөрийгөө шалгах сорил бүхий сургалтын замнал шаардлагатай.
2. Багш, продюсерын бэлтгэсэн төлбөртэй видео хичээлийг хэрэглэгчийн эрхийн удирдлага, хичээлийн явцын бүртгэл, хяналтын самбар болон өгөгдлийн сангийн найдвартай бүтэцтэй холбосноор сургалтын чанар, хүртээмжийг нэмэгдүүлэх боломжтой.
3. Орчин үеийн хэлний загвар болон мэдээлэл сэргээн баяжуулсан генерацийн аргачлал нь суралцагчийн асуултад гаднын мэдлэгийн сангаас сонгосон контекст ашиглан хариулах боломжийг бүрдүүлж байна [1, 2, 3]. Энэ нь хичээл үзэх явцад гарсан тодорхой асуултыг шуурхай тайлбарлах нэмэлт сургалтын хэрэгсэл болж чадна.

Иймээс энэхүү систем нь хөгжмийн сургалтын агуулгыг бүтэцтэй хүргэх, суралцагчийн ахицыг хэмжих, төлбөртэй контентын хандалтыг зохион байгуулах, мөн AI туслахыг хязгаарлагдсан сургалтын контекстэд ашиглах бодит шаардлагад тулгуурлаж байна.

Судалгааны өнөөгийн байдал

Сүүлийн жилүүдэд Transformer архитектур, том хэлний загвар, мэдээлэл сэргээн баяжуулсан генераци (RAG) зэрэг аргачлал боловсролын туслах системд өргөн ашиглагдах болсон [1, 2, 4, 5, 6]. Эдгээр арга нь асуултад шууд хариулах боломжийг нэмэгдүүлэхийн зэрэгцээ эх сурвалжтай контекст ашигласнаар буруу, ерөнхий эсвэл баталгаагүй зөвлөмж өгөх эрсдэлийг бууруулах чиглэлд хөгжиж байна.

Хөгжим-AI чиглэлд дууны долгион болон тэмдэглэгээний түвшний моделчлол, текстээс хөгжим үүсгэх генераци, аудио төлөөлөл суралцах зэрэг судалгаанууд эрчимтэй хөгжиж байна [7, 8, 9, 10, 11, 12]. Гэвч энэхүү дипломын ажлын хүрээнд AI-г бүрэн автомат хөгжим үүсгэгч бус, харин мэргэжлийн багшийн боловсруулсан сургалтын агуулгыг дэмжих, хичээлтэй холбоотой асуултад контексттэй хариулах туслах функц байдлаар авч үзсэн.

Шинэлэг тал

Энэхүү системийн шинэлэг тал нь FL Studio болон хөгжим боловсруулалтын төлбөртэй видео сургалтыг RAG суурьтай AI туслах, хэрэглэгчийн ахицын хяналт, хичээлийн эрхийн удирдлагатай нэг платформд уялдуулсанд оршино. Үүнд:

- хичээл, дадлага, сорил, ахицын мэдээллийг нэг сургалтын урсгалд холбосон хэрэглэгчийн интерфейс;
- төлбөртэй хичээлийн хандалт, хэрэглэгчийн бүртгэл, хяналтын самбар болон ахицын хяналтыг нэг өгөгдлийн загварт нэгтгэсэн шийдэл;
- FL Studio болон хөгжмийн онолын мэдлэгийн сангаас хэрэглэгчийн асуултад тохирох контекст сонгон хариулах AI туслах;
- мэргэжлийн багшийн хичээлд суурилсан агуулгыг AI зөвлөмжөөр нэмэлтээр дэмжсэн сургалтын туршлага.

Ач холбогдол

Энэхүү системийг хэрэгжүүлснээр FL Studio болон хөгжим боловсруулалт сурах хүсэлтэй хэрэглэгчид сургалтын агуулгыг шат дараатай үзэх, өөрийн ойлголтоо сорилоор шалгах, хичээлийн явцаа хянах, тодорхой асуудал гарсан үед AI туслахтай харилцан лавлах боломжтой болно. Энэ нь эхлэн суралцагчдын суралцах тасралтыг бууруулж, дадлагад суурилсан чадвар эзэмшилтийг дэмжинэ.

Мөн багш, продюсеруудад мэргэжлийн контентоо бүтэцтэй сургалтын бүтээгдэхүүн болгон хүргэх, төлбөртэй хандалтаар орлогожуулах, суралцагчийн ахицын мэдээлэлд тулгуурлан сургалтын агуулгаа сайжруулах боломж бүрдэнэ. Бүтээлч салбарын боловсролын хувьд уг ажил нь веб технологи, өгөгдлийн сан, хэрэглэгчийн туршлага болон хиймэл оюун ухааны туслах боломжийг нэгтгэсэн жишээ загвар болох практик ач холбогдолтой.

Хамрах хүрээ

Энэхүү дипломын ажлын хамрах хүрээнд хэрэглэгчийн бүртгэл, нэвтрэлт, курсийн жагсаалт, төлбөртэй хичээлийн хандалт, хичээлийн тоглуулагч, өөрийгөө шалгах сорил, суралцах явцын хяналт, хяналтын самбар, админ удирдлага, мөн хичээлтэй холбоотой асуултад контекстээр дэмжлэг үзүүлэх AI чат-бот багтана. Систем нь FL Studio болон хөгжим боловсруулалтын сургалтын хэрэглээнд төвлөрөх бөгөөд хүний багшийг бүрэн орлох, хэрэглэгчийн өмнөөс бүрэн автомат хөгжим бүтээх, эсвэл студийн бүх үйлдвэрлэлийн процессыг автоматжуулах зорилгогүй.

Тайлангийн бүтэц

Энэхүү тайлангийн бүтэц нь судалгааны үндэслэлээс эхлэн технологийн сонголт, хэрэгжилт, системийн архитектур хүртэл үе шаттайгаар уялдах зарчимд тулгуурлав. Нэгдүгээр бүлэгт төслийн үүсэл, ижил төстэй системийн судалгаа, хөгжүүлэх арга зүйн харьцуулалт болон сонгосон methodology-г тайлбарлана. Хоёрдугаар бүлэгт веб технологи, өгөгдлийн сан, хиймэл оюун ухаан, Transformer, RAG болон хөгжмийн AI онолын үндсийг судална. Гуравдугаар бүлэгт frontend, backend, database, AI chatbot, dataset builder зэрэг бодит хэрэгжилтийн хэсгүүдийг төслийн файлын бүтэцтэй холбон тайлбарлана. Дөрөвдүгээр бүлэгт платформын архитектур, use case, activity, class, ER, deployment болон security бүтэц, мөн системийн чанарын үнэлгээг нэгтгэн авч үзнэ.

БҮЛЭГ 1

Төслийн түүх, холбоотой судалгаа
ба систем хөгжүүлэх арга зүй

1.1 Бүлгийн зорилго ба судалгааны хүрээ

Энэхүү бүлэг нь дипломын ажлын эхний судалгааны суурь хэсэг бөгөөд системийн бодит хэрэгжилт рүү шууд орохоос өмнө төслийн үүсэл, хөгжлийн шалтгаан, ижил төстэй шийдлүүд, хэрэглэгчийн хэрэгцээ, программ хангамж хөгжүүлэх арга зүйн сонголтыг тайлбарлана. Өөрөөр хэлбэл, энэ бүлгийн үндсэн зорилго нь *“яагаад ийм систем хэрэгтэй вэ, ямар төрлийн системүүдтэй хамааралтай вэ, ямар хөгжүүлэлтийн аргачлалаар хэрэгжүүлэх нь зохистой вэ?”* гэсэн гурван асуултад академик байдлаар хариулахад оршино.

Энэхүү төсөл нь FL Studio болон хөгжим боловсруулалт суралцагчдад зориулсан веб сургалтын платформ юм. Систем нь курсийн каталог, төлбөртэй хичээлийн хандалт, хичээлийн тоглуулагч, өөрийгөө шалгах сорил, суралцах явцын хяналт, админ удирдлага, мөн RAG суурьтай AI туслахыг нэгтгэнэ. Гэхдээ эдгээр модулийн техникийн хэрэгжилтийг энэ бүлэгт дэлгэрэнгүй авч үзэхгүй. Харин тэдгээрийг хөгжүүлэхэд ямар судалгааны үндэслэл, ямар арга зүйн сонголт хэрэгтэй байсныг тодорхойлно.

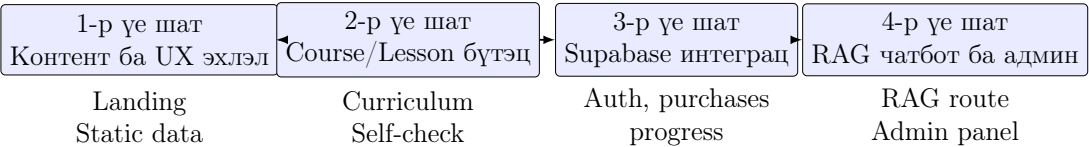
Судалгааны хүрээг дараах байдлаар тогтоов.

- төслийн хөгжлийн үе шат, гол шийдвэрүүдийг нэгтгэн тайлбарлах;
- онлайн сургалтын платформ, хөгжмийн сургалтын систем, AI туслахтай сургалтын орчны ижил төстэй шийдлүүдийг харьцуулах;
- хэрэглэгчийн хэрэгцээ, функциональ болон функциональ бус шаардлагын эхний түвшний шинжилгээ хийх;
- Waterfall, Prototype, Spiral, Agile/Scrum, Kanban, Lean UX, Design Science Research зэрэг аргачлалуудыг харьцуулж, төслийн нөхцөлд тохирох арга зүйг сонгох;
- сонгосон арга зүйг энэ дипломын төслийн бодит ажлын төлөвлөгөөтэй уялдуулах.

1.2 Төслийн үүсэл, хөгжлийн түүх

Энэхүү дипломын төсөл нь анх хиймэл оюун ухааны тусламжтай хөгжим боловсруулалтын вебсайт бүтээх санаанаас эхэлсэн. Хөгжүүлэлтийн явцад уг санаа зөвхөн нэг AI функцтэй туршилтын систем бус, харин сургалтын контент, төлбөртэй хандалт, суралцах явц, өгөгдлийн сан, AI туслахыг нэгтгэсэн сургалтын платформ болж өргөжсөн. Репозиторийн бүтцэд **Frontend**,

dataset, rag-dataset-builder, diplomapaper зэрэг бүрэлдэхүүнүүд зэрэгцэн оршиж байгаа нь төслийг нэг бүтээгдэхүүн, нэг судалгааны объект гэж авч үзэх үндэслэл болно.



ЗУРАГ 1.1: Төслийн хөгжлийн үндсэн үе шат ба бүрэлдэхүүнүүдийн уялдаа

ХҮСНЭГТ 1.1: Төслийн хөгжлийн үе шат ба судалгааны утга

Үе шат	Гол зорилго	Бодит материал	Судалгааны ач холбогдол
I үе шат	Сургалтын бүтээгдэхүүний анхны чиглэлийг тодорхойлох	Landing page, hero болон learning path компонентууд	Суралцагчид зориулсан үнэ цэнийн санал, анхны UX урсгал, бүтээгдэхүүний байршуулалт тодорсон.
II үе шат	Курс, хичээл, дадлагын бүтцийг бүрдүүлэх	Course data, catalog page, course detail page	Сургалтын контентыг курс, lesson, exercise, takeaway хэлбэрээр шаталсан загварт оруулсан.
III үе шат	Хэрэглэгч, төлбөр, явцын өгөгдлийн суурь тавих	Supabase schema, auth болон purchase hooks	Бүртгэл, худалдан авалт, явц хадгалалт зэрэг платформын үндсэн бизнес логик тодорхой болсон.

IV үе шат	AI туслах болон мэдлэгийн санг системд холбох	Chat API, RAG builder, dataset	RAG суурьтай асуулт-хариултын туслахыг сургалтын орчинд нэмэлт функц байдлаар нэгтгэсэн.
V үе шат	Дипломын баримт бичгийн зохион байгуулалт	diplomaper хавтас, диаграм, бүлгүүд	Бүтээгдэхүүний хэрэгжилтийг судалгаа, арга зүй, зохиомж, архитектуртай холбон тайлбарлах шаардлага үүссэн.

Энэ хөгжлийн түүхээс харахад төсөл нь эхнээсээ бүрэн тодорхой шаардлагатай, нэг удаагийн шугаман хөгжүүлэлтээр дуусах систем биш байв. Сургалтын контентын бүтэц, хэрэглэгчийн урсгал, төлбөртэй хандалт, AI туслахын үүрэг зэрэг нь хөгжүүлэлтийн явцад туршилт, засвар, дахин зохион байгуулалтаар боловсорсон. Иймээс арга зүйн хувьд давталттай, уян хатан, эрсдэлийг үе шаттай бууруулдаг хандлага шаардлагатай болсон.

1.3 Судалгааны асуудал

Хөгжим боловсруулалт сурах үйл явц нь програм ашиглах техникийн мэдлэг, хөгжмийн онолын ойлголт, бүтээлч дадлага, сонсголын үнэлгээ, микс ба мастерингийн шийдвэрийг зэрэг шаарддаг. Эхлэн суралцагчид ихэвчлэн YouTube, форум, богино нийтлэл, курсийн хэсэгчилсэн материал зэрэг олон эх сурвалжаас суралцдаг боловч тэдгээр эх сурвалжууд хоорондоо дараалал, түвшин, нэр томъёо, дадлагын аргачлалаараа зөрүүтэй байдаг.

Энэхүү асуудлыг гурван түвшинд авч үзэж болно.

1. **Сургалтын зохион байгуулалтын асуудал:** эхлэн суралцагч ямар хичээлээс эхлэх, дараагийн алхамд ямар чадвар эзэмших, ямар дадлага хийхээ тодорхой мэдэхгүй.
2. **Дадлага ба ахицын хэмжилтийн асуудал:** видео үзсэн эсэхээс гадна тухайн ойлголтыг хэрхэн шалгах, ямар дасгалаар бататгах, ахицаа хэрхэн хадгалах нь тодорхойгүй.

3. **Тухайн мөчийн зөвлөгөөний асуудал:** суралцагч хичээл үзэж байхдаа ойлгоогүй нэр томъёо, plugin тохиргоо, mixing шийдвэр, FL Studio-ийн үйлдэлтэй холбоотой жижиг асуултыг тэр дор нь тодруулах хэрэгцээтэй.

Эдгээр асуудлыг зөвхөн AI чатботоор шийдвэрлэх боломжгүй. AI нь буруу, ерөнхий эсвэл эх сурвалжгүй зөвлөгөө өгөх эрсдэлтэй тул үндсэн сургалтын контентыг орлох бус, харин бүтэцтэй сургалтын урсгалыг дэмжих хэлбэрээр ашиглагдах ёстой. Иймээс төслийн үндсэн судалгааны асуудлыг дараах байдлаар тодорхойлов.

FL Studio болон хөгжим боловсруулалт суралцагчдад зориулсан сургалтын платформд курсийн бүтэц, төлбөртэй хандалт, ахицын хяналт, AI туслахыг хэрхэн уялдаа холбоотой, өргөтгөх боломжтой, хэрэглэгч төвтэй байдлаар хөгжүүлэх вэ?

1.4 Хэрэглэгч ба оролцогч талуудын шинжилгээ

Системийн үндсэн хэрэглэгч нь FL Studio болон хөгжим боловсруулалт шинээр сурах, эсвэл эхний түвшний чадвараа цэгцлэх хүсэлтэй суралцагч юм. Үүнээс гадна багш, продюсер, админ, систем хөгжүүлэгч, контент бэлтгэгч зэрэг оролцогч талууд системд шууд болон шууд бусаар нөлөөлнө.

Хэрэглэгчийн урсгалын хувьд систем нь *танилцах* → *курс сонгох* → *хандалт баталгаажуулах* → *хичээл үзэх* → *өөрийгөө шалгах* → *AI туслахаас лавлах* → *ахицаа хадгалах* гэсэн шатлалтай. Энэ нь цэвэр мэдээллийн сайт бус, харин суралцах үйл явцыг удирдах систем болохыг харуулна.

1.5 Функциональ шаардлагын эхний шинжилгээ

Функциональ шаардлага нь хэрэглэгч системээр ямар үйлдэл хийх ёстойг тодорхойлно. Энэ бүлэгт шаардлагыг ерөнхий түвшинд авч үзэх ба нарийвчилсан зохиомж, өгөгдлийн бүтэц, API урсгалыг дараагийн бүлгүүдэд тайлбарлана.

Хүснэгт 1.3: Системийн үндсэн функциональ шаардлага

Код	Шаардлага	Тайлбар
FR-01	Хэрэглэгч бүртгүүлэх ба нэвтрэх	Суралцагч өөрийн бүртгэлээр системд нэвтэрч худалдан авалт, ахиц, чат түүх зэрэг хувийн мэдээлэлтэй ажиллана.
FR-02	Курс хайх, шүүх, эрэмбэлэх	Хэрэглэгч курсийг ангилал, түвшин, үнэ, хайлтын үгээр шүүж өөрт тохирох сургалтын замналаа сонгоно.
FR-03	Хичээлийн дэлгэрэнгүй үзэх	Курсийн тайлбар, багш, үнэ, free/paid lesson, curriculum, duration зэрэг мэдээллийг харуулна.
FR-04	Төлбөртэй хичээлийн хандалт шалгах	Систем хэрэглэгч тухайн курсийг худалдан авсан эсэхээс хамаарч lesson access-ийг нээж эсвэл хязгаарлана.
FR-05	Хичээл тоглуулах ба lesson brief үзүүлэх	Видео бүхий lesson-г тоглуулах, видео байхгүй үед summary, exercise, takeaway хэлбэрийн богино сургалтын материалыг харуулна.
FR-06	Өөрийгөө шалгах сорил	Хэрэглэгч lesson болон course түвшний асуултад хариулж, ойлголтоо бататгана.
FR-07	Суралцах явц хадгалах	Дуусгасан lesson, course progress, хамгийн сүүлд үзсэн lesson зэрэг мэдээллийг хадгална.
FR-08	AI туслахаас асуулт асуух	Суралцагч FL Studio, mixing, mastering, хөгжмийн онолын асуултаа чатботод илгээж RAG контексттэй хариу авна.
FR-09	Админ удирдлага	Админ курс, lesson, хэрэглэгчийн өгөгдөл, мэдлэгийн сан зэрэг мэдээллийг хянах, засах боломжтой байна.
FR-10	Dataset бэлтгэх ба импортлох	RAG мэдлэгийн сангийн өгөгдлийг JSONL/CSV хэлбэрээр бэлтгэж Supabase knowledge base-д импортлох боломжтой байна.

Хүснэгт 1.2: Оролцогч талууд ба хэрэгцээ

Оролцогч	Гол хэрэгцээ	Системийн хариу шийдэл
Зочин хэрэглэгч	Курсийн мэдээлэл үзэх, үнэ цэнэ ойлгох, эхний хичээлийг турших	Landing page, course catalog, free lesson preview
Суралцагч	Курс үзэх, төлбөртэй хичээлд хандах, ахицаа хадгалах, асуулт асуух	Lesson player, purchase access, self-check quiz, progress tracking, AI mentor
Багш/продюсер	Контентоо бүтэцтэй хүргэх, төлбөртэй бүтээгдэхүүн болгох	Course, lesson, teacher profile, админ удирдлагын боломж
Админ	Курс, хичээл, хэрэглэгч, мэдлэгийн санг хянах	Admin dashboard, Supabase өгөгдлийн загвар, RLS бодлого
Систем хөгжүүлэгч	Модульчлагдсан, засварлахад ойлгомжтой бүтэцтэй кодын бааз	Next.js App Router, компонент бүтэц, API route, Supabase integration

1.6 Функциональ бус шаардлагын эхний шинжилгээ

Функциональ бус шаардлага нь системийн чанарын шинжүүдийг тодорхойлно. Сургалтын платформын хувьд хэрэглэгчийн итгэл, өгөгдлийн хамгаалалт, хариу өгөх хурд, өргөтгөх боломж, засварлахад хялбар байдал чухал.

1.7 Ижил төстэй системүүдийн судалгаа

Ижил төстэй системүүдийг судлахдаа зөвхөн хөгжмийн AI бүтээгдэхүүнүүдийг бус, онлайн сургалтын платформ, хөгжмийн сургалтын орчин, AI туслахтай мэдлэгийн систем, DAW хэрэглэгчийн сургалтын эх сурвалжуудыг хамруулсан. Учир нь энэхүү дипломын төсөл эдгээр дөрвөн чиглэлийн уулзвар дээр байрлаж байна.

1.7.1 Онлайн сургалтын платформууд

Coursera, Udemy, Skillshare зэрэг платформууд курсийн каталог, төлбөртэй контент, багшийн профайл, хэрэглэгчийн явцын хяналт, үнэлгээ зэрэг нийтлэг функцтэй. Эдгээр системийн давуу тал нь сургалтын бүтээгдэхүүнийг худалдаалах, хичээлийг багцлах, суралцагчийн хэрэглээг хэмжих туршлага

Хүснэгт 1.4: Функциональ бус шаардлагын ангилал

Чанарын шинж	Шаардлагын утга	Төслийн хүрээнд хэрэгжүүлэх чиглэл
Ашиглахад хялбар байдал	Анхан шатны суралцагч интерфэйсийг тайлбаргүй ойлгох	Course catalog, lesson player, dashboard, чат товчийг ойлгомжтой байрлуулах
Найдвартай байдал	Хичээлийн хандалт, ахиц, төлбөртэй эрх алдагдахгүй байх	Supabase өгөгдлийн сан, unique constraint, RLS бодлого ашиглах
Аюулгүй байдал	Хэрэглэгч зөвхөн өөрийн өгөгдөлд хандах	Authentication, authorization, row level security
Өргөтгөх чадвар	Шинэ курс, багш, dataset, AI provider нэмэхэд суурь бүтэц эвдрэхгүй байх	Модульчлагдсан компонент, API route, provider abstraction
Гүйцэтгэл	Курсийн жагсаалт, хичээлийн хуудас, чат хариу боломжийн хугацаанд ажиллах	Next.js rendering, local dataset cache, provider fallback
Академик найдвартай байдал	AI хариулт эх сурвалжгүйгээр албан ёсны мэдлэг мэт харагдахгүй байх	RAG context, citation-safe flag, generated guidance-ийг ялгах

юм. Гэвч ерөнхий платформууд нь FL Studio-ийн тодорхой ажлын урсгал, монгол-англи холимог нэр томъёо, тухайн хичээлийн мөчид асуулт асуух AI туслахыг заавал агуулдаггүй.

1.7.2 Хөгжмийн сургалтын эх сурвалжууд

YouTube, FL Studio-ийн албан ёсны гарын авлага, продюсеруудын богино tutorial, хөгжмийн форумууд нь хөгжмийн продакшн сурахад өргөн хэрэглэгддэг. Эдгээр нь хүртээмжтэй боловч эхлэн суралцагчид хэрэгтэй дараалсан замнал, ахицын хэмжилт, төлбөртэй course access, нэгтгэсэн dashboard-оор дутмаг. Иймээс энэ төслийн платформ нь нээлттэй эх сурвалжуудыг бүрэн орлох бус, харин тэдгээрээс сурахад тулгардаг зохион байгуулалтын асуудлыг багасгах зорилготой.

1.7.3 AI хөгжмийн системүүд

Jukebox, MusicLM, MusicGen зэрэг судалгааны ажлууд болон Magenta, Suno, AIVA, SOUNDRAW зэрэг бүтээгдэхүүнүүд нь AI-г хөгжим үүсгэх, audio modeling, бүтээлч санаа гаргах чиглэлд ашиглаж байна [7, 9, 10, 13, 14, 15, 16]. Гэхдээ энэ дипломын ажилд AI-ийн үүрэг нь хөгжмийг хэрэглэгчийн өмнөөс бүрэн үүсгэх бус, харин хүний багшийн сургалтын контентыг дэмжих зөвлөгчийн үүрэг юм. Энэ ялгаа нь системийн эрсдэл, зохиогчийн эрх, сургалтын чанарын талаас чухал.

Хүснэгт 1.5: Ижил төстэй системүүдийн харьцуулалт

Системийн төрөл	Давуу тал	Хязгаарлалт	Энэ төсөлд авсан санаа
Онлайн сургалтын платформ	Курсийн бүтэц, төлбөр, багшийн профайл, ахицын хяналт сайн хөгжсөн	Ерөнхий сэдэвт төвлөрдөг, FL Studio-ийн workflow-д нарийн тохироогүй	Course catalog, paid access, dashboard, progress tracking
YouTube/tutorial эх сурвалж	Хүртээмж өндөр, олон жишээтэй, бодит хэрэглээтэй	Дараалалгүй, түвшин холилдсон, ахиц хадгалахгүй	Free preview, lesson brief, curated learning path

DAW гарын авлага	Албан ёсны, на- рийвчилсан, нэр- томьёо зөв	Эхлэн суралцагчид хэт техникийн, дад- лага багатай	Knowledge base- д эх сурвалжийн metadata хадга- лах
AI music generator	Бүтээлч санаа хур- дан гаргах, генера- тив боломж өндөр	Сургалтын замнал, хүний багшийн тайлбар, ахицын үнэлгээ хангалтгүй	AI-г контент бүтээгч бус, сур- галтын туслах болгон ашиглах
RAG chatbot	Асуултад кон- тексттэй хариулах, мэдлэгийн сан- гаар хязгаарлах боломжтой	Dataset чанар, эх сурвалжийн найдвартай бай- дал, hallucination эрсдэлтэй	RAG dataset, citation-safe flag, fallback logic

1.8 Холбоотой судалгааны ажлууд

Энэхүү дипломын төслийн судалгааны суурь нь хоёр хэсэгтэй. Нэгдүгээрт, программ хангамж хөгжүүлэх арга зүй, шаардлагын шинжилгээ, хэрэглэгч төвтэй дизайн, дизайн шинжлэх ухааны судалгааны ажил; хоёрдугаарт, боловсролын системд AI ашиглах, RAG, аудио AI, хөгжмийн мэдээлэл боловсруулах судалгаанууд юм. AI онолын нарийвчилсан тайлбарыг хоёрдугаар бүлэгт авч үзэх тул энэ хэсэгт зөвхөн төслийн арга зүйтэй шууд холбоотой ажлуудыг нэгтгэнэ.

Хүснэгт 1.6: Арга зүй ба систем хөгжүүлэлтийн холбоотой эх сурвалжууд

#	Эх сурвалж	Гол санаа	Төслийн хэрэглээ
1	Royce-ийн Waterfall загвар [17]	Том системд үе шат, баримтжуулалт, хя- налтыг тодорхойлсон	Дипломын тайлан, шаардлага, зохиом- жийн баримтжуулал- тад хэрэгтэй боловч өөрчлөлт ихтэй веб системд дангаар тохи- ромжгүй.

2	Boehm-ийн Spiral загвар [18]	Эрсдэлд суурилсан давталттай хөгжүүлэлт	AI туслах, dataset чанар, provider сонголтын эрсдэлийг үе шаттай бууруулахад тохиромжтой.
3	Agile Manifesto [19]	Ажиллаж буй бүтээгдэхүүн, өөрчлөлтөд хариу үйлдэл, хэрэглэгчийн хамтын ажиллагааг чухалчилсан	Landing, course, dashboard, chat зэрэг модулийг богино давталтаар сайжруулах үндэс болсон.
4	Scrum Guide [20]	Sprint, backlog, incremental delivery зарчим	Дипломын ажлын feature backlog, iteration, review төлөвлөлтөд ашиглах боломжтой.
5	Kanban аргачлал [21]	Work-in-progress хязгаарлах, урсгалыг харагдуулах	Нэг хүн эсвэл жижиг багийн дипломын төсөлд task flow-ийг хянахад тохиромжтой.
6	Lean UX [22]	Таамаглал, prototype, user feedback-д суурилсан UX хөгжүүлэлт	Сургалтын интерфейс, course discovery, chat drawer зэрэг хэрэглэгчийн урсгалыг туршихад хэрэгтэй.
7	Design Science Research [23]	Артефакт бүтээж, түүний хэрэгцээ ба үр нөлөөг үнэлэх судалгааны арга	Энэхүү дипломын ажил нь судалгаа + бодит платформ гэсэн хоёр үр дүнтэй тул DSR логиктой нийцнэ.

1.9 Систем хөгжүүлэх аргачлалуудын харьцуулалт

Программ хангамжийн аргачлал сонгохдоо төслийн хэмжээ, багийн бүтэц, шаардлагын тогтвортой байдал, эрсдэлийн төрөл, хэрэглэгчийн оролцоо, баримтжуулалтын хэрэгцээг харгалзан үздэг. Энэ дипломын төсөлд шаардлага хөгжүүлэлтийн явцад нарийссан, AI болон dataset хэсэг туршилт ихтэй, frontend UX олон давталтаар сайжирсан тул хатуу шугаман аргачлал дангаараа хангалтгүй.

1.9.1 Waterfall аргачлал

Waterfall аргачлал нь шаардлага, шинжилгээ, зохиомж, хэрэгжилт, туршилт, нэвтрүүлэлт гэсэн дараалсан үе шаттай [17]. Давуу тал нь баримтжуулалт сайн, хяналтын цэг тодорхой, сургалтын байгууллагын тайлангийн бүтэцтэй нийцдэг. Харин сул тал нь шаардлага өөрчлөгдөхөд засварын өртөг өсдөг. Энэхүү төслийн AI туслах, RAG dataset, хэрэглэгчийн урсгал зэрэг нь хөгжүүлэлтийн явцад тодорсон тул Waterfall-ийг зөвхөн баримтжуулалт, тайлангийн логикт ашиглах нь зохистой.

1.9.2 Prototype аргачлал

Prototype аргачлал нь хэрэглэгчийн ойлголт бүрэн тодорхойгүй үед хурдан загвар гаргаж, түүнийгээ туршин сайжруулахад тохиромжтой. Энэ төслийн landing page, course catalog, lesson detail, chat drawer зэрэг интерфэйсүүд prototype хандлагаар боловсорсон. Давуу тал нь хэрэглэгчийн туршлагыг эрт шалгах боломжтой. Сул тал нь prototype-ийг бүтээгдэхүүний эцсийн код болгож үлдээвэл архитектурын өр нэмэгдэх эрсдэлтэй.

1.9.3 Spiral аргачлал

Spiral загвар нь эрсдэлийг давталт бүрд тодорхойлж, шийдэл туршин, дараагийн мөчлөгт шилждэг [18]. AI-тэй системд hallucination, dataset чанар, provider availability, latency, cost зэрэг эрсдэл өндөр байдаг тул энэ аргачлал оновчтой. Жишээлбэл, RAG chatbot эхний шатанд локал JSONL retrieval ашиглаж, дараагийн шатанд Supabase pgvector embedding рүү шилжих боломжтойгоор төлөвлөгдсөн.

1.9.4 Agile ба Scrum

Agile аргачлал нь шаардлага өөрчлөгдөхөд хурдан хариу үйлдэл үзүүлэх, ажиллаж буй бүтээгдэхүүнийг үе шаттай хүргэх, хэрэглэгчийн саналд тулгуурлах зарчимтай [19]. Scrum нь үүнийг sprint, backlog, review, retrospective гэсэн илүү тодорхой процесс болгон хэрэгжүүлдэг [20]. Энэхүү дипломын төсөлд багийн хэмжээ бага боловч feature backlog, богино iteration, incremental delivery зарчим шууд хэрэглэгдэх боломжтой.

1.9.5 Kanban

Kanban нь ажлын урсгалыг харагдуулах, зэрэг эхлүүлсэн ажлыг хязгаарлах, тасралтгүй сайжруулахад төвлөрдөг [21]. Дипломын ажлын нөхцөлд frontend, database, dataset, thesis бичвэр зэрэг олон төрлийн ажил зэрэгцдэг тул Kanban самбар ашиглан *Backlog* → *In progress* → *Review* → *Done* хэлбэрээр хянах нь тохиромжтой.

1.9.6 Lean UX ба хэрэглэгч төвтэй хандлага

Lean UX нь эхлээд том баримт бичиг биш, харин хэрэглэгчийн асуудлын таамаглал, хурдан prototype, туршилт, суралцсан зүйлд тулгуурладаг [22]. Хөгжмийн сургалтын платформын хувьд хэрэглэгч тухайн lesson-оос юу ойлгох, ямар үед AI туслахаас асуух, course card дээр ямар мэдээлэл харвал шийдвэр гаргах зэрэг нь UX туршилтаар тодордог.

1.9.7 Design Science Research

Design Science Research нь мэдээллийн системийн судалгаанд шинэ артефакт бүтээж, түүний хэрэгцээ, зохиомж, үнэлгээг нэгтгэн судлах хандлага юм [23]. Энэхүү дипломын ажил нь зөвхөн онолын судалгаа биш, бодит веб платформ болон RAG dataset pipeline бүтээж байгаа тул DSR-ийн *problem relevance*, *artifact design*, *evaluation*, *research contribution* гэсэн шалгууртай нийцнэ.

ХҮСНЭГТ 1.7: Аргачлалуудын харьцуулсан шинжилгээ

Аргачлал	Давуу тал	Сул тал	Энэ төсөлд тохирох байдал
Waterfall	Баримтжуулалт сайн, үе шат тодорхой	Шаардлага өөрчлөгдөхөд уян хатан бус	Тайлан, шаардлага, зохиомжийг цэгцлэхэд ашиглана.

Prototype	UX болон шаардлагыг эрт шалгана	Prototype кодыг шууд бүтээгдэхүүн болговол өрүүснэ	Landing, course, lesson, chat UI-д тохиромжтой.
Spiral	Эрсдэлд суурилсан, AI туршилтад тохиромжтой	Процесс илүү төвөгтэй, туршлага шаарддаг	Dataset, provider, RAG чанарын эрсдэлийг бууруулахад ашиглана.
Agile/Scrum	Давталттай хөгжүүлэлт, feature backlog, review боломжтой	Бага багт Scrum ceremony бүрэн хэрэгжүүлэх шаардлагагүй	Гол хөгжүүлэлтийн үндсэн аргачлал болгон сонгоно.
Kanban	Ажлын урсгал илтэд, WIP хянахад сайн	Хугацааны хайрцаг сул байж болно	Дипломын олон төрлийн ажлыг зэрэг удирдахад нэмэлтээр ашиглана.
Lean UX	Хэрэглэгчийн урсгал хурдан туршихад сайн	Формаль баримтжуулалт бага	Сургалтын интерфейс, chat interaction-д ашиглана.
DSR	Судалгаа ба бүтээгдэхүүнийг нэгтгэнэ	Үнэлгээний шалгуур тодорхой байх шаардлагатай	Дипломын судалгааны ерөнхий хүрээ болгон ашиглана.

1.10 Сонгосон арга зүйн загвар

Дээрх харьцуулалтаас үзэхэд энэ дипломын төсөлд нэг аргачлал дангаар хангалттай биш. Иймээс **Agile-д суурилсан, Prototype ба Spiral эрсдэлийн хяналтаар баяжуулсан Design Science Research** загварыг сонголоо. Энэ загварын гол санаа нь ажиллаж буй системийг үе шаттай хөгжүүлэхийн зэрэгцээ судалгааны асуудал, артефакт, үнэлгээний логикийг академик байдлаар хадгалах явдал юм.

Хүснэгт 1.8: Сонгосон арга зүйн бүрэлдэхүүн

Арга зүйн бүрэлдэхүүн	Төслийн хэрэглээ	Хүлээгдэж буй үр дүн
Design Science Research	Судалгааны асуудал тодорхойлох, веб платформ ба RAG туслах артефакт бүтээх	Судалгаа ба бүтээгдэхүүний уялдаа баталгаажна.
Agile iteration	Feature backlog үүсгэж, модулийг үе шаттай хэрэгжүүлэх	Landing, course, auth, progress, chat зэрэг модулиуд дараалалтай боловсорно.
Prototype	UX урсгалыг хурдан турших	Course discovery, lesson player, chat drawer ойлгомжтой болно.
Spiral risk review	AI, dataset, provider, security эрсдэлийг давталт бүрд шалгах	RAG хариултын чанар, fallback, citation-safe бодлого сайжирна.
Kanban tracking	Thesis, frontend, database, dataset ажлыг зэрэг хянах	Хөгжүүлэлтийн явц илтэд, удирдах боломжтой болно.

1.11 Арга зүйн үе шат ба ажлын төлөвлөгөө

Сонгосон арга зүйг дипломын ажлын бодит үйл ажиллагаанд дараах үе шаттайгаар хэрэгжүүлнэ.

1. **Судалгааны асуудал тодорхойлох:** FL Studio суралцагчдын контентын тархай байдал, ахицын хяналт, AI туслах хэрэгцээг тодорхойлох.
2. **Ижил төстэй систем судлах:** онлайн сургалтын платформ, хөгжмийн сургалтын эх сурвалж, AI music систем, RAG chatbot загваруудыг харьцуулах.
3. **Шаардлага тодорхойлох:** функциональ ба функциональ бус шаардлагыг backlog хэлбэрээр жагсаах.
4. **Prototype боловсруулах:** landing page, course catalog, lesson detail, chat drawer зэрэг хэрэглэгчийн урсгалыг турших.
5. **Системийн хэрэгжилт:** frontend, backend/API, database, dataset pipeline, AI provider холболтыг үе шаттай хөгжүүлэх.
6. **Эрсдэлийн үнэлгээ:** RAG dataset чанар, source safety, AI хариултын давталт, provider fallback, RLS бодлогыг шалгах.
7. **Баримтжуулалт ба дүгнэлт:** судалгааны үр дүн, зохиомж, архитектур, хэрэгжилт, үнэлгээг дипломын тайланд нэгтгэх.

1.12 Үнэлгээний арга

Энэхүү дипломын ажлын үнэлгээ нь зөвхөн код ажиллаж байгаа эсэхийг шалгах аргаар хязгаарлагдахгүй. Сургалтын платформын хувьд хэрэглэгчийн урсгал, өгөгдлийн бүрэн байдал, AI туслахын хариултын чанар, security policy, өргөтгөх боломж зэрэг олон талаас үнэлэх шаардлагатай.

Хүснэгт 1.9: Үнэлгээний шалгуур

Шалгуур	Үнэлэх зүйл	Баримтлах арга
Функциональ бүрэн байдал	Бүртгэл, курс, хичээл, төлбөр, progress, chatbot ажиллаж байгаа эсэх	Manual scenario test, route бүрийн шалгалт
Өгөгдлийн загвар	Хүснэгтүүдийн relation, unique constraint, RLS бодлого	SQL schema review, Supabase policy review
AI туслахын чанар	Retrieval context, prompt, provider response, fallback	Sample question test, source metadata inspection
UX ойлгомжтой байдал	Course хайх, lesson сонгох, quiz өгөх, chat нээх урсгал	Prototype review, heuristic evaluation
Баримтжуулалт	Судалгаа, арга зүй, зохиомж, архитектурын тайлбар хангалттай эсэх	Thesis chapter review, diagram/table consistency

1.13 Бүлгийн дүгнэлт

Нэгдүгээр бүлгээр төслийн үүсэл, хөгжлийн үе шат, хэрэглэгчийн хэрэгцээ, функциональ болон функциональ бус шаардлагын эхний түвшний шинжилгээ, ижил төстэй системүүдийн судалгаа, систем хөгжүүлэх аргачлалуудын харьцуулалтыг авч үзлээ. Судалгаанаас харахад энэхүү дипломын төсөл нь зөвхөн AI функцтэй туршилтын сайт бус, харин сургалтын платформ, өгөгдлийн сан, төлбөртэй хандалт, RAG туслахыг нэгтгэсэн иж бүрэн веб систем юм.

Арга зүйн хувьд Waterfall дангаараа уян хатан бус, харин Agile, Prototype, Spiral, Kanban, Design Science Research-ийн хосолсон загвар илүү тохиромжтой гэж дүгнэв. Энэ сонголт нь дараагийн бүлгүүдэд авч үзэх технологийн судалгаа, AI онол, платформын хэрэгжилт, системийн архитектурын логик суурь болно.

БҮЛЭГ 2

Технологийн судалгаа ба AI ОНОЛЫН ШИНЖИЛГЭЭ

2.1 Бүлгийн зорилго

Энэ бүлэгт FL Studio болон хөгжим боловсруулалтын сургалтын платформ хөгжүүлэхэд шаардлагатай веб технологи, өгөгдлийн сан, authentication, frontend архитектур, AI туслах, RAG, аудио AI, dataset чанарын онолын үндсийг судална. Нэгдүгээр бүлэгт систем хөгжүүлэх арга зүйн сонголтыг тодорхойлсон бол энэ бүлэг нь “*ямар технологи, ямар AI онол дээр тулгуурлан уг системийг хэрэгжүүлэх вэ?*” гэсэн асуултад хариулна.

Төслийн технологийн судалгааны хүрээг дараах байдлаар ангиллаа.

- веб платформын client/server rendering, route зохион байгуулалт, component-based UI;
- хэрэглэгчийн бүртгэл, өгөгдлийн сан, row-level security, төлбөр ба явцын өгөгдөл;
- AI туслахын retrieval, prompt construction, provider abstraction, fallback logic;
- хөгжим болон аудио AI-ийн онолын чиглэлүүд;
- RAG dataset-ийн бүтэц, эх сурвалжийн найдвартай байдал, ёс зүй ба зохиогчийн эрхийн эрсдэл.

2.2 Веб платформ хөгжүүлэх технологийн судалгаа

2.2.1 Next.js ба React архитектур

Энэхүү төсөл нь Next.js 14, React 18, TypeScript дээр суурилсан [24, 25]. Next.js App Router нь page, layout, API route, dynamic route зэргийг нэг framework дотор зохион байгуулах боломжтой. FL Studio сургалтын платформын хувьд /, /courses, /courses/[slug], /dashboard, /admin, /chat, /checkout зэрэг олон route шаардлагатай тул App Router тохиромжтой.

React-ийн component-based хандлага нь UI-г жижиг, дахин ашиглагдах блок болгон салгадаг. Жишээлбэл, course card, pricing card, video player, chat drawer, navigation, dashboard layout зэрэг компонент нь тус бүр өөр хариуцлагатай. Энэ нь сургалтын платформын интерфэйсийг засварлах, шинэ курсийн card нэмэх, chat drawer-ийг олон хуудсанд ашиглахад давуу талтай.

2.2.2 TypeScript-ийн үүрэг

TypeScript нь JavaScript дээр static type checking нэмдэг. Сургалтын платформд Course, Lesson, User, ChatMessage, AudioFile, Payment зэрэг өгөгдлийн хэлбэр олон тул typed interface ашиглах нь runtime алдааг бууруулна. Frontend/src/lib/types.ts болон Frontend/src/types/index.ts файлууд нь курс, lesson, чат, аудио файл, API response зэрэг өгөгдлийг тодорхойлж байна.

TypeScript-ийн давуу тал нь:

- component хооронд дамжих props-ийн бүтцийг тодорхой болгох;
- Supabase-аас ирэх өгөгдлийг frontend model-д хөрвүүлэх үед алдаа багасгах;
- API response, chat source metadata зэрэг объектын хэлбэрийг тогтвортой хадгалах;
- томрох тусам кодын засварлах чадварыг нэмэгдүүлэх.

2.2.3 Tailwind CSS ба UI зохион байгуулалт

Frontend хэсэгт Tailwind CSS ашигласан [26]. Tailwind нь utility-first CSS хандлагатай бөгөөд layout, spacing, color, typography, responsive style-ийг component дотор тодорхойлох боломж олгодог. Сургалтын платформын хувьд course catalog, lesson player, dashboard, chat drawer зэрэг олон UI төлөвтэй тул class-based хурдан загварчлал тохиромжтой.

Гэхдээ Tailwind ашиглахад class name урт болох, design token-ууд тархах, reusable component-ийн boundary тодорхойгүй болох эрсдэлтэй. Иймээс Button, SectionLabel, CourseCard, PricingCard зэрэг давтагдах UI-г компонент болгон салгах нь зохистой.

2.3 Өгөгдлийн сан ба хэрэглэгчийн эрхийн технологи

2.3.1 Supabase ба PostgreSQL

Supabase нь PostgreSQL өгөгдлийн сан, authentication, storage, API, row-level security зэрэг боломжийг нэг платформд нэгтгэдэг [27]. Энэ дипломын төслийн хувьд Supabase-г хэрэглэгчийн profile, course, lesson, purchased course, completed lesson, course progress, chat message, knowledge base, payment зэрэг өгөгдлийг хадгалахад ашиглаж байна.

PostgreSQL нь relational data model-д тохиромжтой. Course ба Lesson, User ба Progress, User ба PurchasedCourse, ChatMessage ба KnowledgeBase зэрэг холбоосууд нь relational schema-д байгалийн байдлаар дүрслэгдэнэ. Мөн unique constraint, foreign key, timestamp, jsonb, vector field зэрэг төрлүүдийг ашиглаж болох нь AI туслахтай сургалтын платформд хэрэгтэй.

2.3.2 Нэвтрэлт ба эрхийн хяналт

Сургалтын платформд хэрэглэгчийн хувийн өгөгдөл, худалдан авалт, progress, chat history зэрэг эмзэг мэдээлэл орно. Иймээс authentication нь хэрэглэгчийг таних, authorization нь тухайн хэрэглэгч ямар өгөгдөлд хандах эрхтэйг тогтоох үүрэгтэй. Supabase Auth нь хэрэглэгчийн бүртгэл, нэвтрэлт, session удирдлагыг хангана.

Row Level Security (RLS) нь хүснэгтийн түвшинд мөр бүрийн хандалтын дүрэм тогтоодог. Жишээлбэл, хэрэглэгч зөвхөн өөрийн purchased_courses, completed_lessons, course_progress, chat_messages-ийг харах ёстой. Харин админ хэрэглэгч course, lesson, knowledge base зэрэг нийтийн өгөгдлийг удирдах эрхтэй байна.

Хүснэгт 2.1: Өгөгдлийн технологийн сонголтын үндэслэл

Технологи	Давуу тал	Төслийн хэрэглээ
PostgreSQL	Relational schema, constraint, transaction, jsonb, vector extension ашиглах боломж	Course, lesson, progress, payment, knowledge base хадгалах
Supabase Auth	Бүртгэл, session, email/password authentication хурдан хэрэгжүүлэх	Login, register, profile, admin role
Row Level Security	Өгөгдлийн мөрийн түвшний хандалтын дүрэм	Хэрэглэгч зөвхөн өөрийн progress, purchase, chat-г харах
Supabase client	Frontend-ээс typed query хийхэд хялбар	Course list, purchase status, admin data query

2.4 AI туслах системийн онолын үндэс

2.4.1 Transformer архитектур

Орчин үеийн том хэлний загваруудын үндэс нь Transformer архитектур юм [1]. Transformer нь self-attention механизмаар өгүүлбэрийн доторх үг, token хоорондын хамаарлыг зэрэгцээ байдлаар тооцдог. Энэ нь урт context, олон утгатай нэр томьёо, асуулт-хариултын даалгаварт хүчтэй.

FL Studio сургалтын туслахын хувьд хэрэглэгчийн асуулт нь ихэвчлэн Mongolian болон English нэр томьёо холилдсон байдаг. Жишээлбэл, “kick bass muddy байна”, “limiter dynamic range алдагдуулаад байна”, “Piano Roll дээр chord яаж бичих вэ?” гэх мэт. Transformer-д суурилсан хэлний загвар нь ийм холимог хэллэгийн утгыг ойлгох чадвартай боловч domain-specific мэдлэггүй бол ерөнхий, буруу эсвэл хэт итгэлтэй хариу өгөх эрсдэлтэй.

2.4.2 Том хэлний загвар

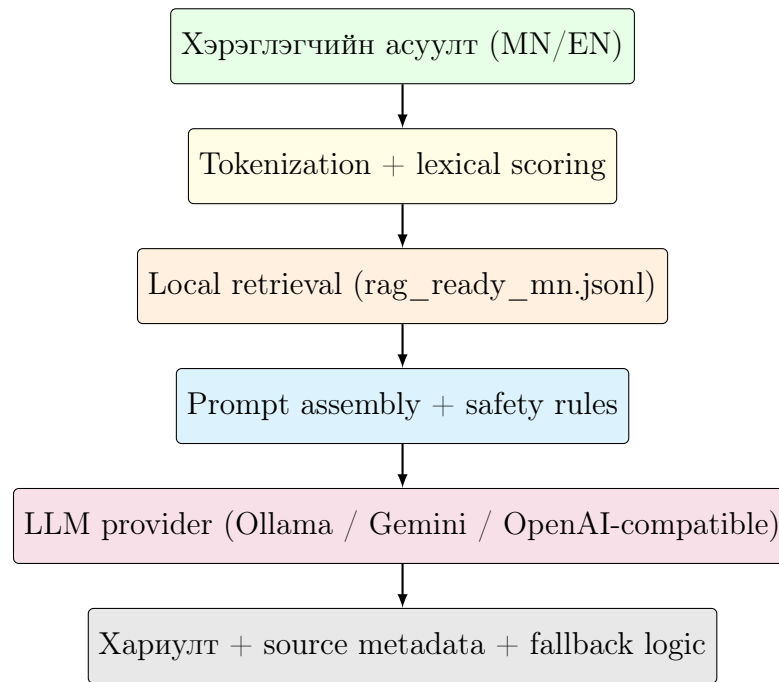
GPT-3, LLaMA зэрэг том хэлний загварууд нь их хэмжээний текст дээр сурч, few-shot learning, instruction following, natural language generation зэрэг чадварыг харуулсан [4, 5]. Эдгээр загвар нь сургалтын чатботод уян хатан тайлбар үүсгэх боломж олгоно. Гэвч тэдний parametric memory нь тухайн сургалтын платформын lesson, dataset, багшийн агуулгыг автоматаар мэдэхгүй. Иймээс external context ашиглах шаардлага гарна.

2.4.3 Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) нь хэрэглэгчийн асуултад хариулахын өмнө гаднын мэдлэгийн сангаас холбогдох context хайж, түүнийг prompt-д оруулан хэлний загвараар хариу үүсгэдэг арга юм [2]. Энэ хандлага нь parametric memory-д бүрэн найдахаас илүү найдвартай бөгөөд domain-specific сургалтын системд тохиромжтой.

RAG-ийн үндсэн алхам:

1. хэрэглэгчийн асуултыг token эсвэл embedding хэлбэрт оруулах;
2. мэдлэгийн сангаас хамгийн ойр context сонгох;
3. сонгосон context-ийг prompt-д нэгтгэх;
4. хэлний загвараар хариу үүсгэх;
5. хариултын эх сурвалж, чанар, fallback нөхцөлийг шалгах.



ЗУРАГ 2.1: RAG суурьтай AI туслахын онолын стек

2.4.4 Sparse, lexical ба dense сэргээн хайлт

Retrieval хийхэд lexical буюу үгийн давхцалд суурилсан арга, dense embedding-д суурилсан арга гэсэн хоёр ерөнхий хандлага байдаг. Lexical арга нь хэрэгжүүлэхэд хялбар, ил тод, жижиг dataset-д тохиромжтой. Харин synonym, paraphrase, morphology ихтэй үед recall буурч болно. Dense Passage Retrieval нь асуулт ба context-ийг embedding орон зайд оруулж, утгын ойролцоогоор хайдаг [3]. Sentence-BERT, SimCSE, ColBERT зэрэг ажлууд нь semantic similarity retrieval-ийн өөр өөр чиглэлийг төлөөлдөг [28, 29, 30].

Одоогийн дипломын хэрэгжилт lexical scoring ашиглаж байгаа нь MVP шатанд тохиромжтой. Харин цаашид Supabase pgvector, HNSW индекс, embedding model ашиглан dense retrieval рүү шилжих боломжтой [31].

2.5 AI үйлчилгээний abstraction давхарга

AI provider abstraction гэдэг нь системийг нэг л LLM үйлчилгээтэй хатуу холбохгүй, Ollama, Gemini, OpenAI-compatible API зэрэг олон provider-ийг сонгох боломжтойгоор зохиомжлохыг хэлнэ. Энэ нь зардал, latency, availability, privacy, чанарын хувьд уян хатан байдал өгдөг.

AI provider abstraction нь системийн архитектурын хувьд чухал. Учир нь сургалтын платформын үнэ цэнэ зөвхөн нэг model-д хэт хамаарвал provider unavailable болох, үнэ өөрчлөгдөх, чанар тогтворгүй болох үед системийн

Хүснэгт 2.2: AI provider сонголтын харьцуулалт

Provider төрөл	Давуу тал	Сул тал	Төслийн байр суурь
Ollama/local	Өгөгдөл локал, туршихад зардал бага	Жижиг model чанар тогтворгүй, hardware хамааралтай	Development ба offline demo-д тохиромжтой
Gemini	Cloud model, чанар ба хурд сайн	API key, зардал, availability хамааралтай	Чанартай demo болон production candidate
OpenAI-compatible	OpenAI, OpenRouter, Groq зэрэг олон API-д холбогдох боломжтой	Provider бүрийн model, үнэ, policy ялгаатай	Vendor lock-in бууруулах архитектур

үндсэн хэрэглээ саатна. Иймээс provider сонголтыг environment variable-аар удирдах, response чанарыг шалгах, fallback хэрэглэх нь практик инженерчлэлийн зөв шийдэл юм.

2.6 Хөгжим ба аудио AI-ийн судалгааны чиглэлүүд

2.6.1 Хөгжмийн ангилал ба аудио төлөөлөл

Хөгжмийн мэдээлэл боловсруулах судалгаанд audio representation learning чухал байр суурьтай. CRNN архитектур нь spectrogram дээр convolutional болон recurrent давхаргыг хослуулан хөгжмийн ангилал хийх боломжтойг харуулсан [32]. PANNs нь том хэмжээний audio pretraining ашиглан олон төрлийн audio pattern recognition даалгаварт дамжуулан суралцах боломжтойг харуулсан [33]. Ийм төрлийн судалгаанууд нь цаашид хэрэглэгчийн upload хийсэн audio дээр автомат feedback өгөх боломжийн онолын үндэс болно.

2.6.2 Эх үүсвэр салгалт

Mixing сургалтын хувьд kick, bass, vocal, drums, harmonic element зэрэг эх үүсвэрийг тусгаарлан ойлгох нь чухал. Wave-U-Net, Demucs, Hybrid Transformer Demucs зэрэг судалгаанууд нь waveform болон hybrid domain дээр source separation хийх боломжийг ахиулсан [34, 35, 36]. Энэ дипломын одоогийн

хүрээнд source separation бүрэн хэрэгжээгүй боловч ирээдүйн audio analysis модулийн судалгааны суурь болно.

2.6.3 Текстээс хөгжим үүсгэх ба аудио генераци

Jukebox, AudioLM, MusicLM, MusicGen зэрэг ажлууд нь raw audio, audio token, text-to-music generation чиглэлд том ахиц гаргасан [7, 8, 9, 10]. Гэхдээ эдгээр системийг сургалтын платформд шууд ашиглахад тооцооллын зардал, зохиогчийн эрх, training data provenance, хэрэглэгчийн хяналт зэрэг асуудал үүснэ. Иймээс энэхүү дипломын ажилд AI-г *хөгжим автоматаар үүсгэгч* бус, *сургалтын зөвлөгч* хэлбэрээр ашигласан нь эрсдэл багатай.

2.6.4 Яриа болон аудио-текстийн хам төлөөлөл

Whisper, wav2vec 2.0 зэрэг ажлууд нь speech/audio representation суралцахад хүчтэй үр дүн үзүүлсэн [11, 12]. MuLan нь music audio болон natural language-ийг хам embedding орон зайд байрлуулах боломжийг харуулсан [37]. Эдгээр чиглэлүүд нь цаашид “хэрэглэгч өөрийн beat-ийг upload хийгээд AI-аар тайлбар авах” төрлийн функцэд онолын үндэс өгнө.

2.7 RAG dataset ба өгөгдлийн чанарын онол

RAG системийн чанар нь зөвхөн LLM model-оос бус, мэдлэгийн сангийн чанараас шууд хамаарна. Энэ төслийн rag-dataset-builder нь FL Studio, music theory, mixing, mastering зэрэг сэдвийн Монгол тайлбаруудыг JSONL/CSV хэлбэрээр бэлтгэх зорилготой. Dataset pipeline нь эх сурвалжаас бүтэн текст хуулбарлахгүй, харин metadata болон богино topic hint ашиглан өөрийн үгээр боловсруулсан тайлбар үүсгэх зарчимтай.

Эндээс харахад RAG dataset нь сургалтын туслахын хэрэглээнд үнэ цэнтэй боловч академик ишлэл эсвэл албан ёсны manual-ийн оронд шууд хэрэглэхэд хязгаарлалттай. Иймээс chatbot хариултад source metadata буцаах, citation-safe flag ашиглах, generated/internal guidance-ийг ялгах нь зайлшгүй.

Хүснэгт 2.3: RAG dataset чанарын үндсэн үзүүлэлт

Үзүүлэлт	Одоогийн dataset-ийн төлөв
Ready dataset records	4990 орчим JSONL бичлэг
Гол ангилал	Mixing, Mastering, FL Studio Basics, Music Theory
Citation-safe records	Цөөн; source-grounded бичлэгийг тусад нь ялгах шаардлагатай
Synthetic/internal guidance	Ихэнх бичлэг нь local/generated зөвлөмж хэлбэртэй
Production хэрэглээний анхаарах зүйл	Generated guidance-ийг албан ёсны эх сурвалж мэт харуулахгүй байх

2.8 Технологийн шаардлага ба шийдлийн уялдаа

Технологи сонгохдоо зөвхөн шинэ, түгээмэл framework ашиглах нь хангалтгүй. Сургалтын платформын зорилго, хэрэглэгчийн урсгал, өгөгдлийн эзэмшил, AI туслахын найдвартай байдал, дипломын төслийн хугацаа зэрэг хүчин зүйлсийг хамтад нь авч үзэх шаардлагатай. Иймээс технологийн судалгааг шаардлага-шинжилгээний үр дүнтэй холбон тайлбарлах нь академик үндэслэлийг нэмэгдүүлнэ.

Хүснэгт 2.4: Шаардлага ба технологийн сонголтын уялдаа

Шаардлагын чиглэл	Технологийн хэрэгцээ	Сонгосон шийдэл	Үндэслэл
Олон хуудаст сургалтын урсгал	Route, layout, dynamic page, server-side logic	Next.js App Router	Course catalog, lesson detail, admin, chat, API route-ууд нэг framework-д багтана.

UI төлөв ба component дахин ашиглалт	Component model, typed props, client state	React ба TypeScript	CourseCard, LessonPlayer, ChatDrawer зэрэг UI-г тусгаарлахад тохиромжтой.
Хэрэглэгчийн бүртгэл ба өгөгдөл хамгаалалт	Authentication, authorization, row-level policy	Supabase Auth ба RLS	User-owned progress, purchase, chat history-г database түвшинд хамгаална.
AI туслахын context-aware хариулт	Retrieval, prompt assembly, provider call	RAG + LLM provider layer	Lesson болон FL Studio мэдлэгийн сантай холбоотой хариулт үүсгэнэ.
Dataset хөгжүүлэлт	Source extraction, generation, validation, import	RAG dataset builder scripts	AI knowledge base-г frontend кодоос тусад нь сайжруулах боломжтой.
Цаашдын audio analysis	File upload, analysis table, model API	Supabase schema ба service layer	Одоогийн MVP-д бүрэн ашиглаагүй ч архитектурын өргөтгөлийн суурь болно.

2.9 Client-server boundary ба rendering онол

Орчин үеийн веб application-д бүх логикийг browser дээр ажиллуулах эсвэл бүх логикийг server дээр ажиллуулах хоёр туйлын аль нь ч дангаар хангалттай биш. Сургалтын платформд хэрэглэгчийн interaction ихтэй хэсэг, өгөгдөл

хамгаалах хэсэг, AI provider-той харилцах хэсэг тус бүр өөр boundary шаарддаг.

Client талд course card hover, lesson tab, chat drawer open state, quiz answer state зэрэг шууд interaction байрлах нь хэрэглэгчийн мэдрэмжийг хурдан болгоно. Харин AI API key, provider сонголт, dataset path, prompt assembly зэрэг sensitive logic server-side API route-д байх ёстой. Ийм хуваарилалт нь security болон maintainability-ийг сайжруулна.

Next.js App Router-ийн хувьд page, layout, route handler гэсэн ойлголтуудыг ялгах нь чухал. Page нь хэрэглэгчийн харах UI-г тодорхойлно. Layout нь navigation болон нийтлэг visual structure-ийг тогтвортой хадгална. Route handler нь HTTP endpoint байдлаар ажиллаж, AI, YouTube metadata, payment webhook зэрэг server-side logic-д ашиглагдана. Энэ төсөлд /api/chat route нь ийм boundary-ийн гол жишээ болно.

ХҮСНЭГТ 2.5: Client ба server талын логикийн ялгаа

Логикийн төрөл	Client талд байрлах шалтгаан	Server талд байрлах шалтгаан
UI interaction	Шууд feedback, animation, local state хэрэгтэй	Зөвхөн UI төлөв тул server шаардахгүй.
Authentication state	Session-ийн UI нөлөө, redirect хийх хэрэгтэй	Token validation, RLS бодит хамгаалалт database талд хэрэгжинэ.
Course filtering	Жижиг dataset-д хурдан filter хийх боломжтой	Олон course үед server-side search хэрэгтэй болно.
AI prompt assembly	Client дээр ил болвол prompt болон key алдагдах эрсдэлтэй	Provider key, context selection, fallback-г хамгаална.
Payment confirmation	UI төлөв харуулна	Webhook, receipt, fraud check server талд байх ёстой.

2.10 Өгөгдлийн загварчлал ба RLS онол

Relational database нь сургалтын платформд course, lesson, user, purchase, progress зэрэг entity хоорондын холбоог тодорхой илэрхийлэхэд тохиромжтой. Энд нэг хэрэглэгч олон course худалдан авч болно, нэг course олон lesson-той байна, нэг lesson олон хэрэглэгчийн completion бичлэгтэй болно.

Ийм холбоог frontend state-д хадгалах нь өгөгдөл давхардах, consistency алдагдах эрсдэлтэй. Иймээс PostgreSQL schema нь системийн truth source байх ёстой.

Row Level Security буюу RLS нь multi-user application-д хэрэглэгч бүр зөвхөн өөрийн өгөгдлийг харах, өөрчлөх боломжтой болгоно. Application code-д алдаа гарсан ч database policy зөв байвал user-owned data хамгаалагдана. Энэ нь дипломын төслийн хувьд онолын хувьд чухал. Учир нь progress, purchase, chat history зэрэг нь хэрэглэгчийн хувийн өгөгдөлд хамаарна.

Хүснэгт 2.6: RLS бодлогын онолын ангилал

Бодлогын төрөл	Зорилго	Энэ төсөлд хэрэглэх жишээ
Owner-based access	Хэрэглэгч өөрийн мөрийг л харах	Progress, purchase, chat message, audio file
Public read access	Нийтийн контентийг login шаардахгүй унших	Active course, lesson preview, knowledge base select
Admin manage access	Зөвхөн админ өгөгдөл удирдах	Course CRUD, lesson CRUD, knowledge base import
Insert with check	Хэрэглэгч өөрийн user id-тэй бичлэг л үүсгэх	completed lesson, purchased course, chat message
Function-based policy	Давтагдах эрх шалгалтыг helper function болгох	admin role шалгах function

2.11 Prompt engineering ба hallucination бууруулах онол

LLM ашигласан сургалтын системийн гол эрсдэл нь буруу эсвэл баталгаагүй зөвлөгөө өгөх явдал юм. Энэ эрсдэлийг бүрэн арилгах боломжгүй боловч prompt design, retrieval context, fallback answer, source metadata, answer boundary зэрэг аргаар бууруулж болно. RAG аргачлал нь үүнд онолын суурь өгдөг. Учир нь model-ийн parametric memory-д найдах бус, тухайн домайн мэдлэгийн сангаас сонгосон context-д тулгуурлан хариу үүсгэнэ.

Энэ төслийн AI туслахын prompt нь гурван зорилготой байх ёстой. Нэгдүгээрт, хариултыг FL Studio болон music production сургалтын хүрээнд барих. Хоёрдугаарт, хэрэглэгч эхлэн суралцагч байж болох тул тайлбарыг алхамчилсан, ойлгомжтой болгох. Гуравдугаарт, dataset context хангалтгүй үед итгэлтэй мэт худал хариулахгүйгээр ерөнхий зөвлөмж эсвэл тодруулах асуулт өгөх.

Хүснэгт 2.7: AI хариултын эрсдэл ба бууруулах арга

Эрсдэл	Шалтгаан	Бууруулах арга
Hallucination	Dataset context бага эсвэл prompt тодорхойгүй	Context-grounded prompt, fallback, source metadata
Хэт ерөнхий хариулт	Асуулт тодорхой бус, lesson context дамжаагүй	Course/lesson metadata нэмэх, follow-up question өгөх
Буруу түвшний тайлбар	Эхлэгч ба ахисан хэрэглэгчийн ялгаа ороогүй	Level metadata ашиглах, beginner-friendly style rule
Provider availability	External API quota, network, local model failure	Provider abstraction, fallback answer, retry policy
Citation misuse	Synthetic entry-г official source мэт харуулах	citation-safe flag, source type ялгах

2.12 Аудио өгөгдөл ба ирээдүйн AI боломж

Одоогийн системийн гол AI функц нь text-based RAG туслах боловч төслийн нэр, schema, service layer нь audio processing чиглэлд өргөжих боломжтой. Хөгжим боловсруулалтын платформд ирээдүйд хэрэглэгч өөрийн beat, loop, vocal recording, mix bounce upload хийж, автомат feedback авах хэрэгцээ үүснэ. Ийм функцэд waveform, spectrogram, loudness, frequency balance, tempo, transient, stereo image зэрэг audio feature ашиглагдана.

Audio feature extraction-ийн хамгийн энгийн түвшин нь signal статистик юм. Жишээлбэл peak level, RMS, loudness, duration, silence ratio зэрэг хэмжүүр mix clipping, дууны түвшин, export quality-г урьдчилан шалгахад хэрэглэгдэнэ. Дараагийн түвшинд spectrogram болон Mel-frequency cepstral coefficient зэрэг frequency-domain feature ашиглаж timbre, instrument, vocal clarity зэрэг шинжийг судалж болно. Илүү ахисан түвшинд pretrained audio model

ашиглан genre, instrument, mood, source separation, mastering suggestion хийх боломжтой.

Хүснэгт 2.8: Ирээдүйн audio AI модулийн боломжит түвшин

Түвшин	Боловсруулах өгөгдөл	Сургалтын плат-формд өгөх үр ашиг
Signal statistics	Duration, peak, RMS, silence	Export болон gain staging зөвлөгөө
Spectral analysis	Frequency balance, spectrogram	EQ, muddiness, harshness тайлбар
Source analysis	Vocal, drums, bass, harmony layer	Mixing decision болон arrangement feedback
Generative support	Melody variation, chord idea	Бүтээлч санаа өгөх боловч хүний хяналттай ашиглах

2.13 Технологийн эрсдэлийн судалгаа

Сонгосон технологи бүр давуу талтай боловч дипломын төслийн хүрээнд тодорхой эрсдэл дагуулна. Frontend framework хурдан хөгждөг тул version compatibility анхаарах шаардлагатай. Supabase нь RLS болон PostgreSQL давуу талтай боловч policy буруу бичигдвэл өгөгдөл нээлттэй болох эрсдэлтэй. LLM provider ашиглахад cost, latency, privacy, availability зэрэг асуудал гарна. Dataset generation хийвэл чанарын хяналт, эх сурвалжийн ялгалт, давхардал, хэлзүйн засвар зайлшгүй хэрэгтэй.

Хүснэгт 2.9: Технологийн эрсдэл ба удирдлагын арга

Технологи	Эрсдэл	Удирдлагын арга
Next.js, React	Client/server boundary буруу тогтох, bundle томрох	Route logic, component logic, API logic-ийг тусгаарлах
TypeScript	Type definition хуучрах эсвэл any ихдэх	Shared type, strict checking, reusable interface
Supabase	RLS policy алдаа, migration drift	SQL review, admin helper function, test user scenario

RAG dataset	Synthetic content чанар жигд бус, citation асуу- дал	Validation script, source type, citation-safe flag
LLM provider	Cost, latency, quota, privacy	Provider abstraction, local model option, fallback rule
Audio AI future	Тооцооллын зардал, model explainability	Feature-based MVP, human- readable feedback, staged rollout

2.14 Security, privacy ба ёс зүйн технологийн су- далгаа

AI туслахтай сургалтын платформд security болон privacy нь зөвхөн нэмэлт шаардлага бус, үндсэн архитектурын нөхцөл юм. Хэрэглэгчийн profile, purchase, progress, chat history, ирээдүйн audio upload зэрэг нь хувийн өгөгдөлд хамаарна. Ийм өгөгдөл frontend дээр ил задгай хадгалагдах, API key client bundle-д орох, RLS policy сул байх, AI provider руу хэт их хувийн context илгээх зэрэг нь эрсдэл үүсгэнэ.

Privacy by design хандлагаар авч үзвэл system design хийх эхний үеэс л өгөгдлийг багасгах, access control хийх, purpose limitation баримтлах, user-owned data-г тусгаарлах хэрэгтэй. Энэ төслийн хувьд хэрэглэгчийн progress болон purchase мэдээлэл зөвхөн тухайн хэрэглэгчид харагдах ёстой. Chat message нь сургалтын чанар сайжруулахад ашиглагдаж болох боловч хэрэглэгчийн хувийн мэдээлэл агуулж болзошгүй тул analytics хийхдээ anonymization шаардлагатай.

Хүснэгт 2.10: Security ба privacy-ийн технологийн шаардлага

Чиглэл	Эрсдэл	Технологийн хамгаалалт
Authentication	Хуурамч хэрэглэгч, session алдагдах	Supabase Auth, session refresh, protected route
Authorization	Өөр хэрэглэгчийн progress харах	RLS owner policy, admin helper function
API key privacy	LLM key browser bundle- д орох	Server-side API route, environment variable

Chat privacy	Хувийн мэдээлэл provider руу дамжих	Context minimization, prompt sanitization future
Dataset ethics	Generated entry-г official source мэт ашиглах	Source type, citation-safe flag, human review
Audio upload future	Хэрэглэгчийн бүтээл ал- дагдах	User-owned storage policy, signed URL, delete control

2.15 UX, accessibility ба сургалтын технологийн судалгаа

Сургалтын платформын технологи зөвхөн backend болон AI-аар хэмжигдэхгүй. Суралцагчийн ойлгох хурд, хичээл сонгох амар байдал, lesson player-ийн төвлөрөл, chat туслахын цаг үеэ олсон байрлал зэрэг нь UX quality-д шууд нөлөөлнө. Хөгжим боловсруулалт суралцагчид ихэвчлэн video үзэх, DAW дээр зэрэг турших, нэр томьёо шалгах, богино зөвлөгөө авах зэрэг олон үйлдлийг зэрэг хийдэг. Иймээс UI нь олон мэдээлэлтэй боловч хэт сарниулахгүй байх шаардлагатай.

Accessibility талаас keyboard navigation, contrast, responsive layout, semantic button, form label, loading state, error state зэрэг нь анхаарах зүйлс юм. Chat drawer эсвэл floating button нь хэрэглэгчийн lesson content-ийг хаахгүй байх, mobile viewport дээр overflow үүсгэхгүй байх хэрэгтэй. Academic хувьд энэ нь usability requirement-ийн хэрэгжилтийн нэг хэсэг гэж үзнэ.

Хүснэгт 2.11: UX ба accessibility технологийн шалгуур

Шалгуур	Тайлбар	Төслийн жишээ
Discoverability	Курс, түвшин, үнэ, duration ойлгомжтой байх	CourseCard, catalog filter
Focused learning	Lesson үзэх үед илүүдэл UI багасгах	LessonPlayer, video area
Timely help	Асуулт гарсан мөчид тусламж авах	ChatDrawer, chat page
Responsive layout	Mobile болон desktop дээр эвдрэхгүй байх	Tailwind responsive class
Accessible control	Button, form, link-ийн semantic хэрэглээ	Auth, checkout, admin form

2.16 AI үнэлгээний онол ба хэмжүүр

AI туслахыг үнэлэхдээ зөвхөн “хариулт өгсөн эсэх”-ээр хэмжих нь хангалтгүй. Сургалтын системд answer relevance, factual grounding, source usefulness, clarity, level appropriateness, safety, latency зэрэг олон хэмжүүр хэрэгтэй. RAG системийн хувьд retrieval recall болон answer faithfulness хамгийн чухал. Хэрэв зөв context сонгогдоогүй бол LLM сайн байсан ч хариулт буруу эсвэл ерөнхий болно. Хэрэв context зөв боловч model хэт их өөрийн таамаг нэмбэл faithfulness буурна.

Хүснэгт 2.12: AI туслахын үнэлгээний боломжит хэмжүүр

Хэмжүүр	Тайлбар	Шалгах арга
Retrieval relevance	Сонгосон context асуулттай холбоотой эсэх	Sample query review
Answer grounding	Хариулт context-д тулгуурласан эсэх	Source-answer comparison
Clarity	Эхлэгч хэрэглэгч ойлгохуйц эсэх	Human review
Level fit	Beginner, intermediate ялгаа тохирсон эсэх	Level-tagged test set
Safety	Буруу, эрсдэлтэй, хэт итгэлтэй зөвлөгөө багассан эсэх	Failure case test
Latency	Хариу өгөх хугацаа боломжийн эсэх	API timing log

2.17 Deployment ба configuration технологийн судалгаа

Web platform хөгжүүлэхэд local development, demo, production орчны ялгаа тодорхой байх шаардлагатай. Local орчинд environment variable, Supabase project URL, anon key, LLM provider key, local Ollama endpoint зэрэг тохиргоо ашиглагдана. Demo орчинд зарим provider unavailable байж болох тул fallback logic хэрэгтэй. Production орчинд key rotation, deployment log, error monitoring, HTTPS, domain, backup, migration control шаардлагатай.

Configuration management-ийн гол зарчим нь secret утгыг source code-д хадгалахгүй байх явдал юм. `.env.example` нь ямар хувьсагч шаардлагатайг баримтжуулахад хэрэгтэй боловч бодит key хадгалах ёсгүй. Next.js build-time

болон runtime environment-ийн ялгааг зөв ойлгохгүй бол provider key client bundle-д алдагдах эрсдэлтэй. Иймээс AI provider call server-side API route-д байрлах нь зөв шийдэл болно.

Хүснэгт 2.13: Deployment configuration-ийн үндсэн ангилал

Тохиргоо	Зориулалт	Анхаарах зүйл
Supabase URL, anon key	Frontend client болон Auth холболт	Public key ба service key-г ялгах
LLM provider key	Gemini/OpenAI-compatible provider call	Зөвхөн server-side ашиглах
Ollama endpoint	Local model demo	Network болон model availability шалгах
Dataset path	Local JSONL retrieval	Production үед DB/vector store руу шилжих
Payment config future	Gateway webhook, secret	Signature verification хэрэгтэй
Build command	Frontend болон thesis build	CI/CD-д давтагдах боломжтой болгох

2.18 Технологийн сонголтын нэгтгэл

Дээрх судалгаан дээр үндэслэн төслийн технологийн сонголтыг дараах байдлаар нэгтгэв.

Хүснэгт 2.14: Технологийн сонголтын нэгтгэл

Давхарга	Сонгосон технологи	Үндэслэл	Эрсдэл ба хяналт
Frontend	Next.js, React, TypeScript	Олон route, component, typed data model-д тохиромжтой	Component давхардал, UI state complexity → reusable component, hook ашиглах

Styling	Tailwind CSS	Prototype хурдан, responsive UI хийхэд хялбар	Utility class уртсах → UI component салгах
Backend/API	Next.js API routes	Frontend-тэй нэг codebase-д chatbot, youtube duration route удирдах	API logic томрох → route-level separation
Database	Supabase PostgreSQL	Auth, relational schema, RLS, vector extension боломжтой	Policy алдаа → RLS review, admin function
AI	RAG + LLM provider abstraction	Context-grounded answer, provider flexibility	Dataset чанар, hallucination → source metadata, fallback
Dataset	JSONL/CSV pipeline	Embedding, Supabase import, human review-д тохиромжтой	Synthetic content их → citation-safe ялгах

2.19 Бүлгийн дүгнэлт

Хоёрдугаар бүлгээр төслийн технологийн болон AI онолын үндсийг судаллаа. Судалгаанаас Next.js, React, TypeScript, Tailwind CSS нь сургалтын веб интерфэйсийг модульчлан хөгжүүлэхэд тохиромжтой; Supabase PostgreSQL нь хэрэглэгч, курс, хичээл, progress, chat, knowledge base зэрэг өгөгдлийг найдвартай удирдах боломжтой; RAG болон provider abstraction нь AI туслахыг үндсэн сургалтын контентыг орлохгүйгээр, харин context-aware дэмжлэг хэлбэрээр хэрэгжүүлэхэд зохистой болох нь харагдлаа.

Мөн аудио AI-ийн судалгаанууд нь ирээдүйн өргөтгөлийн чиглэлийг тодорхойлж байгаа боловч одоогийн дипломын хүрээнд AI-г бүрэн автомат хөгжмийн генераци бус, сургалтын зөвлөгчийн үүргээр ашиглах нь илүү найдвартай гэж дүгнэв. Энэ бүлгийн технологийн сонголт нь гуравдугаар бүлгийн платформын дизайн ба хэрэгжилтийн тайлбарт шууд суурь болно.

БҮЛЭГ 3

Платформын дизайн ба хөгжүүлэлт

3.1 Бүлгийн зорилго

Энэ бүлэгт FL Studio болон хөгжим боловсруулалтын сургалтын платформын бодит дизайн, frontend, backend, database, AI туслах, RAG dataset pipeline-ийн хэрэгжилтийг тайлбарлана. Нэгдүгээр бүлэгт арга зүй, хоёрдугаар бүлэгт технологи ба AI онолыг судалсан бол энэ бүлэг нь тэдгээр сонголтыг бодит кодын баазад хэрхэн хэрэгжүүлснийг харуулна.

Системийн хэрэгжилт нь дараах үндсэн бүрэлдэхүүнтэй.

- Next.js App Router дээр суурилсан frontend application;
- course catalog, course detail, lesson player, dashboard, checkout, admin зэрэг UI урсгал;
- Supabase Auth, PostgreSQL schema, RLS policy, purchase/progress өгөгдлийн давхарга;
- /api/chat route дээр суурилсан RAG chatbot;
- rag-dataset-builder pipeline болон ready dataset;
- хэрэглэгчийн асуултад source metadata бүхий AI хариу буцаах chat UI.

3.2 Төслийн файлын бүтэц

Дипломын ажлын бүх хавтас нэг төслийн бүрэлдэхүүн гэж үзэгдэнэ. Frontend хавтас нь веб application, dataset болон rag-dataset-builder хавтас нь AI мэдлэгийн сан, diplomapaper хавтас нь судалгааны тайлан, диаграм, LaTeX эхийг агуулна.

Хүснэгт 3.1: Төслийн гол хавтас ба үүрэг

Хавтас/файл	Үүрэг	Тайлбар
Frontend	Веб платформын код	Next.js, React, TypeScript, Tailwind CSS, Supabase client, UI компонентууд
src/app	App Router хуудас ба API	Landing, courses, dashboard, admin, chat, checkout, auth routes, /api/chat
src/components	Дахин ашиглагдах UI компонент	CourseCard, LessonPlayer, VideoPlayer, ChatDrawer, Nav, Footer гэх мэт
src/hooks	Client-side state ба data hook	useAuth, usePurchases, useReveal

src/lib	Shared utility ба өгөгдөл	Static course data, Supabase client, database helper, youtube utility
supabase-schema.sql	Өгөгдлийн сангийн schema	Course, lesson, progress, payment, knowledge base, RLS policy
dataset	Том хэмжээний FL Studio зөвлөмж	20000 бичлэгтэй compression/dynamics чиглэлийн JSON dataset
rag-dataset-builder	RAG dataset pipeline	Topic extraction, generation, validation, ready JSONL/CSV, Supabase seed
diplomaper	Дипломын тайлан	LaTeX chapter, references, figures, diagrams, generated PDF

3.3 Фронтенд платформын дизайн

3.3.1 App Router-ийн route бүтэц

Frontend application нь Next.js App Router ашигласан. Энэ нь route бүрийг src/app дотор file-based байдлаар тодорхойлох боломжтой. Сургалтын платформын үндсэн route-уудыг Хүснэгт 3.2-д харуулав.

Нүүр хуудас нь хэрэглэгчид платформын зорилго, суралцах замнал, гол боломжуудыг харуулах үүрэгтэй. Course catalog нь хэрэглэгчийг бодит сургалтын бүтээгдэхүүн сонгох шат руу оруулна. Course detail нь системийн хамгийн төв хэсэг бөгөөд хичээл үзэх, quiz өгөх, төлбөртэй эрх шалгах, AI туслахаас лавлах боломжийг нэг дор нэгтгэнэ.

3.3.2 Компонентын зохион байгуулалт

React компонентуудыг domain болон UI хариуцлагаар салгасан. **CourseCard** нь курсийг жагсаалтад харуулах, **LessonPlayer** нь хичээл тоглуулах, **VideoPlayer** нь YouTube video embed хийх, **ChatDrawer** нь AI туслахын overlay chat интерфэйсийг удирдана. Layout түвшинд **Nav**, **Footer**, **DashboardLayout** зэрэг компонентууд ашиглагдана.

Хүснэгт 3.2: Frontend route-ууд ба хэрэглэгчийн урсгал

Route	Хуудасны үүрэг	Гол компонент/логик
/	Нүүр хуудас, бүтээгдэхүүний танилцуулга	HeroSection, LearningPathSection, FeaturesSection, ChatButton
/courses	Курсийн каталог	Хайлт, ангилал, түвшин, үнэ, эрэмбэлэлт, Supabase course query
/courses/[slug]	Курсийн дэлгэрэнгүй ба lesson player	Access check, free/paid lesson, quiz, mentor chat, YouTube duration
/dashboard	Хэрэглэгчийн самбар	Суралцах явц, худалдан авалт, хэрэглэгчийн мэдээлэл
/admin	Админ удирдлага	Course, lesson, хэрэглэгч, өгөгдөл удирдах интерфэйс
/chat	AI чат хуудас	Chat UI, /api/chat дуудлага
/checkout	Төлбөрийн урсгал	Course purchase simulation, access update

Хүснэгт 3.3: Frontend компонентуудын үүрэг

Компонент	Үүрэг	Системд өгөх ач холбогдол
HeroSection	Нүүр хуудасны эхний хэсэг	Платформын үнэ цэнийг хэрэглэгчид шууд ойлгуулах
LearningPathSection	Суралцах замналыг харуулах	Хэрэглэгчид дараалсан сургалтын ойлголт өгөх
CourseCard	Курсийн товч мэдээлэл харуулах	Каталогийг уншигдахуйц, сонгоход хялбар болгох
LessonPlayer	Хичээл, brief, lock state, completion	Сургалтын үндсэн хэрэглээг хэрэгжүүлэх
VideoPlayer	YouTube хичээл тоглуулах	Гадаад video content-ийг course flow-д нэгтгэх
ChatDrawer	AI mentor chat overlay	Суралцагч тухайн мөчид асуулт асуух боломж олгох
DashboardLayout	Хэрэглэгчийн самбарын layout	Progress болон хувийн мэдээллийг цэгцтэй харуулах

PricingCard	Төлбөрийн сонголт харуулах	Course monetization үрсгалд дэмжлэг үзүүлэх
-------------	----------------------------	---

3.4 Курс ба хичээлийн өгөгдлийн дизайн

3.4.1 Статик курсийн өгөгдөл

Одоогийн курсийн эхний өгөгдөл `Frontend/src/lib/data.ts` файлд тодорхойлогдсон. Энэ файлд багшийн мэдээлэл, course seed, lesson seed, lesson duration, youtubeId, exercise, takeaway зэрэг өгөгдөл орно. Static data нь эхний prototype болон demo-д тохиромжтой бөгөөд дараагийн шатанд Supabase courses, lessons хүснэгтүүдтэй бүрэн синхрончлогдох боломжтой.

Курсийн data model нь дараах үндсэн талбаруудтай.

- id, slug, title, description;
- category, level, price, teacherId;
- duration, lessonsCount, curriculum;
- lesson бүрийн title, youtubeId, durationMinutes, summary, exercise, takeaway, free.

Энэ бүтэц нь сургалтын platform-д хэрэгтэй “контент + дадлага + ахиц” гэсэн гурван талт логикийг дэмжинэ.

3.4.2 Курсийн каталогийн хэрэгжилт

`/courses` хуудас нь static course data болон Supabase-аас ирэх course data-г нэгтгэж харуулна. Хэрэглэгч query, category, level, price type, sort option ашиглан курсийг шүүнэ. Энэ нь платформыг энгийн жагсаалт бус, суралцагч өөрийн түвшин, сонирхол, төлбөрийн боломжид тохируулан шийдвэр гаргах орчин болгодог.

3.5 Хичээлийн дэлгэрэнгүй ба тоглуулагч

3.5.1 Хандалтын хяналт

`/courses/[slug]` хуудас нь хэрэглэгч тухайн курсийг үзэх эрхтэй эсэхийг `usePurchases` hook-оор шалгана. Free lesson бол зочин эсвэл худалдан аваагүй хэрэглэгч үзэж болно. Paid lesson бол хэрэглэгч тухайн course-ийг худалдан авсан үед л нээгдэнэ. Энэ нь төлбөртэй сургалтын платформын үндсэн

Хүснэгт 3.4: Course catalog-ийн гол боломжууд

Боломж	Хэрэгжилтийн логик	Хэрэглэгчийн ач холбогдол
Хайлт	Course title болон description дээр query matching хийх	Тодорхой сэдвийн курс хурдан олох
Ангилал шүүх	Music production, mixing, mastering гэх мэт category filter	Сонирхсон чиглэлээр төвлөрөх
Түвшин шүүх	Beginner, intermediate, advanced	Өөрийн чадварт тохирох курс сонгох
Үнээр шүүх	Free болон paid course ялгах	Туршиж үзэх эсвэл төлбөртэй контент сонгох
Эрэмбэлэх	Lesson count, duration, price	Суралцах хугацаа, үнэ цэнээр харьцуулах

бизнес шаардлага юм.

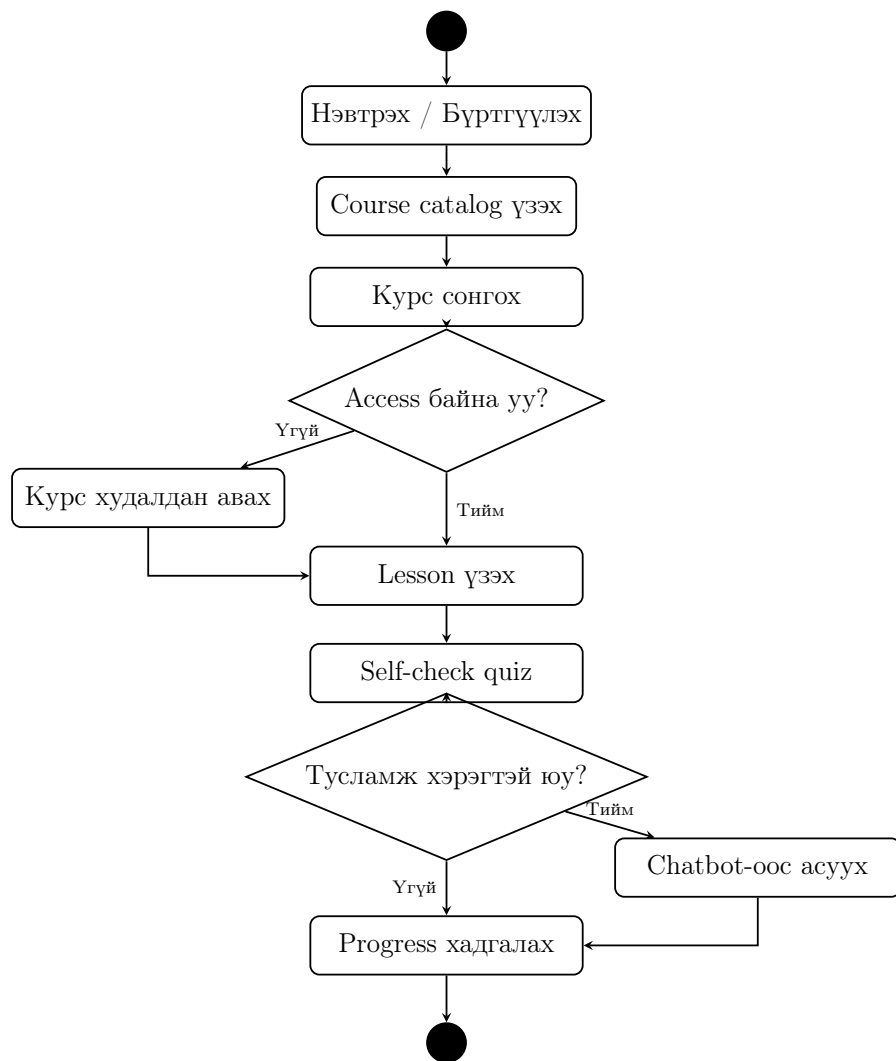
3.5.2 Хичээлийн тоглуулагч

LessonPlayer компонент нь дараах төлөвүүдийг зохицуулна.

- lesson locked бол худалдан авах СТА харуулах;
- video lesson бол YouTube player харуулах;
- video байхгүй lesson бол summary, exercise, takeaway бүхий lesson brief харуулах;
- lesson дуусгасан үед completed state харуулах;
- lesson list дотор current lesson, locked lesson, completed lesson ялгах.

3.5.3 Өөрийгөө шалгах сорил

Course detail хуудас нь lesson бүрийн дараах богино self-check болон final quiz-ийн логик агуулна. Self-check нь сургалтын platform-д хоёр үүрэгтэй. Нэгдүгээрт, хэрэглэгч видео үзээд өнгөрөх бус, гол ойлголтоо шалгана. Хоёрдугаарт, progress tracking нь зөвхөн “үзсэн” төлөвөөс илүү утгатай болно. Quiz нь одоогоор client-side state дээр суурилсан боловч дараагийн шатанд Supabase-д хадгалах боломжтой.



ЗУРАГ 3.1: Суралцагчийн course access, lesson, quiz, chatbot, progress хадгалалтын үйл ажиллагааны урсгал

3.6 Нэвтрэлт, худалдан авалт ба ахицын хэрэгжилт

3.6.1 Нэвтрэлтийн hook

`useAuth.ts` hook нь хэрэглэгчийн session, login, logout, auth state change зэрэг үйлдлийг удирдана. Энэ hook-ийг course detail, dashboard, lesson player зэрэг хэрэглэгчийн төлөв шаарддаг компонентууд ашиглаж болно. Authentication нь зөвхөн UI state бус, Supabase RLS policy-ийн үндэс болдог.

3.6.2 Худалдан авалтын хандалт

`usePurchases.ts` hook нь хэрэглэгчийн худалдан авсан course-уудыг уншиж, `canWatch(courseId, price)` логигоор хичээл үзэх эрхийг тодорхойлно. Үнэгүй курс эсвэл free lesson бол шууд нээгдэнэ. Төлбөртэй lesson-д хандах үед хэрэглэгчийг checkout урсгал руу чиглүүлнэ.

3.6.3 Ахицын хяналт

Progress tracking нь `completed_lessons` болон `course_progress` хүснэгтүүдтэй холбоотой. Хэрэглэгч lesson дуусгасан гэж тэмдэглэхэд lesson progress хадгалагдаж, course-ийн progress percent шинэчлэгдэх боломжтой. Энэ нь сургалтын платформыг статик video library бус, суралцагчийн замналыг хэмждэг систем болгоно.

3.7 Өгөгдлийн сангийн хэрэгжилт

3.7.1 Supabase schema

`supabase-schema.sql` файлд платформын өгөгдлийн үндсэн schema тодорхойлогдсон. Schema нь authentication хэрэглэгчтэй холбогдох profile, course, lesson, purchase, completion, progress, payment, knowledge base, chat message зэрэг хүснэгтүүдээс бүрдэнэ.

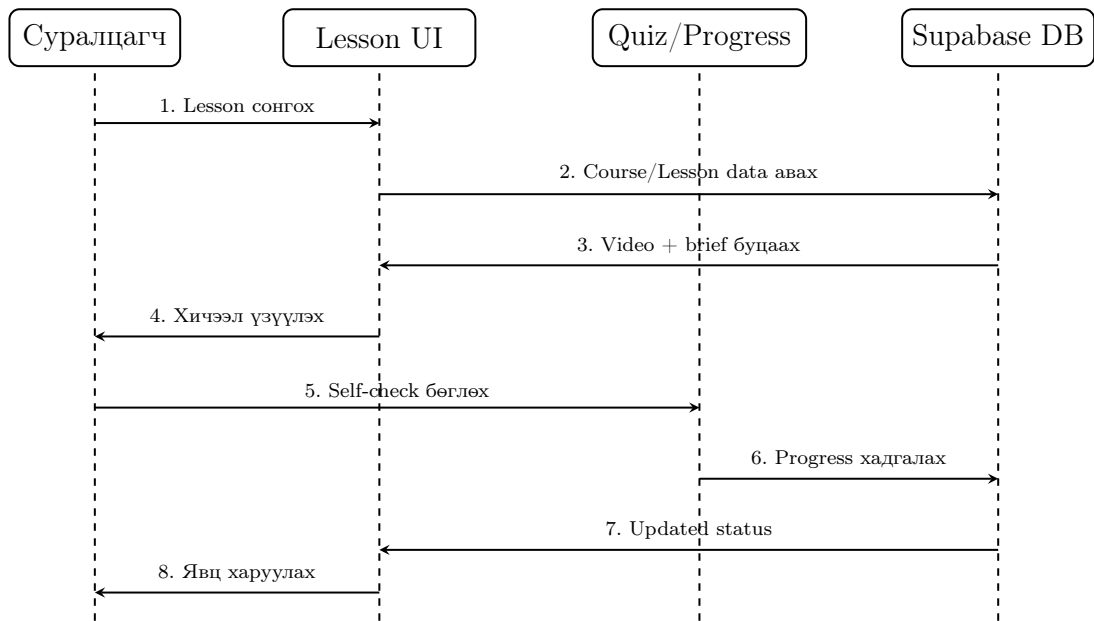
Хүснэгт 3.5: Өгөгдлийн сангийн хүснэгтүүд ба үүрэг

Хүснэгт	Үндсэн өгөгдөл	Системд гүйцэтгэх үүрэг
---------	----------------	-------------------------

profiles	full name, avatar, bio, is_admin	Auth user-ийн нэмэлт мэдээлэл ба admin role
courses	course id, title, description, category, level, price, slug	Курсийн каталог ба админ удирдлагын үндсэн entity
lessons	lesson id, course id, youtube id, summary, exercise, free	Course curriculum ба lesson player-ийн өгөгдөл
purchased_courses	user id, course id, purchased at	Paid access шалгах үндсэн хүснэгт
completed_lessons	user id, lesson id, course id	Lesson completion ба progress тооцоолол
course_progress	user id, course id, progress percent, last lesson	Dashboard болон суралцах явцын мэдээлэл
knowledge_base	category, question, answer, source, embedding	RAG chatbot-ийн мэдлэгийн сан
chat_messages	user message, ai response, sources, confidence	Chat history, quality review, future analytics
payments	user id, course id, amount, currency, status	Төлбөрийн түүх ба purchase flow
audio_files, mixing_analysis	upload metadata, analysis result	Ирээдүйн audio analysis өргөтгөлийн суурь

3.7.2 RLS бодлого

Schema-д бүх гол хүснэгт дээр RLS идэвхжүүлсэн. Энэ нь хэрэглэгч өөрийн progress, purchase, chat message, audio file зэрэг хувийн өгөгдөлд л хандах боломжтой гэсэн зарчимтай. Админ role-той хэрэглэгчид course, lesson, knowledge base, хэрэглэгчийн өгөгдлийг удирдах policy нээгдэнэ. Ингэснээр frontend дээр алдаа гарсан ч database түвшинд хамгаалалт давхар ажиллана.

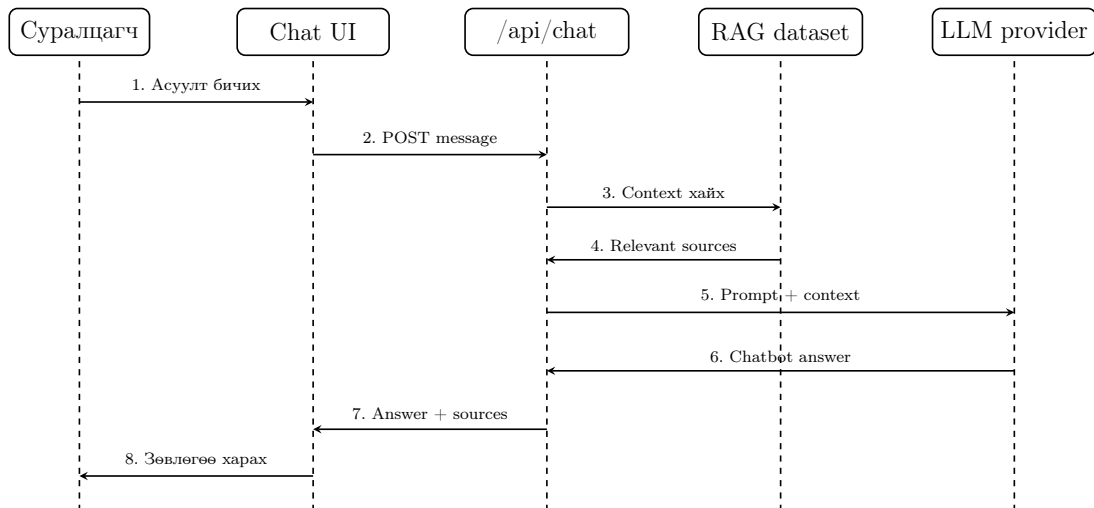


ЗУРАГ 3.2: Lesson үзэх, self-check хийх, progress хадгалах дарааллын диаграм

3.8 AI чатботын хэрэгжилт

3.8.1 API route-ийн үүрэг

AI туслахын backend нь `Frontend/src/app/api/chat/route.ts` файлд байрлана. Энэ route нь хэрэглэгчийн message хүлээн авч, local RAG dataset уншиж, lexical scoring-оор context сонгож, prompt үүсгэж, сонгосон LLM provider руу илгээж, хариулт болон source metadata-г буцаана.



ЗУРАГ 3.3: AI mentor асуулт-хариултын дарааллын диаграм

3.8.2 Мэдээлэл сэргээх хэрэгжилт

RAG route нь `rag_ready_mn.jsonl` dataset-ийг cache хийж уншина. Entry бүрийн title, question, category, content, keywords талбарыг нэг searchable string болгон, token set үүсгэнэ. Хэрэглэгчийн асуултыг tokenize хийж, token overlap, substring match, category match зэрэг энгийн scoring ашиглан хамгийн тохирох context-уудыг сонгоно.

Энэ lexical retrieval нь дараах давуу талтай.

- implementation хялбар, demo хийхэд хурдан;
- жижиг болон дунд хэмжээний dataset-д ойлгомжтой;
- ямар token context сонгоход нөлөөлж байгааг тайлбарлах боломжтой;
- embedding service шаардлагагүй тул local development-д тохиромжтой.

Сул тал нь paraphrase болон semantic similarity-г бүрэн барихгүй. Иймээс дараагийн хувилбарт Supabase pgvector ашиглан embedding search нэмэх боломжтой.

3.8.3 Prompt бүрдүүлэлт

Prompt нь хэрэглэгчийн асуулт, retrieved context, source type, citation-safe тэмдэглэл, AI mentor-ийн persona, хариулах дүрэм зэргийг нэгтгэнэ. Persona нь “Melodex Mentor” буюу FL Studio суралцагчдад дулаан, практик зөвлөгөө өгдөг туслах байдлаар тодорхойлогдсон. Prompt-д “generated tip-ийг official manual мэт бүү танилцуул” гэсэн дүрэм орсон нь dataset-ийн copyright болон source-safety эрсдэлийг бууруулна.

3.8.4 Үйлчилгээний abstraction ба fallback

Route нь Gemini, OpenAI-compatible, Ollama гэсэн provider-уудыг environment variable-аар сонгоно. Ollama provider дээр response чанар тогтворгүй үед repeated text, low unique token ratio, low unique character ratio зэрэг шалгуураар чанар муу хариултыг илрүүлж retry хийдэг. Мөн limiter, EQ muddy, clipping зэрэг нийтлэг асуултад rule-based fallback answer буцаах боломжтой.

Хүснэгт 3.6: AI chatbot-ийн инженерчлэлийн шийдлүүд

Шийдэл	Хэрэгжилт	Ач холбогдол
Dataset cache	JSONL файлыг нэг удаа уншиж IndexedEntry болгон хадгалах	API request бүрд file parse хийх зардлыг бууруулна.

Lexical scoring	Token overlap, substring, category match ашиглах	MVP шатанд хурдан, ойлгомжтой retrieval өгнө.
Prompt assembly	Context, source metadata, persona, rules нэгтгэх	AI хариултыг сургалтын хүрээнд хязгаарлана.
Provider abstraction	Ollama, Gemini, OpenAI-compatible сонголт	Зардал, чанар, privacy-ийн уян хатан байдал олгоно.
Low-quality detection	Давталт, token ratio, character ratio шалгах	Жижиг local model-ийн муу хариултаас хамгаална.
Rule fallback	Нийтлэг mixing/mastering асуудалд шууд хариу	Provider unavailable үед хэрэглэгчид хэрэгтэй тусламж өгнө.
Source metadata	title, category, sourceName, citationSafe буцаах	AI хариултын эх сурвалжийн ил тод байдлыг нэмэгдүүлнэ.

3.9 RAG dataset builder-ийн хэрэгжилт

3.9.1 Pipeline-ийн бүтэц

rag-dataset-builder нь RAG мэдлэгийн санг бэлтгэх тусдаа pipeline юм. Энэ нь source metadata олборлох, Mongolian entry үүсгэх, validation хийх, ready dataset build хийх, Supabase import CSV гаргах алхмуудтай.

1. `extract_topics.py`: source URL-уудаас copyright-safe topic metadata олборлоно.
2. `generate_dataset.py`: topic бүрээс beginner-friendly Mongolian entry үүсгэнэ.
3. `validate_dataset.py`: JSON structure, duplicate, word count, category зэрэг шалгуурыг шалгана.
4. `build_ready_dataset.py`: normalized JSONL/CSV болон Supabase seed CSV үүсгэнэ.
5. `validate_ready_dataset.py`: ready dataset-ийн нийт бүрэн байдлыг шалгана.

3.9.2 Dataset-ийн одоогийн төлөв

Ready dataset report-оос үзэхэд RAG dataset нь 4990 орчим бичлэгтэй бөгөөд голчлон Mixing ба Mastering ангилалд төвлөрсөн. Мөн dataset хавтас дахь FL Studio tips JSON файлд 20000 бичлэгтэй compression/dynamics чиглэлийн зөвлөмж хадгалагдаж байна. Энэ нь системийн AI туслахыг music production domain-д чиглүүлэх суурь өгөгдөл болно.

Хүснэгт 3.7: Dataset бүрэлдэхүүнүүд

Dataset	Хэмжээ	Үүрэг
Ready JSONL	4990 орчим бичлэг	Chatbot retrieval-д ашиглах normalized knowledge entry
Ready CSV	4990 орчим мөр	Хүний review хийхэд тохиромжтой spreadsheet хувилбар
Supabase seed CSV	Import shape	knowledge_base хүснэгтэд import хийх өгөгдөл
FL Studio tips JSON	20000 бичлэг	FL Studio mixing/compression зөвлөмжийн том сан

3.10 Админ удирдлагын хэрэгжилт

Admin page нь course, lesson, хэрэглэгч, өгөгдлийн удирдлагын эхний интерфейс юм. Database түвшинд is_admin талбар болон admin шалгах helper function ашиглан admin policy тогтоосон. Энэ нь зөвхөн frontend дээр admin button нуух биш, database level authorization хийх боломжтой болгож байна.

Админ удирдлагын үндсэн үүрэг:

- course болон lesson мэдээлэл нэмэх, засах;
- хэрэглэгчийн role болон profile мэдээлэл хянах;
- knowledge base entry удирдах;
- payment, purchase, progress мэдээллийг review хийх;
- chatbot-ийн source болон dataset чанарыг ирээдүйд засварлах.

3.11 Хэрэгжилтийн traceability шинжилгээ

Гуравдугаар бүлгийн хэрэгжилтийг зөвхөн файл тус бүрийн тайлбар байдлаар бус, хэрэглэгчийн шаардлагаас эхлэн UI, hook, API, database хүртэл

хэрхэн дамжиж байгаагаар шинжлэх нь илүү оновчтой. Энэ нь системийн шаардлага бодит кодтой холбогдсон эсэхийг шалгах боломж олгоно. Traceability нь дипломын төсөлд хоёр ач холбогдолтой. Нэгдүгээрт, тайлангийн судалгааны хэсэг бодит implementation-тэй тасрахгүй. Хоёрдугаарт, ирээдүйд аль шаардлага ямар module-д нөлөөлөхийг ойлгоход хялбар болно.

Хүснэгт 3.8: Шаардлагаас хэрэгжилт хүртэлх traceability

Шаардлага	Frontend хэрэгжилт	Backend/database хэрэгжилт	Үр дүн
Course discovery	Course catalog, filter, CourseCard	courses table, static data fallback	Суралцагч курс хайж сонгоно.
Lesson access	Course detail, LessonPlayer, lock UI	purchased courses, completed lessons	Free болон paid lesson ялгаатай нээгдэнэ.
Progress tracking	Mark complete, dashboard progress	completed lessons, course progress	Суралцагчийн явц хадгалагдана.
AI mentor	ChatDrawer, chat page, source display	chat API, RAG dataset, provider abstraction	Хичээлтэй холбоотой асуултад context-тэй хариулна.
Admin control	Admin page, CRUD form direction	admin RLS, helper function, management tables	Контент ба хэрэглэгчийн өгөгдлийг удирдах суурь бүрдэнэ.
Future audio	AudioUploader, service layer direction	audio files, mixing analysis, melody variations	Audio analysis модульд өргөжих боломжтой.

3.12 Route бүрийн хэрэгжилтийн дэлгэрэнгүй

Frontend route бүр хэрэглэгчийн тодорхой workflow-тэй холбогдоно. Нүүр хуудас нь бүтээгдэхүүний үнэ цэнийг ойлгуулах, course page нь суралцах сонголт хийх, course detail page нь хичээл үзэх, dashboard нь ахиц харах, checkout

page нь paid access авах, chat page нь AI туслахтай харилцах, admin page нь платформын өгөгдлийг удирдах үүрэгтэй.

Хүснэгт 3.9: Route бүрийн implementation responsibility

Route	UI responsibility	Data responsibility	Access
Home	Hero, learning path, feature summary, CTA	Static section data	Public
Courses	Catalog, search, category, sort, cards	Course list, price, level, duration	Public
Course detail	Lesson player, lock state, quiz, chat button	Course slug, lesson list, purchase state	Mixed
Dashboard	User profile, purchased courses, progress summary	User session, purchase, progress	Private
Checkout	Payment simulation, selected course summary	Course id, user id, purchase insert	Private
Chat	Full chat UI, message history direction	Chat API request, sources, model response	Public/private hybrid
Admin	Course, lesson, user and data management direction	Admin policy, management tables	Admin only

Route design-ийн үед page бүр бие даан бүх logic-ийг агуулах ёсгүй. Давтагдах хэсгийг component болон hook руу салгаснаар засвар хийхэд хялбар болно. Жишээлбэл course card-ийн visual logic catalog болон landing section-д дахин ашиглагдаж болно. Auth state dashboard, course detail, checkout, admin login зэрэг олон route-д хэрэгтэй тул hook хэлбэрээр төвлөрүүлэх нь зөв.

3.13 Component ба hook түвшний хэрэгжилт

Component architecture нь хэрэглэгчийн interface-ийг жижиг, давтагдах боломжтой нэгж болгон салгах зарчимтай. Энэ төслийн хувьд CourseCard нь course мэдээллийг товч харуулна, LessonPlayer нь lesson content, video, lock state, completion logic-ийг нэгтгэнэ, ChatDrawer нь хэрэглэгч хичээл үзэж байхдаа тусламж авах боломжийг өгнө. DashboardLayout нь private learning area-ийн нийтлэг structure-г тогтвортой хадгална.

Hook түвшинд `useAuth`, `usePurchases`, `useReveal` зэрэг logic байрлана. `useAuth` нь хэрэглэгчийн session болон auth state change-г хариуцна. `usePurchases` нь course access шалгах logic-ийг төвлөрүүлнэ. `useReveal` нь scroll animation, section reveal зэрэг presentation logic-ийг component-оос тусгаарлана. Энэ хуваарилалт нь React application-д separation of concerns зарчим хэрэгжиж байгааг харуулна.

Хүснэгт 3.10: Component ба hook-ийн хэрэгжилтийн үүрэг

Нэгж	Гол үүрэг	Дахин ашиглалтын боломж
CourseCard	Course title, price, level, duration, СТА харуулах	Catalog, landing, dashboard
LessonPlayer	Video, brief, lock state, completion action	Course detail page
ChatDrawer	Floating chat interaction, message request	Lesson page, landing page
useAuth	Session, login state, logout, user metadata	Dashboard, checkout, admin
usePurchases	Course access, paid/free logic	Lesson player, checkout, dashboard
useReveal	Scroll-based visual state	Landing sections

3.14 AI chat API-ийн алгоритмын тайлбар

AI chat route нь хэрэглэгчийн асуултыг шууд LLM provider руу дамжуулахгүй. Эхлээд dataset уншиж cache хийх, query-г normalization хийх, lexical scoring гүйцэтгэх, relevant context сонгох, prompt бүрдүүлэх, provider сонгох, fallback шалгах, response metadata буцаах гэсэн алхмуудтай. Энэ нь RAG-ийн үндсэн pipeline-ийг practical implementation болгож байна.

1. Хэрэглэгч UI-гаас message илгээнэ.
2. API route request body-г шалгаж, хоосон эсвэл буруу форматтай message-г буцаана.
3. Ready dataset memory cache-д байгаа эсэхийг шалгана. Байхгүй бол JSONL файл уншиж parse хийнэ.
4. Query token болон dataset entry-ийн title, question, content, category талбаруудыг харьцуулж scoring хийнэ.
5. Өндөр оноотой хэдэн context сонгож, prompt-д оруулна.
6. Environment configuration-д тулгуурлан Ollama, Gemini эсвэл OpenAI-compatible provider сонгоно.

7. Provider хариу амжилтгүй эсвэл чанар муу бол fallback answer үүсгэнэ.
8. Хариулт, sources, provider metadata-г frontend руу буцаана.

Хүснэгт 3.11: AI chat API-ийн хэрэгжилтийн алхам

Алхам	Implementation detail	Чанарын зорилго
Input validation	Message хоосон эсэх, body format шалгах	API misuse бууруулах
Dataset cache	JSONL-г memory-д хадгалж дахин уншихгүй байх	Latency бууруулах
Lexical scoring	Token overlap, category match, substring	MVP retrieval ойлгомжтой байх
Prompt assembly	Context, source, style rule нэгтгэх	Хариултыг сургалтын хүрээнд барих
Provider call	Environment-д тулгуурласан provider сонголт	Deployment flexibility
Fallback	Low quality эсвэл provider failure үед rule answer	Demo stability
Source metadata	Context entry-ийн title, source type буцаах	Transparency нэмэх

3.15 Dataset pipeline-ийн хэрэгжилтийн дэлгэрэнгүй

RAG dataset builder нь дипломын төслийн тусдаа судалгааны artifact гэж үзэж болно. Учир нь AI туслахын чанар зөвхөн chat route-ийн кодоос бус, ямар мэдлэгийн сангаар тэжээгдэж байгаагаас хамаарна. Pipeline нь source topic extraction, generated Mongolian explanation, validation, ready export, Supabase seed гэсэн шаттай.

Энд онцлох зүйл нь copyright-safe хандлага юм. Dataset нь эх сурвалжийн урт текстийг шууд хуулбарлах бус, topic metadata болон богино чиглүүлэгч мэдээлэлд тулгуурлан өөрийн үгээр боловсруулсан сургалтын тайлбар бэлтгэнэ. Гэхдээ generated/internal tip-ийг official citation мэт харуулах ёсгүй. Иймээс source type, citation-safe, source-grounded зэрэг metadata-г ялгах шаардлагатай.

Хүснэгт 3.12: Dataset pipeline-ийн artifact-ууд

Artifact	Үүрэг	Чанарын шалгуур
Sources list	Судлах эх сурвалжийн URL болон topic direction	Давхардал бага, domain холбоотой байх
Raw topics	Heading, snippet, page title metadata	Хэт урт copied text агуулахгүй байх
Generated dataset	Монгол тайлбар, keyword, category, level	Уншигдахуйц, beginner-friendly байх
Ready JSONL	Chat API-д унших normalized entry	Stable schema, id, content талбартай байх
Ready CSV	Хүний review хийх spreadsheet хэлбэр	Review хийхэд ойлгомжтой багануудтай байх
Supabase seed	knowledge base-д import хийх shape	Database schema-тай нийцэх

3.16 Өгөгдлийн сангийн implementation pattern

Supabase schema нь MVP болон future audio module-ийг хамтад нь дэмжихээр зохион байгуулагдсан. Одоогийн course, lesson, purchase, progress, knowledge base хүснэгтүүд нь сургалтын платформын үндсэн өгөгдлийг хадгална. Audio files, mixing analysis, melody variations зэрэг хүснэгтүүд нь дараагийн шатны AI audio processing-д бэлтгэсэн өргөтгөлийн суурь юм.

Database implementation-ийн үед foreign key, unique constraint, RLS policy, admin helper function зэрэг элементүүдийг хамтад нь хэрэглэх шаардлагатай. Foreign key нь entity хоорондын integrity-г хангана. Unique constraint нь давхар purchase эсвэл completion бичлэг үүсэхээс хамгаална. RLS нь хэрэглэгч өөрийн өгөгдөлдөө л хандах боломжтой болгоно. Admin helper function нь policy давхардлыг багасгана.

Хүснэгт 3.13: Database implementation pattern

Pattern	Төслийн жишээ	Ач холбогдол
Foreign key	user id, course id, lesson id relation	Өгөгдлийн холбоо алдагдахаас хамгаална.
Unique constraint	User-course purchase, user-lesson completion	Давхар бичлэг үүсэхээс сэргийлнэ.

Public read table	Courses, lessons, knowledge base select	Зочин хэрэглэгч preview харах боломжтой.
Owner protected table	Progress, chat messages, payments	Хувийн өгөгдөл хамгаалагдана.
Admin policy	Course, lesson, knowledge base manage	Контент удирдлагыг зөвшөөрөлтэй болгоно.

3.17 Туршилт ба шалгалтын хэрэгжилт

Дипломын MVP түвшний системд automated test бүрэн бичигдээгүй байсан ч implementation чанарыг шалгах тодорхой арга хэрэгтэй. Frontend route бүр гараар нэвтэрч үзэх, auth state өөрчлөх, paid lesson lock шалгах, chat API-д sample question илгээх, dataset validation script ажиллуулах, LaTeX build хийх зэрэг нь practical verification болно.

ХҮСНЭГТ 3.14: Implementation verification checklist

Шалгах хэсэг	Шалгах арга	Хүлээгдэх үр дүн
Course catalog	Search, filter, course card click	Course detail нээгдэнэ.
Lesson access	Free болон paid lesson түрших	Lock state зөв харагдана.
Auth flow	Login, register, logout scenario	Session state шинэчлэгдэнэ.
Progress	Lesson complete action түрших	Progress state хадгалагдана.
Chatbot	FL Studio, mixing, theory асуулт илгээх	Context-тэй хариулт ирнэ.
Dataset	Validation script ажиллуулах	Schema болон duplicate шалгагдана.
Thesis PDF	LaTeX build хийх	PDF, citation, figure/table reference гарна.

3.18 Хэрэгжилтийн эрсдэл ба сайжруулалтын чиглэл

Одоогийн хэрэгжилт нь дипломын MVP түвшинд сургалтын платформын үндсэн логикийг харуулж байгаа боловч дараах сайжруулалт шаардлагатай.

Хүснэгт 3.15: Хэрэгжилтийн эрсдэл ба сайжруулалтын төлөвлөгөө

Эрсдэл	Одоогийн төлөв	Сайжруулах чиглэл
Static ба database course data давхардах	Static <code>data.ts</code> болон Supabase course query зэрэгцэж байна	Course data-г бүрэн Supabase migration хийх
Lexical retrieval-ийн хязгаар	Token overlap-д тулгуурласан тул semantic match сул	Embedding, pgvector, HNSW search нэмэх
Citation-safe бичлэг цөөн	Ready dataset-д source-grounded бичлэг бага	Official/manual source-той curated entry нэмэх
Lesson progress helper зөрүү	Зарим helper <code>lesson_progress</code> гэж нэрлэсэн, schema-д <code>completed_lessons</code> байгаа	Database helper-ийг schema-тай бүрэн нийцүүлэх
Payment integration mock түвшинд	Payment flow simulation байна	Бодит payment provider, webhook, receipt validation нэмэх
Admin UX эхний шатанд	Admin dashboard basic удирдлагатай	CRUD form, validation, audit log нэмэх

3.19 Бүлгийн дүгнэлт

Гуравдугаар бүлгээр платформын бодит дизайн ба хэрэгжилтийг тайлбарлав. Frontend талд Next.js App Router, React component, course catalog, lesson player, self-check quiz, chat drawer зэрэг хэрэглэгчийн үндсэн урсгалууд хэрэгжсэн. Backend ба өгөгдлийн талд Supabase schema, authentication, RLS policy, purchase/progress table, knowledge base, chat message зэрэг бүтэц тодорхойлогдсон. AI туслах нь `/api/chat` route, local JSONL retrieval, prompt assembly, provider abstraction, fallback logic дээр тулгуурласан.

Энэхүү хэрэгжилт нь сургалтын платформын MVP-г бүрдүүлж байгаа боловч production түвшинд хүргэхийн тулд course data migration, embedding retrieval, citation-safe dataset нэмэгдүүлэлт, бодит payment integration, admin CRUD сайжруулалт хийх шаардлагатай. Дараагийн бүлэгт энэхүү хэрэгжилтийн архитектур, системийн бүтэц, давхарга хоорондын уялдааг илүү өргөн хүрээнд авч үзнэ.

БҮЛЭГ 4

Платформын архитектур ба системийн бүтэц

4.1 Бүлгийн зорилго

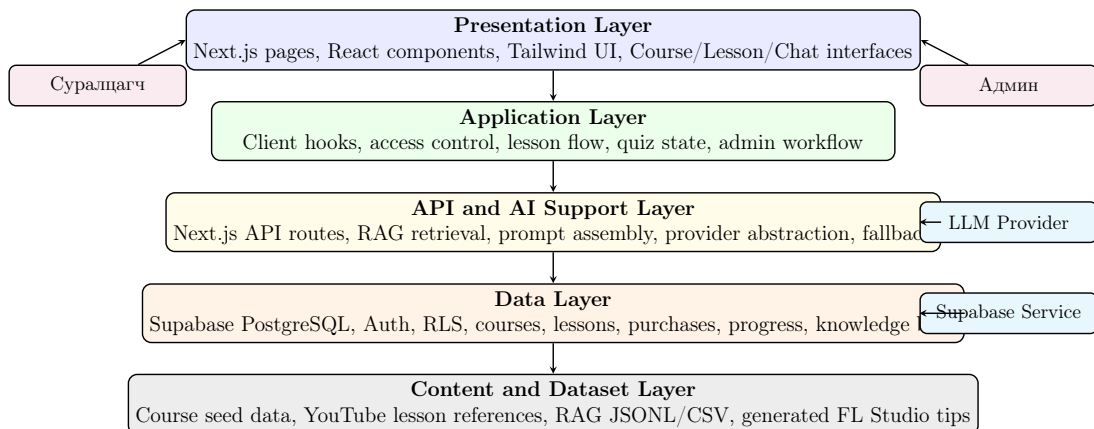
Энэ бүлэгт өмнөх бүлэгт тайлбарласан хэрэгжилтийг системийн архитектурын түвшинд нэгтгэн авч үзнэ. Гуравдугаар бүлэг нь *ямар модулийг хэрхэн хэрэгжүүлсэн бэ?* гэдэгт төвлөрсөн бол дөрөвдүгээр бүлэг нь *эдгээр модуль хоорондоо хэрхэн холбогдож, ямар давхарга, ямар өгөгдлийн урсгал, ямар бүтцээр систем болж ажиглаж байна вэ?* гэдэгт хариулна.

Архитектурын судалгаанд дараах асуудлыг хамруулав.

- системийн логик давхарга ба хариуцлагын зааг;
- хэрэглэгч, админ, AI provider, Supabase зэрэг гаднын оролцогчийн холбоо;
- use case, activity, sequence, class, ER диаграммын тайлбар;
- frontend, application, API, data, AI support, dataset давхаргын бүтэц;
- security, scalability, maintainability, deployment-ийн архитектурын шийдэл;
- ирээдүйн өргөтгөлийн архитектурын чиглэл.

4.2 Системийн ерөнхий архитектур

FL Studio сургалтын платформ нь сургалтын контент, хэрэглэгчийн эрх, суралцах явц, AI туслах, dataset pipeline гэсэн таван гол бүрэлдэхүүнээс бүрдэнэ. Эдгээрийг нэг кодын бааз дотор бүрэн хольж бичвэл систем засварлахад хүндрэлтэй болно. Иймээс архитектурын үндсэн зарчим нь **давхарга тусгаарлах, өгөгдлийн ownership тодорхойлох, AI-г туслах давхарга байдлаар хязгаарлах, өргөтгөх боломж хадгалах** юм.



ЗУРАГ 4.1: Платформын давхаргат архитектур

Зураг 4.1-д системийн үндсэн давхаргыг харуулав. Presentation layer нь хэрэглэгчийн интерфэйсийг хариуцна. Application layer нь user flow, access control, lesson state, quiz state, admin workflow зэрэг бизнес логикийг frontend талд зохицуулна. API and AI support layer нь chatbot, YouTube duration, provider abstraction зэрэг server-side logic-ийг агуулна. Data layer нь Supabase PostgreSQL, authentication, RLS, course/progress/payment/knowledge base өгөгдлийг удирдана. Content and Dataset layer нь course seed data, YouTube lesson reference, RAG JSONL/CSV, generated FL Studio tips зэрэг сургалтын болон AI өгөгдлийн эх үүсвэрийг хадгална.

4.3 Оролцогч ба use case бүтэц

Системийн use case нь зочин, суралцагч, админ гэсэн гурван үндсэн actor-той. Зочин хэрэглэгч курс хайх, free preview үзэх, бүртгүүлэх боломжтой. Суралцагч бүртгэлээр нэвтэрч, курс худалдан авч, lesson үзэж, self-check quiz өгч, AI mentor-оос асуулт асууж, ахицаа хадгална. Админ course, lesson, knowledge base, хэрэглэгчийн мэдээллийг удирдана.



ЗУРАГ 4.2: FL Studio сургалтын платформын Use Case диаграм

ХҮСНЭГТ 4.1: Use case тайлбар

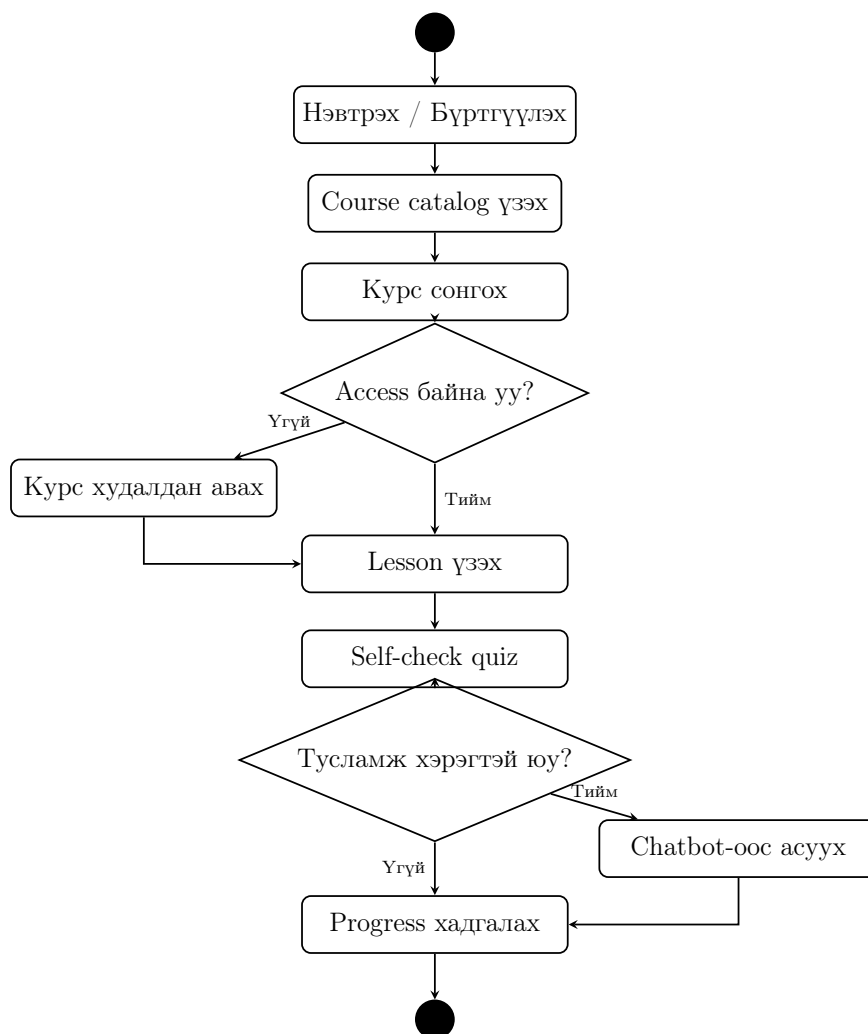
Actor	Use case	Тайлбар
Зочин	Курс хайх/шүүх	Системд бүртгүүлээгүй хэрэглэгч course catalog-оос өөрт тохирох сургалтын чиглэл хайна.

Зочин	Free preview үзэх	Бүртгэл хийхээс өмнө course-ийн эхний lesson, танилцуулга, сургалтын чиглэлийг шалгаж үзнэ.
Зочин	Бүртгүүлэх/нэвтрэх	Free preview-ээс цааш ахиц хадгалах, төлбөртэй lesson авахын тулд account үүсгэнэ.
Суралцагч	Хичээл үзэх	Free lesson эсвэл purchased course-ийн lesson үзнэ.
Суралцагч	Self-check quiz өгөх	Lesson ойлголтоо шалгаж, course progress-д утга нэмнэ.
Суралцагч	Явц хадгалах	Үзсэн lesson, бөглөсөн quiz, course completion зэрэг суралцах төлөвийг бүртгэнэ.
Суралцагч	Chatbot-оос асуух	FL Studio, mixing, mastering, music theory-ийн асуултад context-aware зөвлөгөө авна.
Суралцагч	Эх сурвалжтай хариу авах	AI mentor нь RAG мэдлэгийн сангаас авсан context, source metadata-д тулгуурлан хариулт үүсгэнэ.
Суралцагч	Курс худалдан авах	Paid content access нээх business flow.
Админ	Course/Lesson удирдах	Контентын бүтэц, үнэ, ангилал, lesson metadata-г засварлана.
Админ	Knowledge base удирдах	RAG chatbot-ийн эх сурвалж, асуулт-хариулт, category-г хянаж сайжруулна.

4.4 Үйл ажиллагааны урсгал

Суралцагчийн үндсэн workflow нь course discovery-оос эхэлж, access check, lesson, quiz, chatbot, progress хадгалалт гэсэн дараалалтай. Энэ урсгалд chatbot нь үндсэн сургалтыг тасалдуулахгүйгээр туслах функц хэлбэрээр байрлана.

Activity diagram-аас харахад төлбөртэй access нь lesson flow-ийн өмнөх шийдвэрийн цэг юм. Хэрэв access байхгүй бол хэрэглэгч purchase flow руу

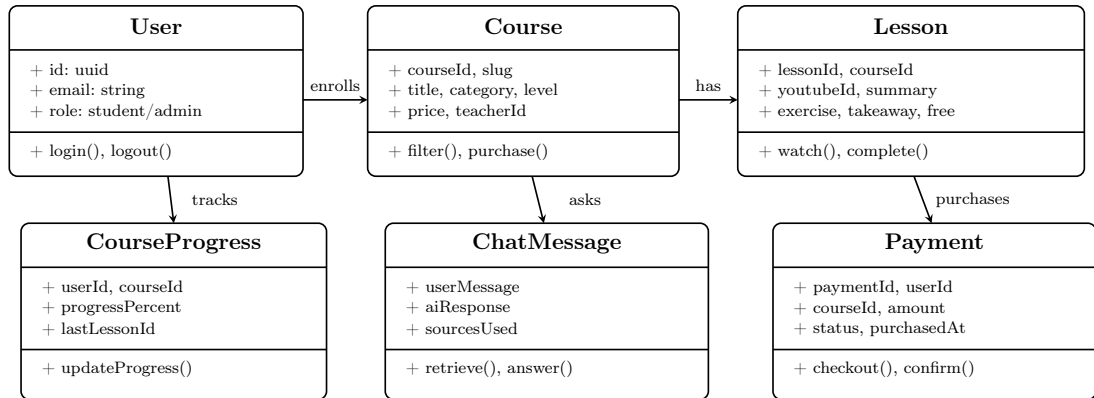


ЗУРАГ 4.3: Суралцагчийн үйл ажиллагааны диаграм

шилжинэ. Access нээгдсэний дараа lesson үзэх, self-check хийх, шаардлагатай үед AI туслахаас асуух, эцэст нь progress хадгалах шат хийгдэнэ.

4.5 Класс ба домайн бүтэц

Системийн домайн объектуудыг User, Course, Lesson, CourseProgress, ChatMessage, Payment зэрэг үндсэн class-аар дүрсэлж болно. Энэ class diagram нь frontend type model, Supabase schema, UI flow гурвыг нэгтгэн ойлгоход тусална.



ЗУРАГ 4.4: Платформын домайн объектуудын класс диаграм

Class diagram-д Course нь олон Lesson агуулна. User нь Course-д enroll хийх, purchase хийх, progress хадгалах, chat message үүсгэх боломжтой. ChatMessage нь RAG retrieval болон AI response-тэй холбоотой. Payment нь course access-ийн business object болно. Энэ бүтэц нь системийн object model-ийг өгөгдлийн сангийн relational model-тэй холбох суурь болдог.

4.6 Өгөгдлийн сангийн архитектур

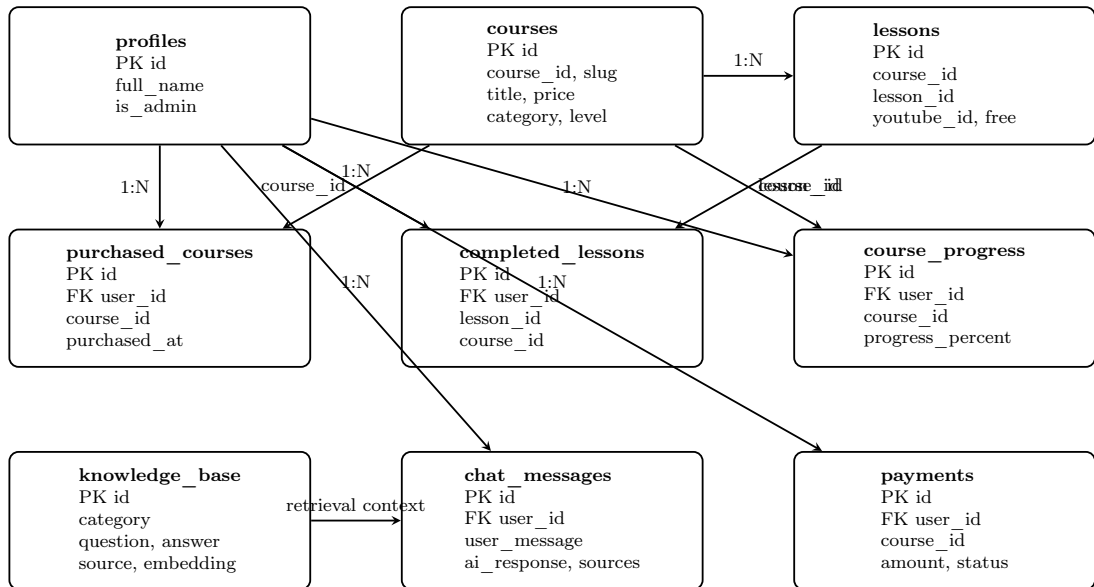
4.6.1 ER загвар

Өгөгдлийн сангийн ER загвар нь хэрэглэгчийн бүртгэл, сургалтын контент, худалдан авалт ба ахиц, мөн мэдлэгийн сан ба чат гэсэн дөрвөн бүлэг entity-г нэгтгэнэ.

4.6.2 Entity бүлгүүд

Өгөгдлийн сангийн entity-г дараах бүлэгт хувааж үзнэ.

1. **Identity бүлэг:** profiles хүснэгт нь Supabase Auth хэрэглэгчтэй холбогдож, full name, avatar, admin role зэрэг мэдээлэл хадгална.



ЗУРАГ 4.5: Системийн үндсэн өгөгдлийн сангийн ER диаграм

2. **Learning content** бүлэг: `courses` болон `lessons` нь сургалтын контентын үндсэн бүтэц юм.
3. **Access and progress** бүлэг: `purchased_courses`, `completed_lessons`, `course_progress`, `payments` нь суралцагчийн төлөв, төлбөр, ахицыг хадгална.
4. **AI support** бүлэг: `knowledge_base` болон `chat_messages` нь chatbot-ийн context болон хариултын түүхийг хадгална.
5. **Future audio** бүлэг: `audio_files`, `mixing_analysis`, `melody_variations` нь ирээдүйн audio analysis болон генератив хөгжмийн модульд бэлтгэсэн өргөтгөлийн суурь юм.

4.6.3 Constraint ба integrity

Өгөгдлийн integrity-г дараах аргаар хангана.

- user id-г Supabase `auth.users(id)`-тай foreign key-ээр холбох;
- `purchased_courses` дээр `unique(user_id, course_id)` ашиглан давхар худалдан авалт үүсэхээс сэргийлэх;
- `completed_lessons` дээр `unique(user_id, lesson_id)` ашиглан давхар completion бичлэгээс хамгаалах;
- course ба lesson identifier-ийг slug болон lesson id-тай уялдуулах;
- admin role-ийг `profiles.is_admin` болон helper function-аар шалгах.

4.7 Frontend архитектур

Frontend архитектур нь route, component, hook, shared lib гэсэн дөрвөн түвшинтэй.

Хүснэгт 4.2: Frontend архитектурын түвшин

Түвшин	Жишээ файл	Үүрэг
Route layer	Courses page, admin page	Хуудасны entry point, data loading, page-level state
Component layer	CourseCard, LessonPlayer, ChatDrawer	UI болон хэрэглэгчийн interaction-ийг жижиг нэгж болгон салгах
Hook layer	useAuth, usePurchases, useReveal	Session, purchase, animation зэрэг давтагдах logic-ийг дахин ашиглах
Shared lib layer	data.ts, supabase.ts, database.ts, youtube.ts	Course seed, Supabase client, database helper, external utility

Энэ бүтэц нь separation of concerns зарчимтай нийцнэ. Route нь бүх UI-г өөртөө агуулахгүй, component-ууд reusable байх ёстой. Hook нь session болон purchase state-г нэг газарт төвлөрүүлнэ. Shared lib нь өгөгдлийн эх үүсвэр, API helper, utility-г UI-гаас тусгаарлана.

4.8 Backend ба API архитектур

Backend логик нь Next.js API route хэлбэрээр хэрэгжсэн. Одоогийн гол route нь /api/chat болон YouTube duration route юм. Chat route нь RAG retrieval, prompt assembly, LLM provider call, fallback, response formatting зэрэг server-side logic агуулна. Энэ нь AI API key, local dataset path, provider configuration зэрэг sensitive logic-ийг client талд ил гаргахгүй байх давуу талтай.

Цаашид payment webhook, admin CRUD API, dataset import API, chat analytics API зэрэг route нэмэгдэх боломжтой. Үүний тулд API route бүр нэг domain responsibility-той байх нь засварлахад хялбар.

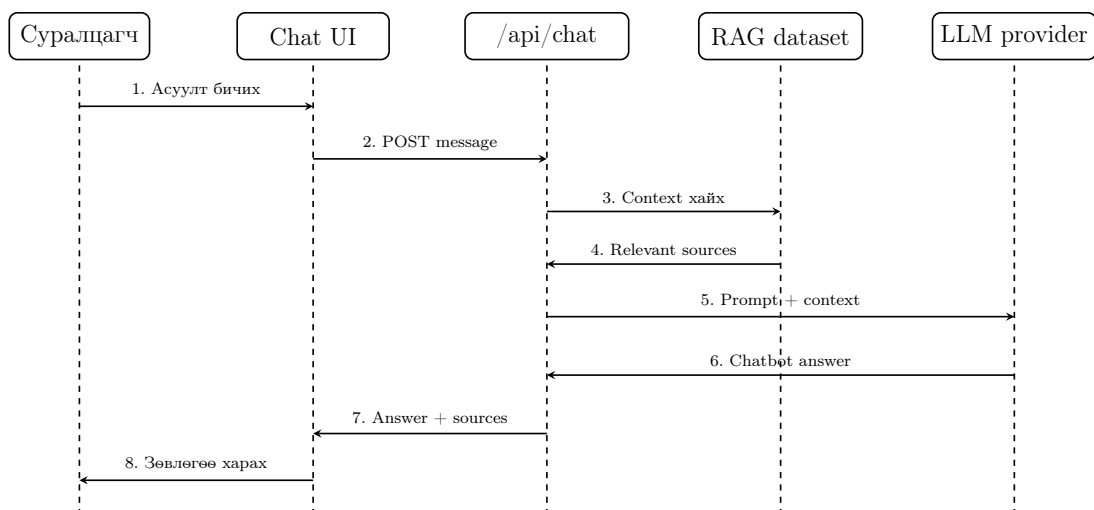
Хүснэгт 4.3: API архитектурын үүрэг

API	Гол үүрэг	Архитектурын ач холбогдол
/api/chat	Хэрэглэгчийн асуултыг авч RAG context-тэй AI хариу үүсгэх	AI logic-ийг frontend-ээс тусгаарлаж, provider abstraction хийх
YouTube duration API	YouTube video duration авах	Lesson metadata-г UI дээр бодитоор харуулах

4.9 AI туслахын архитектур

AI туслах нь платформын үндсэн сургалтын контентыг орлохгүй, харин суралцагчийн асуултад хариулах туслах давхарга юм. Архитектурын хувьд AI туслах нь дараах дэд бүрэлдэхүүнтэй.

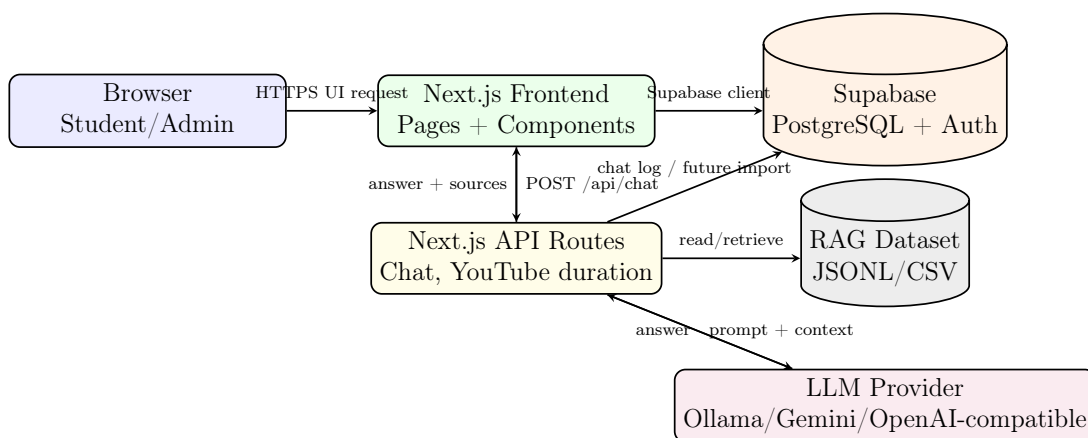
- query preprocessing буюу tokenization;
- local retrieval буюу RAG dataset-ээс context сонгох;
- prompt assembly буюу context, persona, safety rule-ийг нэгтгэх;
- provider abstraction буюу Ollama/Gemini/OpenAI-compatible model сонгох;
- response validation буюу давталт, хоосон хариу, чанаргүй хариуг шалгах;
- fallback буюу rule-based answer эсвэл retry.



ЗУРАГ 4.6: AI туслахын асуулт-хариултын дарааллын архитектур

4.10 Нэвтрүүлэлтийн архитектур

Deployment талаас системийг browser, Next.js application, Supabase service, RAG dataset, LLM provider гэсэн deployment node-уудаар авч үзнэ.



ЗУРАГ 4.7: Платформын deployment architecture

Browser нь хэрэглэгчийн UI request илгээнэ. Next.js frontend нь page render, client interaction, Supabase query, API call-ийг зохицуулна. Supabase нь database, auth, RLS-ийн үүргийг гүйцэтгэнэ. RAG dataset нь development шатанд local JSONL файл хэлбэртэй боловч production шатанд Supabase knowledge base эсвэл vector index рүү шилжих боломжтой. LLM provider нь local эсвэл cloud байж болно.

4.11 Аюулгүй байдлын архитектур

Сургалтын платформд security нь гурван түвшинд хэрэгжинэ.

1. **Client-level guard:** UI дээр paid lesson-г lock хийх, admin link-ийг role-оор хязгаарлах, form validation хийх.
2. **API-level guard:** API route дээр message хоосон эсэх, provider тохиргоо зөв эсэх, low-quality response илрүүлэх, environment variable ашиглах.
3. **Database-level guard:** Supabase RLS policy, foreign key, unique constraint, admin helper function ашиглах.

Client-level guard дангаараа хангалтгүй. Хэрэглэгч browser талын кодыг өөрчилж чаддаг тул paid access болон user data protection-ийг database түвшинд давхар хамгаалах ёстой. RLS нь энэ шаардлагыг хангах гол механизм юм.

Хүснэгт 4.4: Security control matrix

Эрсдэл	Хамгаалалт	Хэрэгжүүлэх түвшин
Өөр хэрэглэгчийн progress харах	RLS: <code>auth.uid() = user_id</code>	Database
Paid lesson-г эрхгүй үзэх	Purchase check + database purchase record	Client + Data
Admin өгөгдөлд эрхгүй хандах	<code>current_user_is_admin()</code> function	Database
API key ил гарах	Server-side API route, environment variable	API/Server
AI хариулт буруу эх сурвалжтай харагдах	source metadata, citation-safe flag	AI support

4.12 Өргөтгөх боломж ба ачаалал даах төлөв

Системийн scalability-г хэрэглэгчийн тоо, course content, RAG dataset, AI provider load гэсэн дөрвөн талаас авч үзнэ.

4.12.1 Курсийн өгөгдлийн өргөтгөл

Одоогийн `static data.ts` нь prototype-д тохиромжтой боловч олон багш, олон course, олон lesson нэмэгдэхэд database-driven content management шаардлагатай. Supabase `courses` болон `lessons` хүснэгтүүд энэ шилжилтийн суурь болж өгнө. Ирээдүйд admin dashboard-оор course CRUD хийх, lesson ordering, thumbnail upload, teacher profile management нэмэх боломжтой.

4.12.2 RAG мэдлэгийн сангийн өргөтгөл

Одоогийн RAG retrieval нь local JSONL файл болон lexical scoring ашиглаж байна. Dataset 5000 орчим бичлэгтэй үед энэ нь demo-д боломжийн боловч dataset 50000, 100000 бичлэг болох үед vector index, database query, caching шаардлагатай. Supabase pgvector, HNSW index, embedding pipeline ашиглах нь дараагийн архитектурын алхам юм.

4.12.3 AI үйлчилгээний өргөтгөл

Provider abstraction нь нэг provider-ийн latency, quota, cost-д хэт хамаарах эрсдэлийг бууруулна. Production орчинд provider routing, rate limit, request

queue, response cache, fallback priority зэрэг нэмэлт механизмууд хэрэгтэй.

Хүснэгт 4.5: Өргөтгөлийн чиглэл

Өргөтгөх чиглэл	Одоогийн архитектур	Дараагийн алхам
Course management	Static data + Supabase table coexistence	Бүх course/lesson-г Supabase-driven болгох
Search	Client-side filter	Server-side search, full-text index
RAG retrieval	Local JSONL lexical scoring	Embedding, pgvector, HNSW, hybrid retrieval
AI provider	Environment-based provider selection	Provider routing, cost-aware fallback
Payment	Mock/initial flow	Payment gateway, webhook, invoice, refund state
Analytics	Basic progress state	Learning analytics, quiz performance, chat usefulness
Audio analysis	Schema-level бэлтгэл	Upload pipeline, source separation, mix feedback

4.13 Засварлахад хялбар байдал ба кодын зохион байгуулалт

Maintainability буюу засварлахад хялбар байдал нь дипломын төсөлд чухал. Учир нь систем нь frontend, backend, database, AI, dataset гэсэн олон төрлийн бүрэлдэхүүнтэй. Кодын зохион байгуулалт дараах зарчмыг баримтална.

- route бүр өөрийн page-level logic-ийг хариуцах;
- shared UI-г component болгож салгах;
- session, purchase, reveal зэрэг давтагдах state logic-ийг hook болгох;
- database helper-ийг schema-тай нийцүүлэх;
- AI route дотор retrieval, prompt, provider, fallback logic-ийг функцээр салгах;
- dataset pipeline-ийг frontend-ээс тусдаа хадгалах.

Одоогийн кодын баазад сайжруулах нэг чухал хэсэг нь database helper болон schema-ийн нэршлийн нийцэл юм. Жишээлбэл, schema-д `completed_lessons`

хүснэгт байгаа боловч зарим helper logic `lesson_progress` нэр ашиглаж байвал production түвшинд алдаа үүснэ. Иймээс architecture review-ийн нэг зорилго нь code/schema consistency шалгах явдал юм.

4.14 Чанарын баталгаажуулалтын архитектур

Системийн чанарыг дараах түвшинд шалгана.

1. **Static quality:** TypeScript type-check, ESLint, Prettier.
2. **Build quality:** Next.js production build, LaTeX PDF build.
3. **Scenario quality:** хэрэглэгч бүртгүүлэх, курс хайх, free lesson үзэх, paid lesson lock, quiz, AI chat асуулт гэх мэт manual scenario.
4. **Data quality:** RAG dataset validation, duplicate check, citation-safe flag, category distribution.
5. **Security quality:** RLS policy review, admin role, user-owned data access.

Хүснэгт 4.6: Чанарын шалгалтын түвшин

Түвшин	Шалгах зүйл	Жишээ команд/арга
Code quality	TypeScript, lint, formatting	<code>npm run type-check</code> , <code>npm run lint</code>
Build	Frontend build, thesis PDF	<code>npm run build</code> , <code>latexmk -pdf main.tex</code>
Dataset	JSONL structure, duplicate, word count	Dataset validation scripts
Scenario	User journey, course, payment, chat	Manual walkthrough
Security	RLS policy, admin role, API key exposure	SQL policy review, environment review

4.15 Архитектурын шийдвэрийн бүртгэл

Системийн гол шийдвэрүүдийг архитектурын шийдвэрийн бүртгэл хэлбэрээр нэгтгэв.

Хүснэгт 4.7: Архитектурын шийдвэрүүд

ADR	Шийдвэр	Үндэслэл	Үр дагавар
-----	---------	----------	------------

ADR-01	Next.js App Router сонгох	Олон page, API route, client/server logic нэг framework-д хэрэгтэй	Frontend ба API нэг codebase-д төвлөрнө.
ADR-02	Supabase ашиглах	Auth, PostgreSQL, RLS, future pgvector нэг дор хэрэгтэй	Backend infrastructure хурдан бүрдэнэ.
ADR-03	AI-г support layer болгох	Хүний багшийн сургалтын контентыг орлуулахгүй байх	Chatbot нь context-aware нэмэлт туслах хэлбэртэй байна.
ADR-04	RAG dataset-ийг тусдаа pipeline болгох	Dataset чанар, validation, import logic frontend-ээс тусдаа байх ёстой	AI knowledge base-г independent байдлаар сайжруулна.
ADR-05	Provider abstraction хэрэглэх	Нэг LLM provider-д lock-in болохгүй байх	Олон AI үйлчилгээтэй ажиллах сонголттой болно.
ADR-06	Static course data + DB schema зэрэг бэлтгэх	Prototype хурдан гаргахын зэрэгцээ production migration-д бэлдэх	Дараагийн шатанд data migration хийх шаардлагатай.
ADR-07	RLS ашиглах	User-owned data хамгаалах шаардлагатай	SQL policy-г тогтмол шалгах хэрэгтэй.

4.16 Архитектурын view model

Системийн архитектурыг нэг зураг эсвэл нэг ER диаграмаар бүрэн тайлбарлах боломжгүй. Иймээс архитектурын view model ашиглан системийг олон өнцгөөс тайлбарлах нь зүйтэй. Энэ дипломын төслийн хувьд logical view, process view, development view, data view, deployment view, security view гэсэн зургаан өнцөг чухал.

Logical view нь хэрэглэгчийн функц, domain object, module хоорондын холбоог харуулна. Process view нь хэрэглэгчийн request систем дотор ямар дарааллаар дамжихыг тайлбарлана. Development view нь кодын хавтас, component,

hook, API route, dataset builder зэрэг implementation нэгжүүдийг харуулна. Data view нь Supabase schema, RLS, relation, dataset import logic-ийг харуулна. Deployment view нь browser, Next.js runtime, Supabase service, LLM provider, local dataset хоорондын байрлалыг тодорхойлно. Security view нь user-owned data, admin access, API key хамгаалалтыг авч үзнэ.

Хүснэгт 4.8: Архитектурын view model

View	Асуулт	Энэ төсөлд үзүүлэх зүйл
Logical view	Систем ямар domain объекттой вэ?	User, Course, Lesson, Progress, Chat, KnowledgeBase
Process view	Хэрэглэгчийн request хэрхэн дамжих вэ?	Course access, chat request, progress update дараалал
Development view	Код хаана байрлаж, хэрхэн хуваагдсан бэ?	App Router, components, hooks, lib, API route, dataset builder
Data view	Өгөгдөл ямар бүтэцтэй вэ?	Supabase schema, ER relation, RLS policy, dataset import
Deployment view	Runtime бүр хаана байрлах вэ?	Browser, Next.js, Supabase, LLM provider, local JSONL
Security view	Эрх ба хамгаалалт яаж хэрэгжих вэ?	Auth, RLS, admin policy, server-side provider key

4.17 Data lifecycle архитектур

Сургалтын платформд өгөгдөл нэг удаа үүсээд зогсохгүй, хэрэглэгчийн interaction бүрээр өөрчлөгдөж байдаг. Course болон lesson нь контентын өгөгдөл, purchase нь access өгөгдөл, progress нь learning analytics-ийн эх сурвалж, chat message нь AI interaction history, knowledge base нь AI context source болно. Иймээс өгөгдлийн lifecycle-ийг архитектурын түвшинд тодорхойлох шаардлагатай.

Course data эхэндээ static файл болон Supabase schema хоёрын хооронд coexistence төлөвтэй байна. Энэ нь prototype хурдан гаргах давуу талтай боловч production үед data migration хийх шаардлагатай. Purchase data checkout flow-оор үүсэж, lesson access-д шууд нөлөөлнө. Progress data lesson complete action-аар үүсэж dashboard болон course detail page-д буцаж харагдана. Chat

message болон source metadata нь AI response-ийн transparency-г нэмэгдүүлнэ. Knowledge base data нь dataset builder-ээс Supabase seed хэлбэрээр орж ирэх боломжтой.

ХҮСНЭГТ 4.9: Өгөгдлийн lifecycle

Өгөгдөл	Үүсэх эх үүсвэр	Ашиглагдах газар	Хяналт
Course, lesson	Static data, admin CRUD future, seed data	Catalog, detail, lesson player	Admin
Purchase	Checkout flow, payment future	Access control, dashboard	User + admin
Progress	Lesson complete, quiz future	Dashboard, course progress	User-owned
Chat message	Chat UI болон API response	Chat history, quality review	User-owned
Knowledge base	RAG builder, Supabase seed, admin edit	AI retrieval context	Admin curated
Audio file	Upload future	Analysis, feedback, storage	User-owned

4.18 Failure mode ба resilience

AI-тэй сургалтын платформ нь зөвхөн happy path дээр ажиллах ёсгүй. Provider unavailable, dataset file missing, Supabase session expired, RLS policy deny, network latency, invalid YouTube id, payment confirmation failure зэрэг алдаа гарч болно. Architecture нь эдгээрийг user experience сүйтгэхгүйгээр зохицуулах ёстой.

ХҮСНЭГТ 4.10: Failure mode ба resilience strategy

Failure mode	Нөлөөлөл	Архитектурын хариу арга
LLM provider unavailable	Chat хариу өгөхгүй, хэрэглэгч хүлээнэ	Fallback answer, provider priority, retry limit
Dataset унших алдаа	RAG context сонгогдохгүй	Local fallback message, log, dataset validation

Auth session expired	Dashboard, checkout, progress алдаа өгнө	Auth hook state refresh, login redirect
RLS deny	User data унших эсвэл бичих боломжгүй	SQL policy review, clear error message
Course data mismatch	Static data ба DB зөрөх	Migration plan, source of truth тодорхойлох
Payment webhook missing	Paid access баталгаа-жихгүй	Payment state table, manual admin review future
YouTube metadata failure	Duration харагдахгүй	Optional metadata, UI fallback

Resilience-ийн хувьд хамгийн чухал зарчим нь graceful degradation юм. AI provider ажиллахгүй үед бүх платформ унах ёсгүй. Chatbot хариу өгөхгүй байлаа ч course catalog, lesson viewing, progress dashboard ажиллах ёстой. Үүний адил YouTube duration авах API амжилтгүй болсон ч lesson itself харагдах боломжтой байх хэрэгтэй.

4.19 Performance architecture

Performance нь сургалтын платформд хоёр түвшинд чухал. Нэгдүгээрт, хэрэглэгчийн UI хурдан нээгдэх ёстой. Хоёрдугаарт, AI chat request хэт удаан хүлээлгэхгүй байх ёстой. Frontend талд component rendering, image/media loading, route splitting, animation cost зэрэг хүчин зүйлс нөлөөлнө. Backend/API талд dataset унших, retrieval scoring, provider latency, prompt size зэрэг хүчин зүйлс нөлөөлнө.

4.20 Observability ба operation

Production системд алдаа хаана гарч байгааг мэдэхгүй бол засвар хийхэд хүндрэлтэй. Дипломын MVP-д full observability хэрэгжээгүй боловч архитектурын түвшинд log, metric, audit, validation report гэсэн ойлголтуудыг төлөвлөх хэрэгтэй. Ялангуяа AI chat route дээр provider latency, selected source count, fallback rate, user feedback зэрэг metric ирээдүйд чанарын үнэлгээнд хэрэг болно.

ХҮСНЭГТ 4.11: Performance bottleneck ба optimization

Bottleneck	Шалтгаан	Optimization
Initial page load	Том component, media, animation	Component split, responsive media, lightweight section
Course catalog filter	Course count өсөх	Server-side search, index, pagination
Chat dataset load	JSONL-г request бүр унших	Memory cache, database index future
Retrieval scoring	Entry count өсөх	Embedding index, hybrid retrieval
Provider latency	External LLM response удаан	Streaming future, fallback, provider routing
Dashboard query	Олон progress/purchase row	Indexed user id, summarized progress view

ХҮСНЭГТ 4.12: Observability төлөвлөгөө

Хэмжих зүйл	Яагаад хэрэгтэй вэ?	Ирээдүйн хэрэг-жилт
API latency	Chat болон metadata route удааширч буй эсэх	Request log, timing metric
Provider success rate	LLM provider тогтвортой эсэх	Provider result status хадгалах
Fallback rate	RAG/LLM чанар хангалттай эсэх	Chat response metadata analytics
Source usage	Ямар knowledge base entry их ашиглагдаж буй	Source id count, category distribution
Progress event	Суралцагч хичээл дуусгаж буй эсэх	Completion event log
Dataset validation	Knowledge base чанар буурч буй эсэх	Validation report versioning

4.21 Architecture evaluation

Архитектурын үнэлгээг ISO чанарын шинжүүдтэй ойролцоо байдлаар авч үзвэл usability, reliability, security, maintainability, scalability, portability гэсэн чиглэлүүд чухал. Энэ төслийн хувьд usability нь course flow ойлгомжтой

байх, reliability нь core workflow тасрахгүй байх, security нь user-owned data хамгаалах, maintainability нь component/hook/API separation, scalability нь dataset болон course content өсөхөд бэлтгэлтэй байх, portability нь provider abstraction болон Supabase-based deployment ашиглахтай холбоотой.

Хүснэгт 4.13: Архитектурын чанарын үнэлгээ

Чанарын шинж	Одоогийн шийдэл	Сайжруулах чиглэл
Usability	Course catalog, lesson player, chat drawer	User testing, accessibility review
Reliability	Fallback answer, static data fallback	Automated tests, provider retry strategy
Security	Supabase Auth, RLS, admin policy	Policy test, API key audit, payment webhook validation
Maintainability	App route, component, hook, lib separation	Shared types, service layer consistency
Scalability	DB schema, dataset pipeline, provider abstraction	Vector search, server-side pagination, cache
Portability	Web stack, environment-based provider	Deployment documentation, container option future

4.22 Migration ба хөгжүүлэлтийн roadmap

Одоогийн архитектур нь MVP болон future production-ийн дунд шатанд байна. Иймээс дараагийн хөгжүүлэлтийн roadmap тодорхой байх хэрэгтэй. Нэгдүгээр шатанд static course data-г database-driven болгоно. Хоёрдугаар шатанд payment simulation-г real gateway болон webhook-той холбоно. Гуравдугаар шатанд RAG retrieval-ийг embedding болон vector index рүү шилжүүлнэ. Дөрөвдүгээр шатанд admin dashboard-аар content CRUD, dataset import, user management хийх боломж нэмнэ. Тавдугаар шатанд audio upload болон analysis feedback module хэрэгжүүлнэ.

Хүснэгт 4.14: Архитектурын roadmap

Шат	Гол ажил	Архитектурын өөрчлөлт	Эрсдэл
-----	----------	-----------------------	--------

1	Course data migration	Static data source-ийг Supabase content source болгох	Data consistency
2	Payment integration	Checkout webhook, payment state, receipt	Security, fraud
3	Vector RAG	Embedding pipeline, pgvector index, hybrid search	Cost, quality
4	Admin CMS	Course, lesson, knowledge base CRUD	Authorization
5	Audio analysis	Upload, feature extraction, feedback API	Compute cost
6	Analytics	Progress, quiz, chat usefulness dashboard	Privacy

4.23 C4 загварын тайлбар

Системийн архитектурыг C4 загварын context, container, component түвшинд тайлбарлаж болно. Context түвшинд платформ нь зочин хэрэглэгч, суралцагч, админ, AI provider, Supabase service, YouTube content source зэрэг гаднын оролцогчтой харилцана. Container түвшинд browser client, Next.js application, Supabase PostgreSQL/Auth, RAG dataset builder, LLM provider гэсэн үндсэн runtime болон supporting artifact ялгарна. Component түвшинд frontend route, reusable component, auth hook, purchase hook, chat API, retrieval module, database policy зэрэг жижиг нэгжүүд оршино.

Хүснэгт 4.15: C4 архитектурын тайлбар

Түвшин	Системийн нэгж	Тайлбар
Context	Суралцагч, админ, AI provider, Supabase	Платформ ямар external actor болон service-тэй холбогдохыг харуулна.
Container	Browser, Next.js app, Supabase, RAG builder	Runtime болон өгөгдлийн контейнеруудыг ялгана.

Component	Route, component, hook, API route, policy	Кодын зохион байгуулалт ба responsibility-г харуулна.
Code	TypeScript component, SQL policy, Python script	Implementation түвшний artifact-уудыг илэрхийлнэ.

С4 загвараар тайлбарлахын давуу тал нь дипломын тайланд оролцогч талуудын ойлгох түвшин бүрт тохирсон дүрслэл өгөх боломжтой. Удирдагч эсвэл шаллагч context болон container түвшинд системийн ерөнхий санааг ойлгоно. Харин хөгжүүлэгч component болон code түвшинд засвар, өргөтгөл хийх боломжтой.

4.24 Security zone ба trust boundary

Security architecture-г илүү нарийвчлан авч үзвэл системд public zone, authenticated user zone, admin zone, server-only zone, external provider zone гэсэн trust boundary байна. Public zone-д зочин хэрэглэгч course preview, landing page, public lesson metadata харна. Authenticated zone-д хэрэглэгч өөрийн dashboard, purchase, progress, chat history харна. Admin zone-д course, lesson, knowledge base, user management хийгдэнэ. Server-only zone-д API key, provider selection, prompt assembly, dataset path байрлана. External provider zone-д LLM service болон YouTube metadata source орно.

ХҮСНЭГТ 4.16: Security zone ба trust boundary

Zone	Зөвшөөрөгдсөн үйлдэл	Хамгаалалтын арга
Public zone	Landing, course preview, public knowledge select	Public read policy, no sensitive key
Authenticated zone	Purchase, progress, chat history	Auth session, owner-based RLS
Admin zone	Course, lesson, knowledge base manage	Admin role, helper function, policy
Server-only zone	Prompt assembly, provider key, fallback	API route, environment variable

External provider zone	LLM response, YouTube metadata	Minimal context, timeout, fallback
------------------------	--------------------------------	------------------------------------

Trust boundary тодорхой байхгүй бол frontend дээр харагдах UI guard-г security гэж эндүүрэх эрсдэлтэй. Жишээлбэл paid lesson button-г нуух нь хэрэглэгчийн туршлагын guard болохоос database security биш. Бодит хамгаалалт нь purchase state, RLS policy, server-side validation-аар баталгаажих ёстой.

4.25 Configuration ба environment архитектур

Configuration architecture нь development, demo, production орчны ялгааг тодорхойлно. Development орчинд local Supabase project эсвэл remote dev project, local Ollama, generated dataset file ашиглаж болно. Demo орчинд зарим provider key байхгүй байж болох тул fallback answer хэрэгтэй. Production орчинд key rotation, logging, payment webhook, database backup, vector index migration зэрэг илүү хатуу шаардлага тавигдана.

ХҮСНЭГТ 4.17: Environment architecture

Орчин	Гол тохиргоо	Архитектурын анхаарах зүйл
Local development	Local env, dev Supabase, local dataset	Хурдан туршилт хийх, key-г commit хийхгүй байх
Demo	Hosted frontend, limited provider key	Fallback logic, sample data, stable presentation
Production future	Real payment, provider routing, backup	Security audit, monitoring, migration control
Research artifact	Dataset reports, thesis PDF, diagrams	Reproducibility, versioned documentation

4.26 Decision trade-off шинжилгээ

Архитектурын шийдвэр бүр trade-off-той. Next.js сонгосноор frontend болон API нэг codebase-д төвлөрөх боловч framework-specific dependency үүснэ. Supabase сонгосноор Auth, PostgreSQL, RLS хурдан бүрдэх боловч vendor dependency

нэмэгдэнэ. Local JSONL RAG сонгосноор MVP хурдан хэрэгжих боловч dataset томроход performance асуудал үүснэ. Provider abstraction сонгосноор уян хатан байдал нэмэгдэх боловч configuration complexity өснө.

Хүснэгт 4.18: Архитектурын trade-off шинжилгээ

Шийдвэр	Давуу тал	Сул тал ба нөхөх арга
Next.js App Router	Route, API, layout нэг framework-д төвлөрнө	Framework update дагах шаардлагатай
Supabase	Auth, Postgres, RLS хурдан бүрдэнэ	Vendor dependency, migration plan хэрэгтэй
Static + DB coexistence	Prototype хурдан, production бэлтгэлтэй	Data duplication, source of truth тодорхойлох хэрэгтэй
Local JSONL RAG	Ойлгомжтой, хурдан demo хийх боломжтой	Dataset өсөхөд vector DB рүү шилжинэ
Provider abstraction	Нэг provider-ээс хамаарахгүй	Error handling, config management нэмэгдэнэ
Future audio schema	Өргөтгөлийн суурь бэлэн	Одоогийн MVP-д зарим table ашиглагдахгүй

4.27 System integration scenario

Системийн бүрэлдэхүүнүүд тусдаа зөв ажиллах нь хангалтгүй. Integration scenario дээр тэдгээр нь хамтдаа хэрэглэгчийн зорилгыг биелүүлэх ёстой. Жишээлбэл суралцагч course сонгох, login хийх, paid lesson авах, lesson үзэх, quiz бөглөх, AI-аас асуух, progress хадгалах, dashboard дээр буцаж харах гэсэн урсгал нь frontend, hook, API, database, RAG, provider бүх давхаргыг хамарна.

1. Зочин хэрэглэгч course catalog нээж, FL Studio эхлэгчийн course сонгоно.
2. Төлбөртэй lesson дээр access шаардлагатай тул login эсвэл checkout flow руу шилжинэ.

3. Хэрэглэгч худалдан авалт хийсний дараа purchase state database-д хадгалагдана.
4. LessonPlayer purchase state-г ашиглан locked lesson-г нээж харуулна.
5. Суралцагч video үзээд self-check бөглөж completion event үүсгэнэ.
6. Хичээлтэй холбоотой асуулт гарвал ChatDrawer-аар message илгээнэ.
7. Chat API RAG context сонгож, provider-оор хариу үүсгээд sources буцаана.
8. Dashboard нь progress болон purchased course мэдээллийг нэгтгэн харуулна.

ХҮСНЭГТ 4.19: Integration scenario дахь давхаргын оролцоо

Давхарга	Оролцох үе	Хариуцах зүйл
Presentation	Catalog, lesson, dashboard, chat UI	Хэрэглэгчийн харах урсгал
Application	Auth state, purchase state, lesson completion	UI logic ба state coordination
API/AI	Chat request, provider call, fallback	Server-side intelligence
Data	Purchase, progress, chat, knowledge base	Persistence ба access control
Dataset	RAG context, source metadata	AI хариултын grounding

4.28 Архитектурын баримтжуулалтын ач холбогдол

Дипломын төсөлд architecture documentation нь зөвхөн зураг нэмэх зориулалттай биш. Энэ нь систем яагаад ийм бүтэцтэй болсон, ямар trade-off хийсэн, ямар хэсэг MVP, ямар хэсэг future-ready болохыг тайлбарладаг. Ялангуяа AI-тэй системд model, dataset, prompt, provider, fallback, source metadata зэрэг зүйлсийг кодоос гадна архитектурын баримтаар тайлбарлах шаардлагатай.

Баримтжуулалт нь дараагийн хөгжүүлэгч, шалгагч, удирдагч, хэрэглэгчийн төлөөлөлд өөр өөр ашигтай. Хөгжүүлэгч component ба data flow ойлгоно. Шалгагч шаардлага ба хэрэгжилтийн уялдааг шалгана. Удирдагч судалгааны contribution болон system artifact-ийн холбоог харна. Хэрэглэгчийн төлөөлөл platform workflow болон AI туслахын үүргийг ойлгоно.

Хүснэгт 4.20: Архитектурын баримтжуулалтын хэрэглэгч

Оролцогч	Ямар мэдээлэл хэрэгтэй вэ?	Баримтжуулалтын хэсэг
Хөгжүүлэгч	Module boundary, API, database relation	Component, ER, sequence, ADR
Шалгагч	Requirement coverage, quality attribute	Traceability, evaluation, risk table
Удирдагч	Судалгааны үндэслэл, contribution	Methodology, technology, architecture summary
Админ хэрэглэгч	Course, lesson, knowledge base удирдлага	Admin workflow, security policy
Суралцагчийн төлөөлөл	Learning flow, AI туслахын үүрэг	Use case, activity flow, UX notes

4.29 Architecture review checklist

Архитектурын загварыг зөвхөн диаграмын бүрэн байдлаар бус, системийн шаардлага, хэрэгжилтийн нотолгоо, цаашдын өргөтгөх боломжтой холбон үнэлэх шаардлагатай. Иймээс review checklist нь дипломын тайлангийн дөрвөн бүлгийг хооронд нь холбох үүрэгтэй. Нэгдүгээр бүлгийн methodology судалгаа нь ямар асуудлыг ямар аргаар шийдэхийг тодорхойлсон. Хоёрдугаар бүлгийн технологийн судалгаа нь сонгосон шийдлийн онолын үндэслэлийг гаргасан. Гуравдугаар бүлэгт код, dataset, database migration, frontend component, API route зэрэг бодит хэрэгжилтийг тайлбарласан. Харин энэ бүлгийн architecture review нь эдгээр бүх нотолгоо нэг системийн бүтэц болж чадсан эсэхийг шалгана.

Review хийхдээ “feature байгаа эсэх” гэсэн энгийн шалгалтаар зогсохгүй, тухайн feature ямар давхаргад байрлаж, ямар өгөгдөл ашиглаж, ямар security boundary давж, ямар failure mode-той болохыг тодруулна. Жишээлбэл AI chat ажиллаж байгаа нь хангалттай биш. Chat request server-only API route-оор дамжиж байгаа эсэх, prompt-д орох context эх сурвалжтай эсэх, provider алдаа гарвал fallback хариу байгаа эсэх, хэрэглэгчийн chat history owner-based policy-д захирагдаж байгаа эсэхийг хамтад нь шалгах хэрэгтэй.

Хүснэгт 4.21: Архитектурын review checklist

Шалгах талбар	Шалгах асуулт	Нотолгоо ба шийд- вэр
User flow completeness	Зочин, бүртгэлтэй суралцагч, админ хэрэглэгчийн үндсэн урсгал тасралтгүй юу?	Use case, activity diagram, route responsibility table
Layer responsibility	UI, API, AI, database, dataset үүрэг давхцаагүй юу?	Layered architecture, component table, sequence diagram
Data ownership	Хэрэглэгчийн progress, purchase, chat өгөгдөл эзэмшигчтэйгээ холбогдсон уу?	ER diagram, RLS policy, domain class model
AI grounding	AI хариулт сургалтын эх сурвалж, metadata, fallback logic-той юу?	RAG pipeline, knowledge base table, chat API algorithm
Security boundary	Provider key, admin operation, paid access frontend дээр ил үлдээгүй юу?	Server-only zone, Auth guard, RLS security matrix
Scalability path	Dataset, course, payment, audio analysis өргөжихөд schema ба module бэлэн үү?	Roadmap table, future audio schema, migration plan
Maintainability	Component, hook, API route, dataset builder тусдаа өөрчлөгдөх боломжтой юу?	File structure, ADR, quality attribute evaluation
Academic traceability	Судалгааны зорилго, технологийн үндэслэл, хэрэгжилтийн нотолгоо холбогдсон уу?	Requirement traceability, chapter conclusions, references

Энэхүү checklist-ийг хэрэглэснээр architecture document нь static зураг биш, харин төслийн чанарыг шалгах идэвхтэй хэрэгсэл болно. Мөн дараагийн хөгжүүлэлтийн үед шинэ feature нэмэгдэх бүрд тухайн feature аль давхаргад харьяалагдах, ямар өгөгдөл үүсгэх, ямар API эсвэл provider-тай холбогдох, ямар security policy шаардлагатай болохыг нэг ижил зарчмаар үнэлэх

боломж бүрдэнэ.

4.30 Бүлгийн дүгнэлт

Дөрөвдүгээр бүлгээр FL Studio сургалтын платформын архитектур ба системийн бүтцийг нэгтгэн тайлбарлав. Систем нь presentation, application, API/AI support, data, content/dataset гэсэн давхаргат бүтэцтэй. Use case, activity, class, ER, deployment диаграмуудаар хэрэглэгчийн урсгал, домайн объект, өгөгдлийн холбоо, AI туслахын дараалал, deployment node-уудын уялдааг харууллаа.

Архитектурын гол дүгнэлт нь AI туслахыг үндсэн сургалтын контентын оронд тавихгүй, харин RAG мэдлэгийн сан, provider abstraction, fallback logic бүхий туслах давхарга байдлаар тусгаарлах нь системийн найдвартай байдал, академик үнэн зөв, өргөтгөх боломжийг нэмэгдүүлнэ. Мөн Supabase RLS, relational schema, Next.js component architecture, dataset pipeline зэрэг шийдлүүд нь цаашид course management, embedding retrieval, payment integration, audio analysis зэрэг өргөтгөлийг хэрэгжүүлэх суурь болж байна.

Хавсралтаа энд бичнэ.

Ном зүй

- [1] Ashish Vaswani ба бусад. *Attention Is All You Need*. 2017. arXiv: 1706.03762 [cs.CL]. URL: <https://arxiv.org/abs/1706.03762>.
- [2] Patrick Lewis ба бусад. “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”. Дотор: *Advances in Neural Information Processing Systems* (2020). URL: <https://arxiv.org/abs/2005.11401>.
- [3] Vladimir Karpukhin ба бусад. *Dense Passage Retrieval for Open-Domain Question Answering*. 2020. arXiv: 2004.04906 [cs.CL]. URL: <https://arxiv.org/abs/2004.04906>.
- [4] Tom B. Brown ба бусад. *Language Models are Few-Shot Learners*. 2020. arXiv: 2005.14165 [cs.CL]. URL: <https://arxiv.org/abs/2005.14165>.
- [5] Hugo Touvron ба бусад. *LLaMA: Open and Efficient Foundation Language Models*. 2023. arXiv: 2302.13971 [cs.CL]. URL: <https://arxiv.org/abs/2302.13971>.
- [6] Gautier Izacard, Edouard Grave, ба бусад. *Atlas: Few-shot Learning with Retrieval Augmented Language Models*. 2022. arXiv: 2208.03299 [cs.CL]. URL: <https://arxiv.org/abs/2208.03299>.
- [7] Prafulla Dhariwal ба бусад. *Jukebox: A Generative Model for Music*. 2020. arXiv: 2005.00341 [eess.AS]. URL: <https://arxiv.org/abs/2005.00341>.
- [8] Zalan Borsos ба бусад. *AudioLM: a Language Modeling Approach to Audio Generation*. 2022. arXiv: 2209.03143 [cs.SD]. URL: <https://arxiv.org/abs/2209.03143>.
- [9] Jade Copet ба бусад. “Simple and Controllable Music Generation”. Дотор: *arXiv preprint arXiv:2306.05284* (2024). URL: <https://arxiv.org/abs/2306.05284>.
- [10] Andrea Agostinelli ба бусад. “MusicLM: Generating Music From Text”. Дотор: *arXiv preprint arXiv:2301.11325* (2023). URL: <https://arxiv.org/abs/2301.11325>.

- [11] Alexei Baevski ба бусад. *wav2vec 2.0: A Framework for Self-Supervised Learning of Speech Representations*. 2020. arXiv: 2006.11477 [cs.CL]. URL: <https://arxiv.org/abs/2006.11477>.
- [12] Alec Radford ба бусад. *Robust Speech Recognition via Large-Scale Weak Supervision*. 2022. arXiv: 2212.04356 [eess.AS]. URL: <https://arxiv.org/abs/2212.04356>.
- [13] Magenta. *Magenta: Make Music and Art Using Machine Learning*. 2026. URL: <https://magenta.tensorflow.org/> (үзсэн огноо 04/28/2026).
- [14] Suno. *Suno: The AI Music App for Every Creator*. 2026. URL: <https://suno.com/1/ai-music-app> (үзсэн огноо 04/28/2026).
- [15] AIVA Technologies. *AIVA, the AI Music Generation Assistant*. 2026. URL: <https://www.aiva.ai/t/ai-music-generation/> (үзсэн огноо 04/28/2026).
- [16] SOUNDRAW. *SOUNDRAW: AI Music Generator*. 2026. URL: <https://soundraw.io/> (үзсэн огноо 04/28/2026).
- [17] Winston W. Royce. “Managing the Development of Large Software Systems”. Дотор: *Proceedings of IEEE WESCON*. 1970.
- [18] Barry W. Boehm. “A Spiral Model of Software Development and Enhancement”. Дотор: *Computer* 21.5 (1988), pp. 61–72.
- [19] Beck, Kent and Beedle, Mike and van Bennekum, Arie and Cockburn, Alistair and Cunningham, Ward and Fowler, Martin and Grenning, James and Highsmith, Jim and Hunt, Andrew and Jeffries, Ron and Kern, Jon and Marick, Brian and Martin, Robert C. and Mellor, Steve and Schwaber, Ken and Sutherland, Jeff and Thomas, Dave. *Manifesto for Agile Software Development*. 2001. URL: <https://agilemanifesto.org/> (үзсэн огноо 04/30/2026).
- [20] Schwaber, Ken and Sutherland, Jeff. *The Scrum Guide*. 2020. URL: <https://scrumguides.org/scrum-guide.html> (үзсэн огноо 04/30/2026).
- [21] David J. Anderson. *Kanban: Successful Evolutionary Change for Your Technology Business*. Blue Hole Press, 2010.
- [22] Jeff Gothelf and Josh Seiden. *Lean UX: Applying Lean Principles to Improve User Experience*. O'Reilly Media, 2013.
- [23] Alan R. Hevner ба бусад. “Design Science in Information Systems Research”. Дотор: *MIS Quarterly* 28.1 (2004), pp. 75–105.
- [24] Vercel. *Next.js Documentation*. 2026. URL: <https://nextjs.org/docs> (үзсэн огноо 04/28/2026).

- [25] React Team. *React Documentation: Learn*. 2026. URL: <https://react.dev/learn> (үзсэн огноо 04/28/2026).
- [26] Tailwind Labs. *Tailwind CSS Documentation*. 2026. URL: <https://tailwindcss.com/docs> (үзсэн огноо 04/28/2026).
- [27] Supabase. *Supabase Documentation*. 2026. URL: <https://supabase.com/docs> (үзсэн огноо 04/28/2026).
- [28] Nils Reimers and Iryna Gurevych. *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks*. 2019. arXiv: 1908.10084 [cs.CL]. URL: <https://arxiv.org/abs/1908.10084>.
- [29] Tianyu Gao, Xingcheng Yao, and Danqi Chen. *SimCSE: Simple Contrastive Learning of Sentence Embeddings*. 2022. arXiv: 2104.08821 [cs.CL]. URL: <https://arxiv.org/abs/2104.08821>.
- [30] Omar Khattab and Matei Zaharia. *ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT*. 2020. arXiv: 2004.12832 [cs.IR]. URL: <https://arxiv.org/abs/2004.12832>.
- [31] Yu. A. Malkov and D. A. Yashunin. *Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs*. 2018. arXiv: 1603.09320 [cs.DS]. URL: <https://arxiv.org/abs/1603.09320>.
- [32] Keunwoo Choi ба бусад. *Convolutional Recurrent Neural Networks for Music Classification*. 2016. arXiv: 1609.04243 [cs.NE]. URL: <https://arxiv.org/abs/1609.04243>.
- [33] Qiuqiang Kong ба бусад. *PANNs: Large-Scale Pretrained Audio Neural Networks for Audio Pattern Recognition*. 2020. arXiv: 1912.10211 [cs.SD]. URL: <https://arxiv.org/abs/1912.10211>.
- [34] Daniel Stoller, Sebastian Ewert, and Simon Dixon. *Wave-U-Net: A Multi-Scale Neural Network for End-to-End Audio Source Separation*. 2018. arXiv: 1806.03185 [cs.SD]. URL: <https://arxiv.org/abs/1806.03185>.
- [35] Alexandre Defossez ба бусад. *Music Source Separation in the Waveform Domain*. 2021. arXiv: 1911.13254 [cs.SD]. URL: <https://arxiv.org/abs/1911.13254>.
- [36] Simon Rouard, Francisco Massa, and Alexandre Defossez. *Hybrid Transformers for Music Source Separation*. 2022. arXiv: 2211.08553 [eess.AS]. URL: <https://arxiv.org/abs/2211.08553>.

- [37] Po-Sen Huang ба бусад. *MuLan: A Joint Embedding of Music Audio and Natural Language*. 2022. arXiv: 2208.12415 [eess.AS]. URL: <https://arxiv.org/abs/2208.12415>.